

Security Focused API-Centric Live Scoring Application

COMP 8045 – 9 Credit – Major Project 1 – Network Security and Admin
Option

Robert Purdey – A00860008
7-29-2019

Table of Contents

1 Introduction	5
1.1 Student Background	5
1.1.1 Education	5
1.1.2 Work Experience	5
1.2 Nature of the Project	5
1.2.1 Problems to be Solved	5
1.2.2 Goals/Objectives	5
2 Body	6
2.1 Project Background	6
2.1.1 Description	6
2.1.2 Problem Statement	7
2.2 Alternative Solutions	7
2.2.1 Stack Choice	7
2.2.2 Different Protocols	7
2.2.3 Hosting Choices	7
2.3 Chosen Solution	7
2.3.1 Stack Choice	7
2.3.2 Protocols	8
2.4 Feasibility Assessment Report	8
2.4.1 14.1 Secure Data Transmission	9
2.4.2 14.2 Data Restrictions	10
2.4.3 14.3 Live Game Data Viewing	10
2.4.4 Deployment	11
2.5 API Server Design	13
2.5.1 Class Diagrams	14
2.5.2 Sequence Diagrams	31
2.5.3 API Calls	35
2.6 Broadcast Server Design	45
2.6.1 Classes	45
2.6.2 Sequence Diagrams	47
2.6.3 Flowcharts	49
2.7 UI Design	49
2.7.1 Screens	49
2.7.2 Use Cases	67
2.7.3 Encryption	76
2.7.4 UI Code Structure	76
2.8 Testing	78
2.8.1 Android App – Client	78

2.8.2 ASP.NET Web API – API server	79
2.8.3 ASP.NET Core App – Score Broadcaster Server	80
2.9 Implications	90
2.9.1 Performance	90
2.9.2 Functionality	90
2.9.3 Portability	90
2.9.4 Maintainability	90
2.10 New Technologies	91
2.11 Future Enhancements	91
2.11.1 Automatic Course Creation	91
2.11.2 Detailed Statistics	91
2.11.3 More Communication between the Client and Broadcast Server	91
2.11.4 Permissions Set by Users	92
2.11.5 Different Game Types	92
2.12 Timeline/Milestones	92
2.13 Installation Manual	94
2.13.1 Android Application	94
2.13.2 API Server	95
2.13.3 Broadcast Server	98
2.14 User Manual	100
2.14.1 Android Application	100
3 Conclusion	126
3.1 Enhancements to Network and Security Expertise	126
3.1.1 Networking	126
3.1.2 Security	127
3.2 Lessons Learned Above and Beyond the Classroom Teachings	127
3.2.1 Combined Different Technologies into a Single Solution	127
3.2.2 Hosting Servers	127
3.2.3 Use of MVVM pattern for Android	128
3.2.4 Setup OAuth Authentication	128
4 References	128

1 Introduction

1.1 Student Background

I am a software developer planning to graduate October 2019 with a Bachelor of Technology that focuses on network security. During my education and 1+ years' work experience, I gained strong knowledge of object-oriented programming languages, standard coding practices and teamwork.

My experience includes network security/communication and web development.

1.1.1 Education

Bachelor of Technology, Network Security Administration (Expected) Oct 2019

British Columbia Institute of Technology – Burnaby, BC

88% GPA – Current GPA

2-Year Diploma, Computer Systems Technology Dec 2014

British Columbia Institute of Technology – Burnaby, BC

89% GPA – Earned Distinction

1.1.2 Work Experience

Junior Software Developer Mar 2015 - Aug 2016

Pelesys Learning Systems – Richmond, BC

- Worked in a team of 3-4 alongside a QA team to produce well-tested code
- Wrote unit tests for features being developed
- Proposed/implemented bug fixes based on failing unit tests and smoke tests
- Implemented small features and helped others with larger features
- Fixed issues with the automated build process
- Setup database update scripts to go along with version releases
- Managed time effectively to meet weekly/monthly deadlines

1.2 Nature of the Project

The project focuses on developing a set of applications that communicate securely using HTTPS and TCP protocols. The project consists of 3 applications:

1. (Client) Android App – Users use this to interact with the servers.
2. (Server) ASP.NET Web API – Executes requests from clients and interacts stores/retrieves from the database.
3. (Server) ASP.NET Web Core Console App – Manages the communication of game scores to/from clients announcing and spectating games.

1.2.1 Problems to be Solved

- Using AES/RSA encryption together so data can be sent securely
- Broadcasting data from one client to many
- Storing sensitive data securely
- Hosting a project in the cloud

1.2.2 Goals/Objectives

- Display knowledge of courses taken at BCIT
- Learn how to develop a full-stack project

- Better understand the deployment process of applications into the cloud
- Create an application that connects users giving them an experience they can share

2 Body

2.1 Project Background

2.1.1 Description

This project focuses on implementing data communication between a client and server as well as data storage securely. This will involve:

- data being encrypted before sending and decrypted after receiving (for API and TCP communication)
- ensuring API calls are being made from valid clients
- storing hashed versions of passwords in the database
- permission-based application functionality (group data access restrictions)

The communication between the client and server is implemented using API calls and TCP connections. The API calls will handle requests made to store/retrieve data that changes less frequent, such as a user creating an account or creating a friend group. TCP connections will be used to transmit live game data to users that are viewing the game so that they can get constant updates.

This project is broken down into the following sub-projects:

- API-Centric Server
- Broadcast Server
- Client Application

2.1.1.1 API-Centric Server

The server is developed using C# and will use API calls to communicate with the client applications. All business logic will be contained on the server making it easier to implement clients for different mediums (android, iPhone, etc.) as they will only be required to implement the front-end portion of the project.

A heavy focus is on the implementation of a data encryption layer and verification layer. The encryption layer is responsible for encrypting/decrypting data using a combination of RSA and AES encryption. The verification layer is responsible for ensuring an API call is being made by a valid client.

2.1.1.2 Broadcast Server

The broadcast server handles sending game updates from the client announcing to the clients spectating. Clients make requests to the server to join as either an Announcer (a client that sends game updates) or Spectator (a client that receives games updates). Client requests are validated before they can communicate with the server.

The server is multithreaded allowing client requests to be handled asynchronous:

- 1 thread listening for TCP connection requests
- A new thread for every connection being validated
- A new thread for every announcer connected

- A new thread for every game update sent to spectators

2.1.1.3 Client Application

The client is developed as an Android application. The implementation includes development of a front-end that allows a user to create an account, join private groups and score disc golf games amongst these private groups. Furthermore, the clients can announce or spectate games live. This will allow friends to follow along with the action. The application will be password protected and data being sent to the server will be encrypted using RSA/AES.

2.1.2 Problem Statement

The problem with disc golf scoring applications that I investigated is most don't allow sharing data and the ones that did, had no security documentation which makes them untrustworthy for storing personal data such as user passwords. My project will be a great solution for disc golfers who want to be able to share disc golf data amongst a group of friends without it being accessible by unwanted users.

Another problem area for all scoring applications is that they do not allow live updates. The common scoring algorithm is that games are only updated after being completed. Rather than using this scoring algorithm, I will provide updates to the game on a per hole basis. This will enable my application to display live game results such as leaderboards and hole-by-hole results to other players, so they can follow along, on or off the course. Overall, providing live data gives users a more enriched experience as they can always be kept in the action.

2.2 Alternative Solutions

2.2.1 Stack Choice

The project that I developed could be implemented using a wide variety of technologies. For example, I could have used MongoDB instead of SQL Server for the database. For the API server, I could have used ASP.NET Core or gone serverless with AWS functions instead of using ASP.NET Web API. Because the goal of this project was less about choosing the newest technologies and more about making technologies work together, these alternative solutions were not investigated in much detail.

2.2.2 Different Protocols

I decided to use the TCP protocol to enable the broadcasting of data from one client to many. UDP could have been used as well but that would have required each request to be validated whereas the TCP requires one.

2.2.3 Hosting Choices

The two big names when it comes to cloud hosting are Azure and AWS. Both capable of hosting my project's servers. Initially, I attempted to use Azure but after using the free trial for a bit, I began to be charged for database usage. It was confusing as to why I was charged which was upsetting so I switched to AWS.

2.3 Chosen Solution

2.3.1 Stack Choice

2.3.1.1 Android (Client App)

I chose android as my client application because I have experience with it from courses taken at BCIT. Another determining factor is that I own several android devices.

2.3.1.2 ASP.NET Web API (Server API)

I chose ASP.NET Web API as my API framework for a few reasons. The main reason being there appeared to be enough resources to aid me as I have not used an API framework before. Also, as I took the Technical Programming option as my CST option, I have a good amount of experience with C#, ASP.NET, .NET Framework, Visual Studios, and IIS.

2.3.1.3 ASP.NET Core App (Live Score Broadcasting App)

I chose ASP.NET Core as my broadcasting application because of my experience with Microsoft technologies. Furthermore, ASP.NET Core is set to replace ASP.NET so I wanted to gain experience with it so that I don't fall behind in current technology trends.

2.3.1.4 SQL Server (MySQL)

I chose SQL Server because it works well with the entity framework and I am comfortable using it. I was reluctant to try a new database style for this project because I wanted more time for the focus of my project.

2.3.1.5 AWS (Hosting)

I chose AWS over Azure for multiple reasons. First, when attempting to use Azure I kept running into issues with costs during my free trial. Second, I wanted to bring my technological scope outside of the Microsoft world. Finally, AWS had a lot more understandable documentation for setting up a hosting environment.

2.3.2 Protocols

I choose to use TCP over UDP for my broadcast server because I believe it made more sense given the structure of my server. First, the broadcast server requires authentication/authorization which I did not want to do every time I wanted to send updates to the server from the client. Second, using TCP meant I only had to manage authentication/authorization going to the server from the client and not the other way around as well.

2.4 Feasibility Assessment Report

The problem with disc golf scoring applications that I investigated is most don't allow sharing data and the ones that did, had no security documentation which makes them untrustworthy for storing personal data such as user passwords. My project will be a great solution for disc golfers who want to be able to share disc golf data amongst a group of friends without it being accessible by unwanted users.

Another problem area for all scoring applications is that they do not allow live updates. The common scoring algorithm is that games are only updated after being completed. Rather than using this scoring algorithm, I will provide updates to the game on a per hole basis. This will enable my application to display live game results such as leaderboards and hole-by-hole results to other players, so they can follow along, on or off the course. Overall, providing live data gives users a more enriched experience as they can always be kept in the action.

Similar applications looked at in the market

Application	Found On	Shortcomings
Disc Caddy	Google Play store (no longer available)	No data sharing

		Requires paid option for stat tracking
Infinite Disc Golf Scorecard	Google Play store	- Too much personal information required to sign up (first name, last name, sex, etc) - No mention of how it is secure - Shared data is public to all (displayed on a website that all can view)
Discores	Google Play store	- No syncing/sharing of data - Importing data wipes the current data stored on the application.

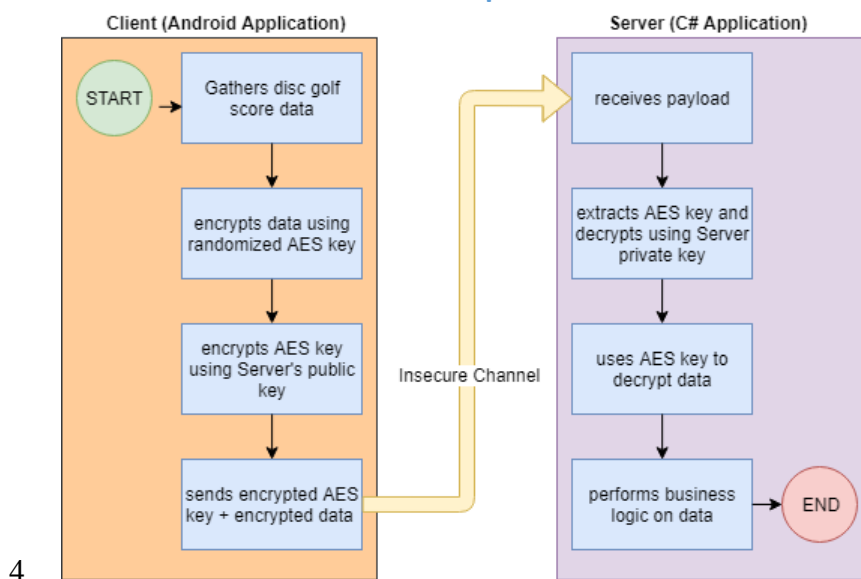
My project focuses on developing an application that can improve on the shortcomings of the applications mentioned by sharing live game data securely and privately.

2.4.1 14.1 Secure Data Transmission

I will transfer data securely using RSA/AES encryption and make my implementations public knowledge. Doing so will provide two benefits:

1. users will become more trusting of the project due to the openness of what was implemented.
2. if the security implementation is incomplete, people can point it out so that it can be fixed.

3. Data Transmission Process Example



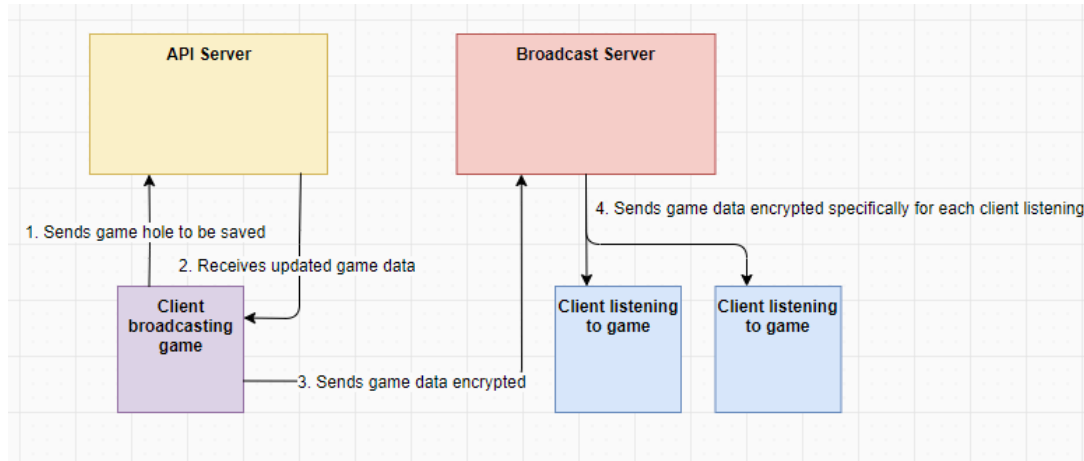
2.4.2 14.2 Data Restrictions

My application will allow for data to be shared amongst a friend group. Any user not part of the friend group will be unable to access the data via API calls.

2.4.3 14.3 Live Game Data Viewing

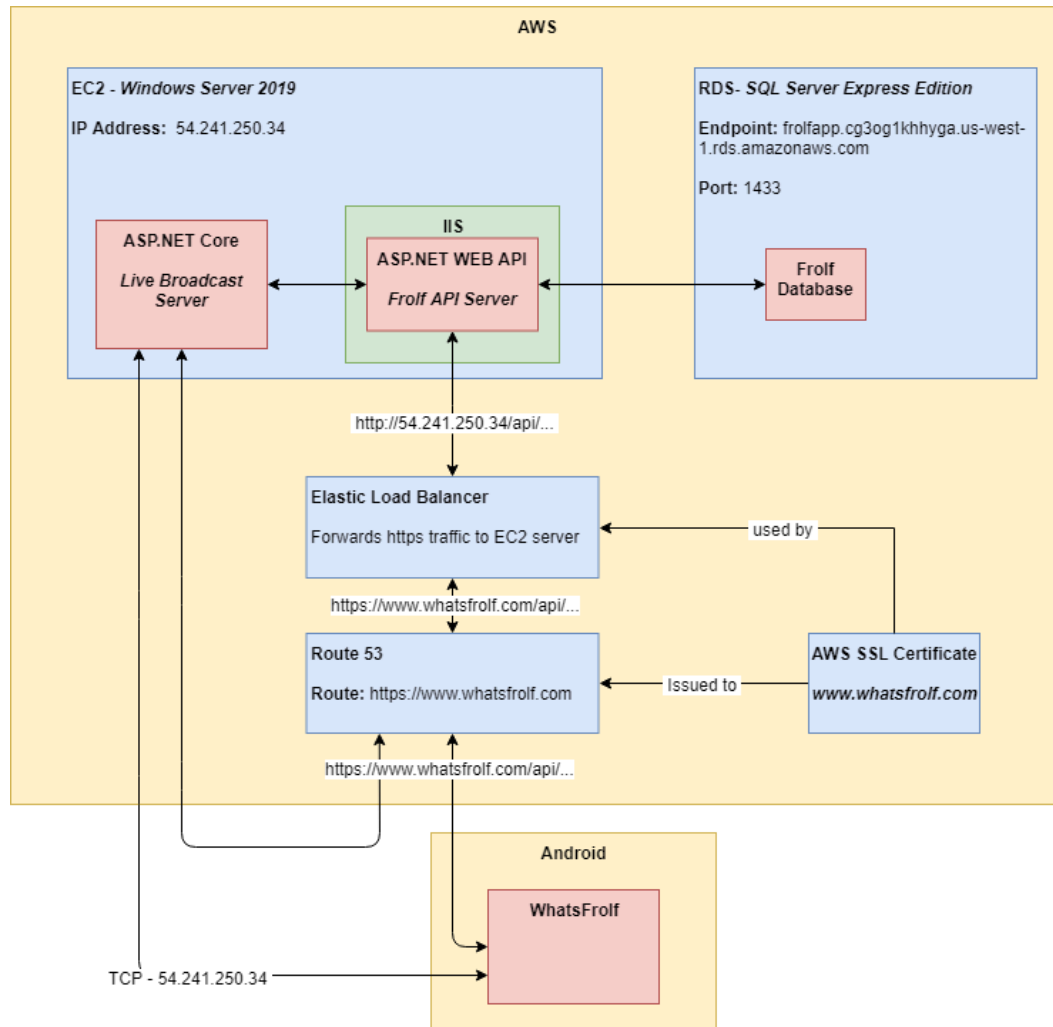
My project will use a hole-by-hole scoring algorithm which will allow players to view games being scored as they're being scored whether they are at the course or at home.

Broadcast Game Update Example



2.4.4 Deployment

2.4.4.1 Deployment Diagram



2.4.4.1.1 Definitions

2.4.4.1.1.1 AWS

Amazon web services cloud hosting. Both servers and the database are hosted here.

2.4.4.1.1.2 EC2 – Windows Server 2019

A windows server 2019 instance that hosts the live broadcasting server and API server.

Inbound Firewall Rules – Allows any IP access to the broadcaster but only the load balancer can access the API server.

Type ⁱ	Protocol ⁱ	Port Range ⁱ	Source ⁱ	Description ⁱ
HTTP	TCP	80	172.31.0.0/20	Allow from Balance...
Custom TCP Rule	TCP	45000	0.0.0.0/0	Score Broadcaster
Custom TCP Rule	TCP	45000	::/0	Score Broadcaster

2.4.4.1.1.3 Route 53

Amazon domain hosting. I purchased whatsfrolf.com. Two A records are set up to point whatsfrolf.com and www.whatsfrolf.com to the load balancer. CNAMEs have been added for both domains to the SSL certificate.

☐	whatsfrolf.com.	A	ALIAS www.whatsfrolf.com . (z1zmnedymnzn)
☐	www.whatsfrolf.com.	A	ALIAS dualstack.frolfapibalancer-644146965.us-wes
☐	_2bdf42e136397b7a1ecb6c63ffc786.www.whatsfrolf.com.	CNAME	_71574924cea75bf2a1bb93781242954d.duyqrilejt.a
☐	_33dbc8c80e0bf1207b18099e12cf038e.whatsfrolf.com.	CNAME	_9e1835107f8a80d3069fac8fb561da76.duyqrilejt.ac

2.4.4.1.1.4 Elastic Load Balancer

The load balancer distributes to EC2 instances on behalf of API requesters. It only accepts HTTPS requests.

Inbound Firewall Rules – Allow any HTTPS traffic

Type	Protocol	Port Range	Source
HTTPS	TCP	443	0.0.0.0/0
HTTPS	TCP	443	::/0

2.4.4.1.1.5 AWS Certificate

SSL Certificate. Good for whatsfrolf.com and www.whatsfrolf.com

Status

Status Issued
Detailed status The certificate was issued at 2019-08-15T05:34:11UTC

Domain	Validation status
whatsfrolf.com	Success

Add the following CNAME record to the DNS configuration for your domain. The procedure for adding CNAME records depends on your DNS service Provider. [Learn more.](#)

Name	Type	Value
_33dbc8c80e0bf1207b18099e12cf038e.whatsfrolf.com.	CNAME	_9e1835107f8a80d3069fac8fb561da76.duyqrilejt.acm-validations.aws

Note: Changing the DNS configuration allows ACM to issue certificates for this domain name for as long as the DNS record exists. You can revoke permission at any time by removing the record. [Learn more.](#)

[Create record in Route 53](#) Amazon Route 53 DNS Customers ACM can update your DNS configuration for you. [Learn more.](#)

Domain	Validation status
www.whatsfrolf.com	Success

Add the following CNAME record to the DNS configuration for your domain. The procedure for adding CNAME records depends on your DNS service Provider. [Learn more.](#)

Name	Type	Value
_2bdf42e136397b7a1ecb6c63ffc786.www.whatsfrolf.com.	CNAME	_71574924cea75bf2a1bb93781242954d.duyqrilejt.acm-validations.aws

Note: Changing the DNS configuration allows ACM to issue certificates for this domain name for as long as the DNS record exists. You can revoke permission at any time by removing the record. [Learn more.](#)

[Create record in Route 53](#) Amazon Route 53 DNS Customers ACM can update your DNS configuration for you. [Learn more.](#)

2.4.4.1.1.6 IIS

Hosts the API server, making it publicly accessible.

2.4.4.1.1.7 ASP.NET Core – Live Score Broadcast Server

The live score broadcast server. Handles requests from the android application to either broadcast a game live or receive live game updates. Requests are made from the broadcast server to the API server on behalf of the requester to validate if the android application request can be fulfilled.

2.4.4.1.1.8 ASP.NET Web API – Frolf API Server

Provides entry points to interact with the frolf app database in meaningful ways and handles business logic. Uses OAuth to handle account authentication and authorization.

2.4.4.1.1.9 RDS – SQL Server Express Edition

Hosts the database that will store all application related data.

Firewall Inbound Rules – Only allows the EC2 server access (accepts the EC2's private IP)

Type 	Protocol 	Port Range 	Source 	Description 
MS SQL	TCP	1433	172.31.7.8/32	

2.4.4.1.1.10 Frolf Database

The database that holds all related information for the application.

2.4.4.1.1.11 Android

The mobile platform that runs the frontend application.

2.4.4.1.1.12 Frolf Application

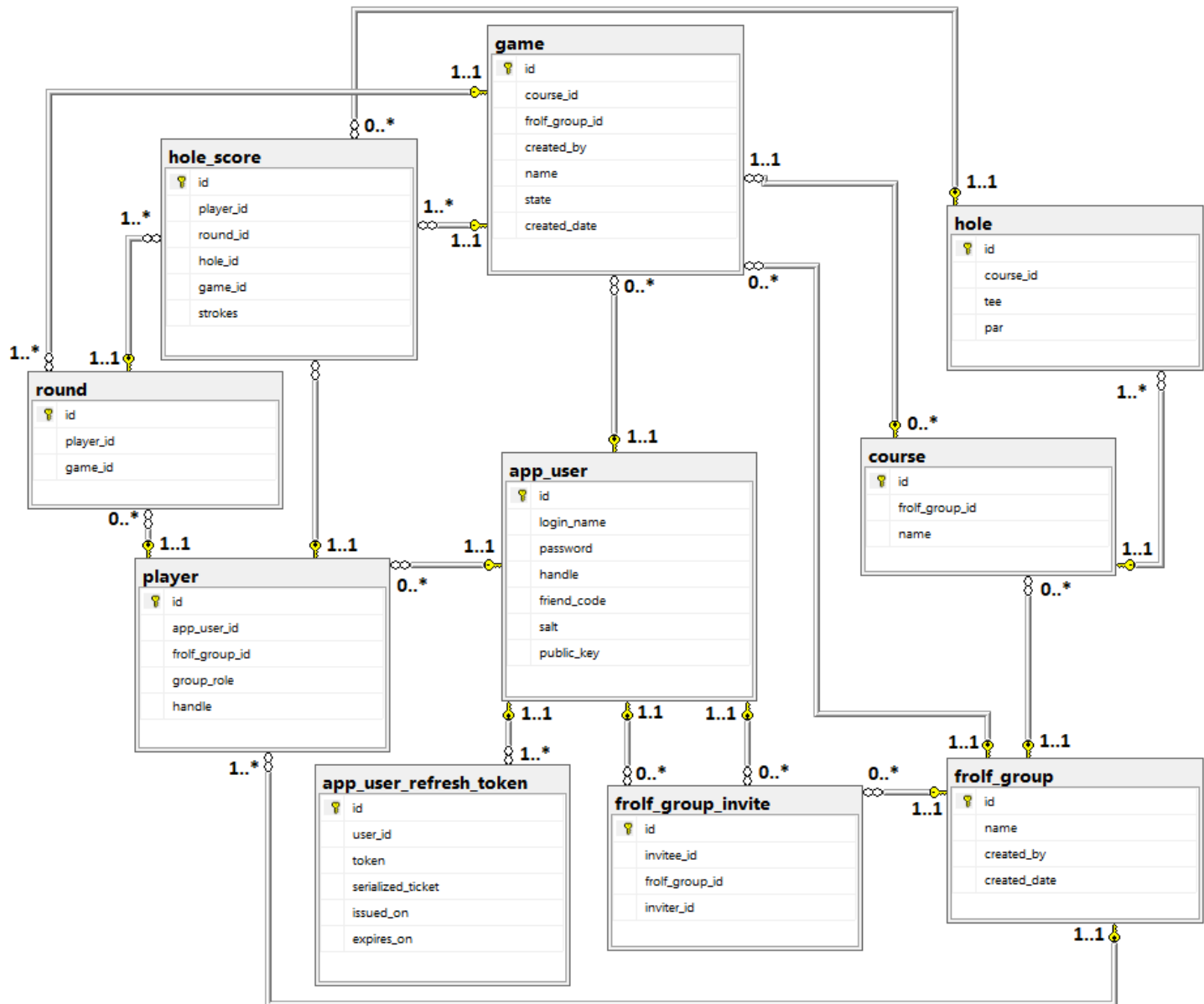
The android application that provides user interaction with the servers and displays data from the servers.

2.5 API Server Design

The server handles requests in the form of API calls. These API calls allow other applications to perform actions on the database in a controlled way. OAuth is used to authenticate users. Authenticated users receive a token if authenticated. The token is used to make API calls rather than a login name and password. Refresh tokens are implemented allowing applications to refresh expired tokens as needed. The refresh tokens durations are set to 6 hours.

2.5.1 Class Diagrams

2.5.1.1 Domain Entity Relationships



2.5.1.1.1 App User

Holds the user's application data. Importantly, this entity holds their credentials and RSA public key.

2.5.1.1.2 App User Refresh Token

Hold's information used to build a user's refresh token.

2.5.1.1.3 Frolf Group

Holds information about a group. A group is how users' data is kept private (within the group).

2.5.1.1.4 Frolf Group Invite

Holds information for a pending group invite.

- Who the invite is for
- Who sent the invite
- What group is the invite to

2.5.1.1.5 Player

Associates an app user to a frolf group. Allows a user to be part of many groups.

2.5.1.1.6 Course

Holds data about a frolf course. Forced to be associated with a single frolf group.

2.5.1.1.7 Hole

Holds data about a single course hole. Forced to be associated with a single frolf group. A set of holes associated with the same course represents the holes for the course. They should be ordered from 1..n.

2.5.1.1.8 Game

Holds data about a game. This includes 1 group, 1 course from the group, 1..n players from the group, and the scores for each player through association.

2.5.1.1.9 Round

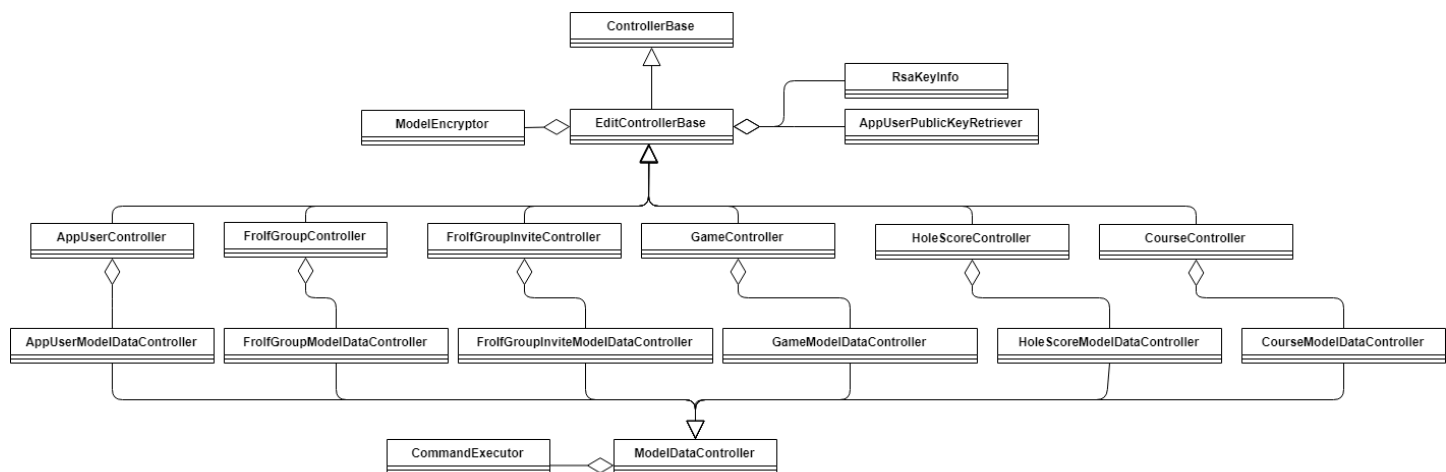
Holds a player's data for a game. Associated with 1 game.

2.5.1.1.10 Hole Score

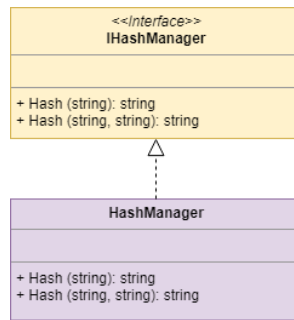
Holds a player's score on a single hole. A set of hole scores associated with the same round represents a player's score for a game.

2.5.1.2 API Server - High-Level API

The diagrams were getting too cluttered when putting them into one diagram. Here is a high-level look at how the classes on the backend are working together. I've broken up the diagram into smaller sections and gone into detail for each one.



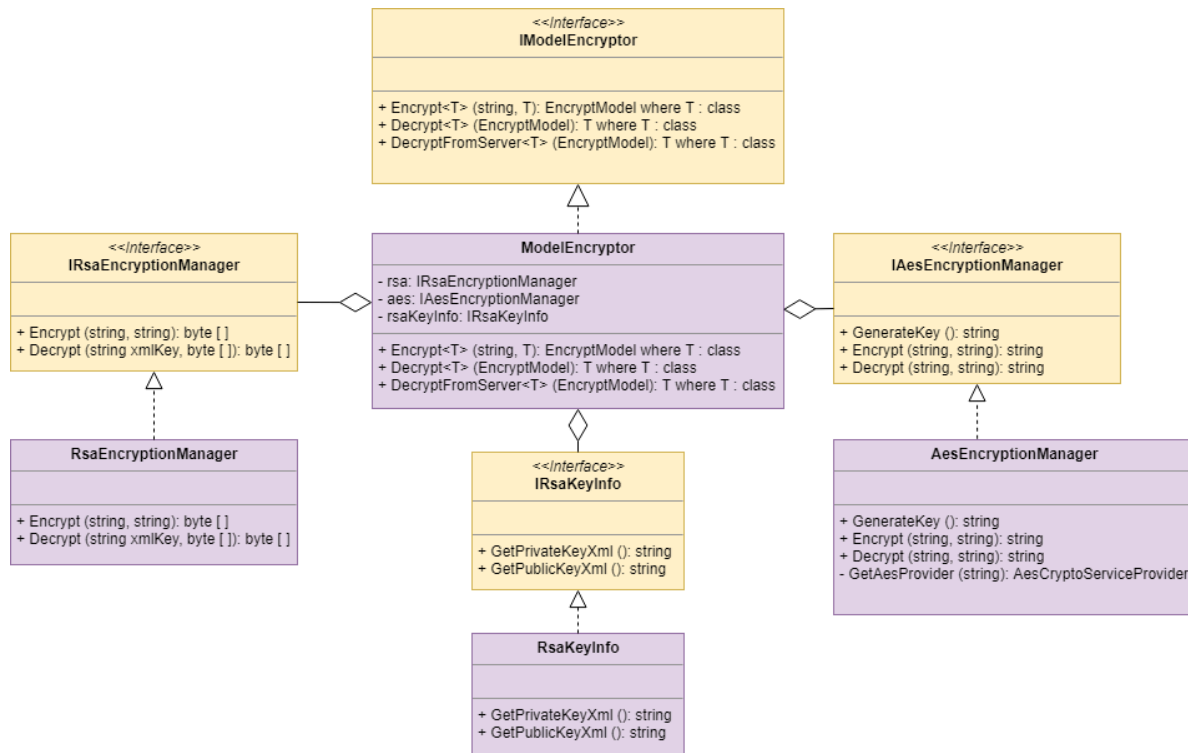
2.5.1.3 Hashing



2.5.1.3.1 HashManager

Hashes strings using a C# provided class SHA256CryptoServiceProvider.

2.5.1.4 Encryption



2.5.1.4.1 RsaEncryptionManager

Encrypts strings and decrypts byte arrays using C#'s RSACryptoServiceProvider. Expects RSAPParameters for keys.

RSACryptoServiceProvider

```

//
// Summary:
//     Performs asymmetric encryption and decryption using the implementation of the
//     System.Security.Cryptography.RSA algorithm provided by the cryptographic service
//     provider (CSP). This class cannot be inherited.
public sealed class RSACryptoServiceProvider : RSA, ICspAsymmetricAlgorithm
  
```

RSAParameters

```
//
// Summary:
//     Represents the standard parameters for the System.Security.Cryptography.RSA algorithm.
public struct RSAParameters
```

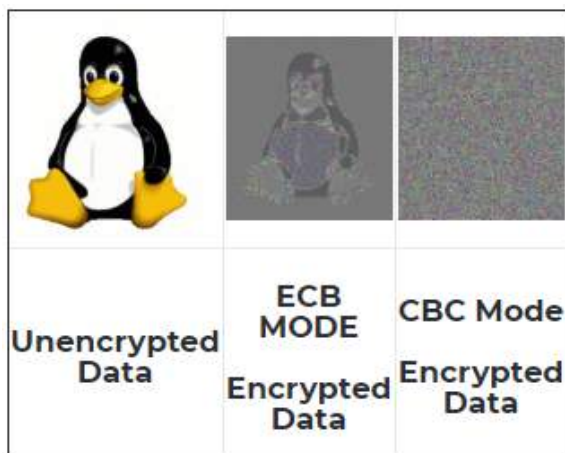
2.5.1.4.2 AesEncryptionManager

Encrypts and decrypts strings using C#'s AesCryptoServiceProvider. The AES encryption uses cipher blocker chaining (CBC) which is more secure than using electronic codebook (ECB).

AesCryptoServiceProvider

```
//
// Summary:
//     Performs symmetric encryption and decryption using the Cryptographic Application
//     Programming Interfaces (CAPI) implementation of the Advanced Encryption Standard
//     (AES) algorithm.
public sealed class AesCryptoServiceProvider : Aes
```

An example between the difference in security between CBC and ECB



The Manager also provides AES key generation using C#'s RNGCryptoServiceProvider.

RNGCryptoServiceProvider

```
//
// Summary:
//     Implements a cryptographic Random Number Generator (RNG) using the implementation
//     provided by the cryptographic service provider (CSP). This class cannot be inherited.
// [ComVisible(true)]
public sealed class RNGCryptoServiceProvider : RandomNumberGenerator
```

2.5.1.4.3 RsaKeyInfo

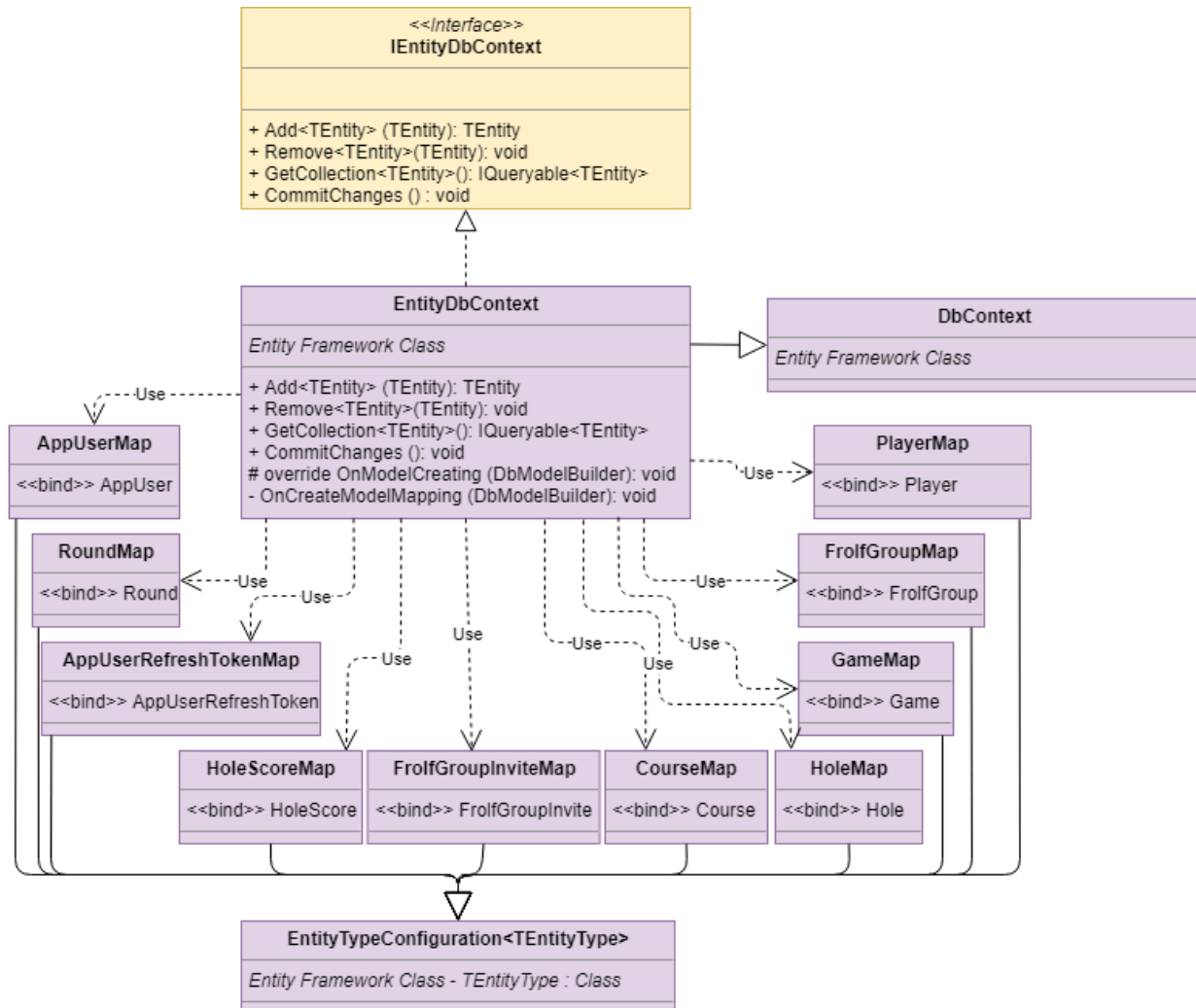
Provides the server's RSA key info as XML formatted strings.

2.5.1.4.4 ModelEncryptor

Encrypts and decrypts model data using a combination of AES and RSA encryption. See the [sequence diagrams](#) for more detail.

- **Encryption:** First a random key is generated and used to encrypt the model data. Then RSA encryption is used to encrypt the AES key. The RSA encrypted key and AES encrypted model data is packaged together and returned.
- **Decryption:** The reverse process of encryption.

2.5.1.5 Entity Database Context



2.5.1.5.1 EntityTypeConfiguration

A C# class that allows mapping an object's properties to database columns. Each entity type has their own configuration. Here is an example of the mapping for a course:

```
public class CourseMap : EntityTypeConfiguration<Course>
{
    public CourseMap()
    {
        ToTable("course");

        // Primary key
        HasKey(c => c.EntityKey);

        // Properties
        Property(c => c.EntityKey)
            .HasColumnName("id");

        Property(c => c.FrolfGroupId)
            .HasColumnName("frolf_group_id")
            .IsRequired();

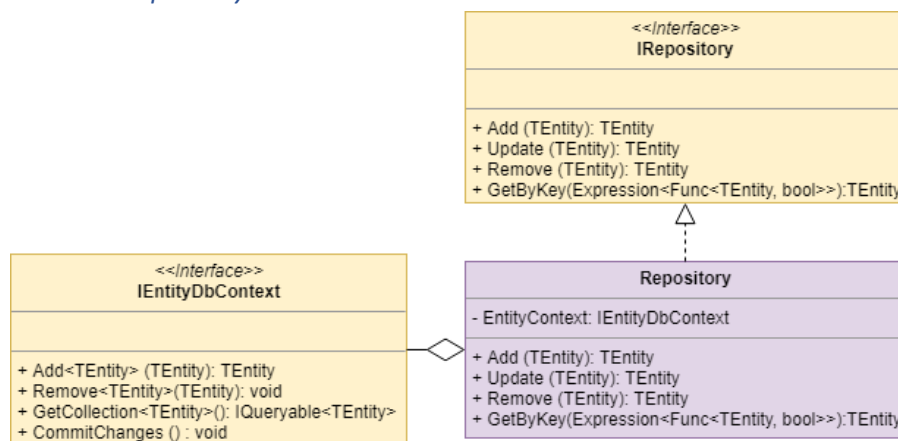
        Property(c => c.Name)
            .IsRequired()
            .HasMaxLength(50)
            .HasColumnName("name");

        // Relationships
        // - Link to frolf group
        HasRequired(c => c.FrolfGroup)
            .WithMany(fg => fg.Courses)
            .HasForeignKey(c => c.FrolfGroupId);
    }
}
```

2.5.1.5.2 EntityDbContext

Provides interaction with the database. Allows for adding, removing and retrieving. This class also sets up the entity to database table mappings.

2.5.1.6 Repository



2.5.1.6.1 Repository

A wrapper for the EntityDbContext. Allows controlling main database entry responsibilities at a high level. For example, before being added to the database, new entities are given a new key.

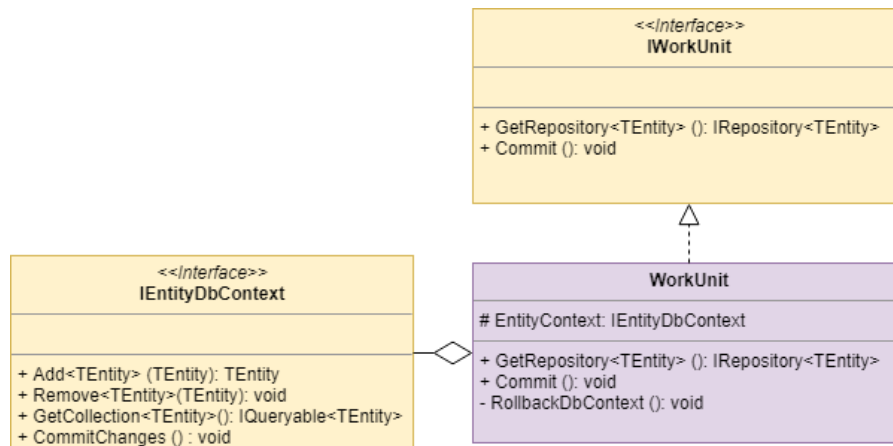
New entity creation in Repository.Add

```
public virtual TEntity Add(TEntity newEntity)
{
    if (newEntity is IGuidEntity guidEntity)
    {
        guidEntity.EntityKey = Guid.NewGuid();
    }

    if (newEntity is IOwnable ownableEntity)
    {
        ownableEntity.CreatedBy = UserExtensions.GetCurrentUserId();
        ownableEntity.CreatedDate = DateTime.UtcNow;
    }

    return entityContext.Add(newEntity);
}
```

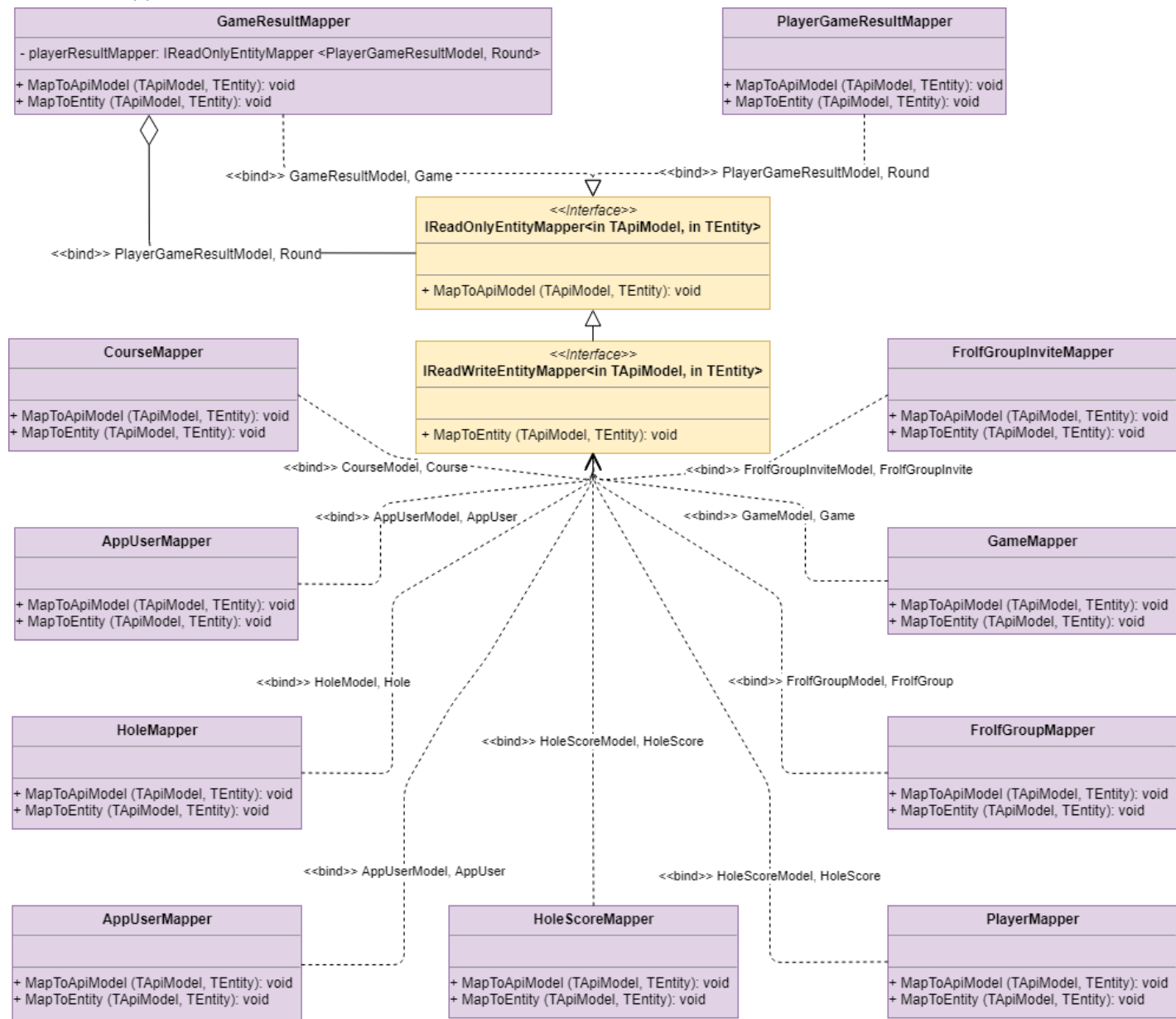
2.5.1.7 Work Unit



2.5.1.7.1 Work Unit

Creates repositories, passing in the current entity context property. This ensures all repositories requested make changes to the same context, so when the work unit commits changes on the context property, all changes are committed (saved) that were made by each repository.

2.5.1.8 Mappers



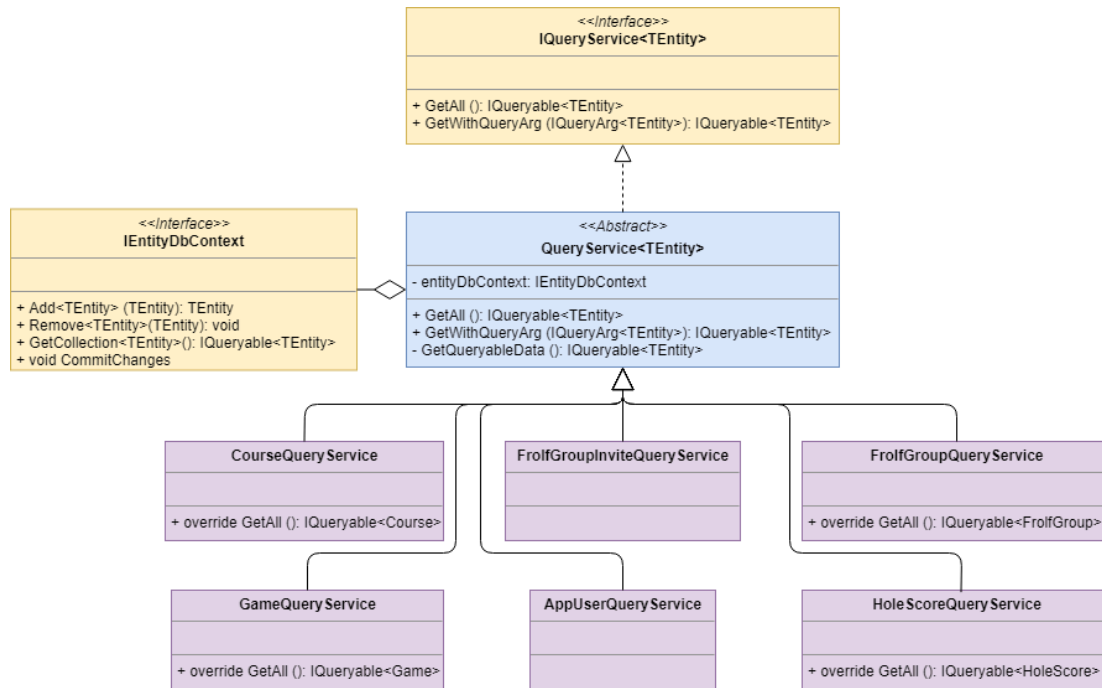
2.5.1.8.1 ReadOnlyEntityMappers

Maps properties from an entity to a model. Uses generics to determine which entity and model to use.

2.5.1.8.2 ReadWriteEntityMapper

Provides the capabilities of a read only mapper as well as mapping from a model's properties to an entity's properties.

2.5.1.9 Query Services



2.5.1.9.1 QueryService

Provides retrieval of entity data from the database. The data from the database is returned as an `IQueryable`, meaning data has not yet been put into memory. Only when the queryable is enumerated is the data put into memory. Often queries reduce the set of data to a small enough set that putting it into memory can be handled. For example, a game query being used can query for a set of games related to a single user and put that into memory rather than all games in the database.

2.5.1.10.2 CommandPreHandler

Handlers that are run before command handlers. These are used to validate commands before they can be handled. For example, the AddCourseCommandPreHandler:

```
public class AddCourseCommandValidation : CommandPreHandler<AddCourseCommand>
{
    private readonly IQueryService<FrolfGroup> frolfGroupQuery;

    public AddCourseCommandValidation(IQueryService<FrolfGroup> frolfGroupQuery, IWorkUnit workUnit)
        : base(workUnit)
    {
        this.frolfGroupQuery = frolfGroupQuery;
    }

    public override void OnPreHandleCommand(AddCourseCommand command)
    {
        var course = command.newEntity;

        Assert(!string.IsNullOrEmpty(course.Name), "Course name must be set");

        AssertFrolfGroupExists(course);
        AssertCurrentUserInFrolfGroup(course);
        AssertCourseHolesSet(course);
        AssertCourseHolePars(course);
        AssertCourseHoleOrderUnique(course);
        AssertCourseHoleTeeSequence(course);
    }
}
```

Any assertions that fail will throw an exception which is caught and handled by the API server.

2.5.1.10.3 CommandHandler

Handlers that run if their pre handler passed all assertions. Command handlers commit the object held by the command to the database in most cases. The counterpart to the AddCourseCommandPreHandler is the AddCourseCommandHandler, it adds a course that passes through the pre handler, to the database.

AddCourseCommandHandler

```
public class AddCourseCommandHandler : CommandHandler<AddCourseCommand>
{
    public AddCourseCommandHandler(IWorkUnit workUnit)
        : base(workUnit)
    {
    }

    protected override void OnHandleCommand(AddCourseCommand command)
    {
        GetRepository<Course>().Add(command.newEntity);
        AddHoles(command.newEntity);
    }
}
```

2.5.1.10.4 CommandLocator

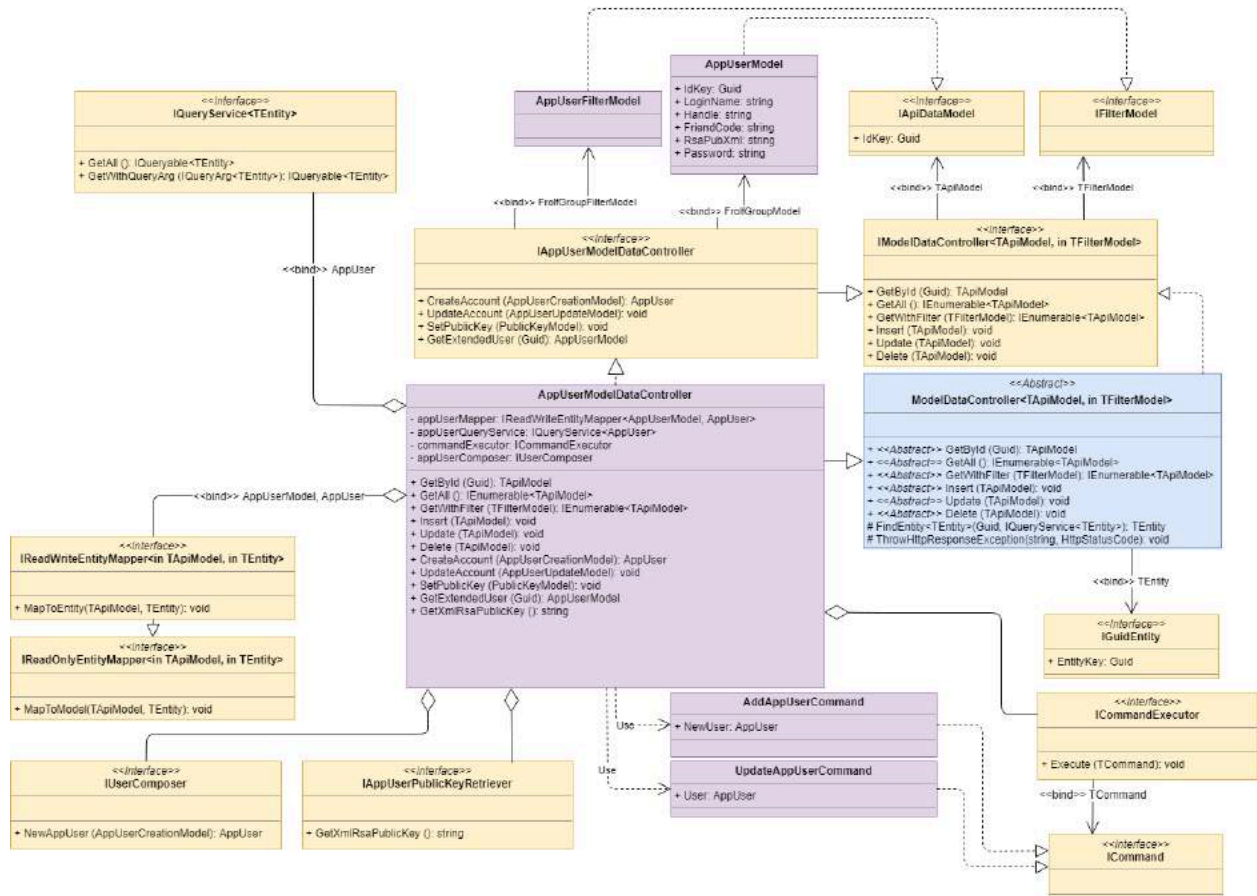
Uses the dependency injection to retrieve the command pre handler and handler for a given command.

2.5.1.10.5 CommandExecutor

Executes the handlers for the given command. First the pre handler is executed and then the handler.

2.5.1.11 App User

2.5.1.11.1 Model Data Controller



2.5.1.11.1.1 AppUserModel

Holds data mapped from an AppUserEntity

2.5.1.11.1.2 AppUserFilterModel

Holds values to filter app users by

2.5.1.11.1.3 AppUserModelDataController

Coordinates the App User entity. Provides methods to add/update/retrieve the entity. The AppController uses the model data controller as its entry point to an entity.

2.5.1.11.1.4 AddAppUserCommand

Holds data required to add a user to the database.

2.5.1.11.1.5 UpdateAppUserCommand

Holds data required to update a user in the database.

2.5.1.11.1.6 User Composer

Builds an app user based on the app creation request model. Removes complexity from the controller.

2.5.1.11.1.7 AppUserPublicKeyRetriever

Retrieves the calling user's public key.

2.5.1.12.1.6 UpdateFrolfGroupCommand

Holds data to update a frolf group.

2.5.1.12.1.7 AddGameCommand

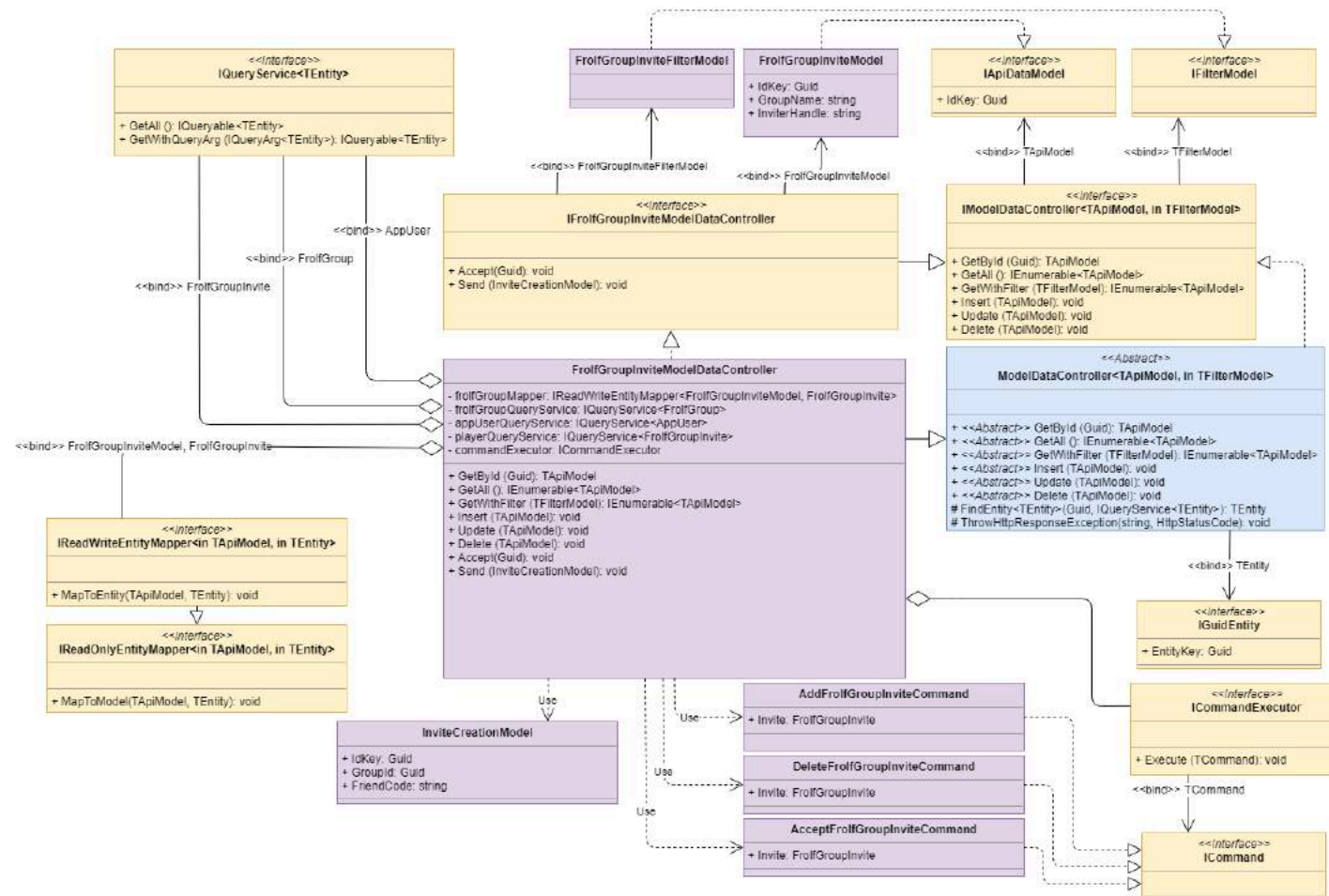
Holds data required to add a game to a frolf group.

2.5.1.12.1.8 GameComposer

Creates a game based on a game creation model given by the requesting user. Helps keep the model data controller less complicated.

2.5.1.13 Frolf Group Invite

2.5.1.13.1 Model Data Controller



2.5.1.13.1.1 FrolfGroupInviteModel

Holds data mapped from a frolf group invite entity.

2.5.1.13.1.2 FrolfGroupInviteFilter

Holds values used to filter frolf group invites by.

2.5.1.13.1.3 FrolfGroupInviteModelDataController

Coordinates the Frolf Group Invite entity. Provides methods to add/update/retrieve the entity. The **FrolfGroupInviteController** uses the model data controller as its entry point to the entity.

2.5.1.13.1.4 AddFrolfGroupInviteCommand

Holds data required to add a frolf group invite to the database.

2.5.1.13.1.5 DeleteFrolfGroupCommand

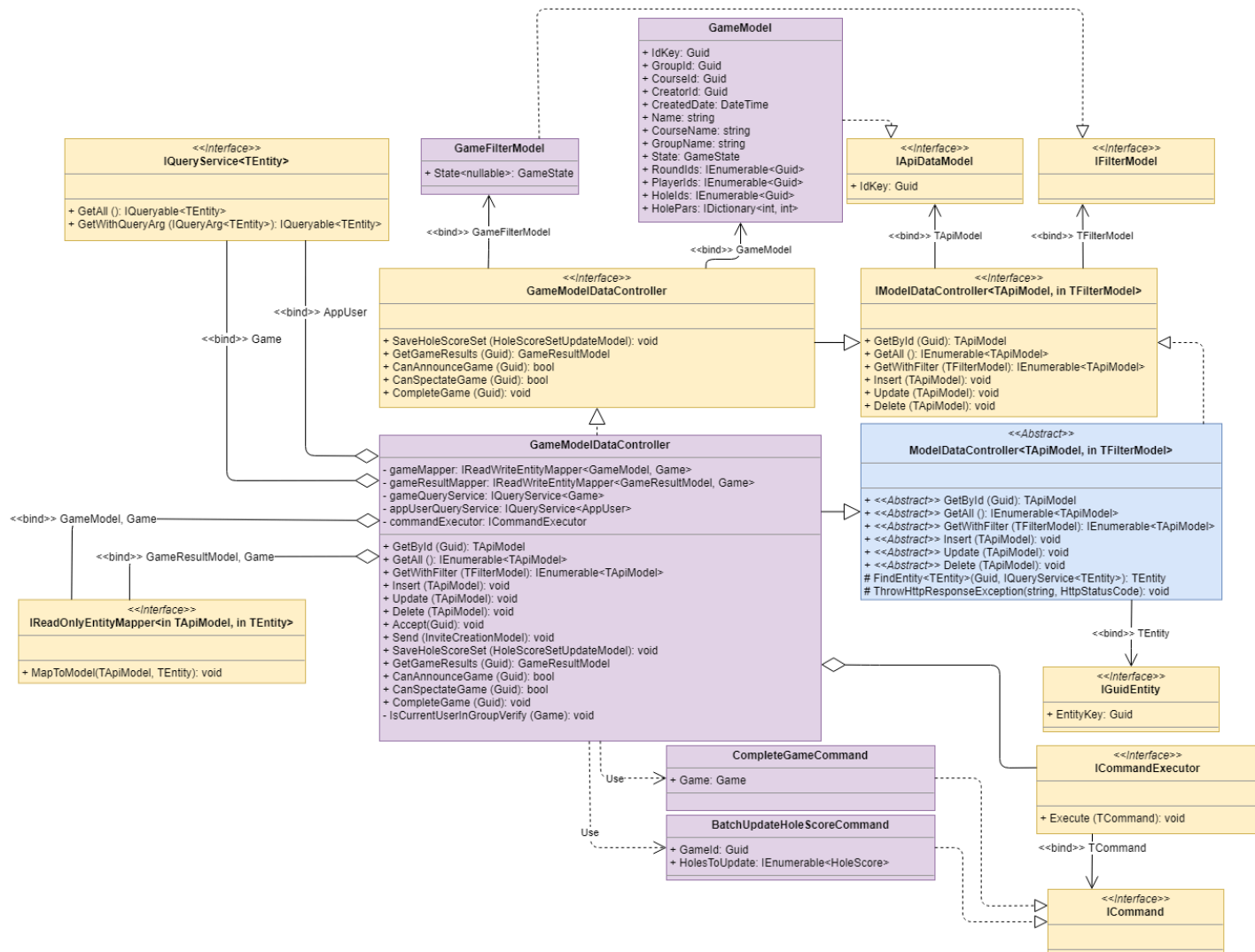
Holds data required to delete a frolf group invite from the database.

2.5.1.13.1.6 AcceptFrolfGroupInviteCommand

Holds data required to accept a frolf group invite.

2.5.1.14 Game

2.5.1.14.1 Model Data Controller



2.5.1.14.1.1 GameModel

Holds data mapped from a game entity

2.5.1.14.1.2 GameFilterModel

Holds values to filter games by.

2.5.1.14.1.3 GameModelDataController

Coordinates the Game entity. Provides methods to add/update/retrieve the entity. The GameController uses the model data controller as its entry point to the entity.

2.5.1.14.1.4 CompleteGameCommand

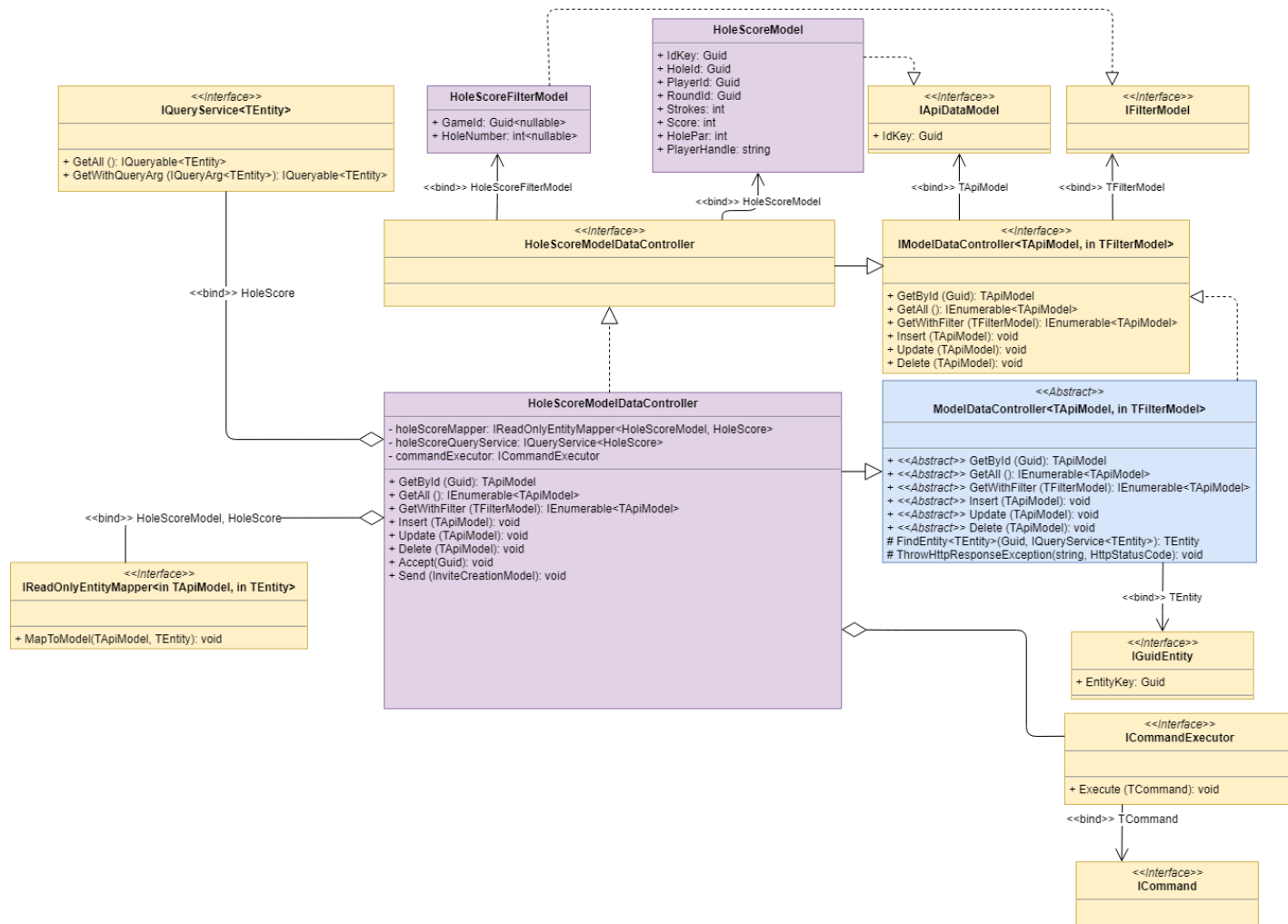
Holds data to complete a game command.

2.5.1.14.1.5 BatchUpdateHoleScoreCommand

Holds data required to update a set of hole scores.

2.5.1.15 Hole Score

2.5.1.15.1 Model Data Controller



2.5.1.15.1.1 HoleScoreModel

Holds data mapped from a hole score.

2.5.1.15.1.2 HoleScoreFilterModel

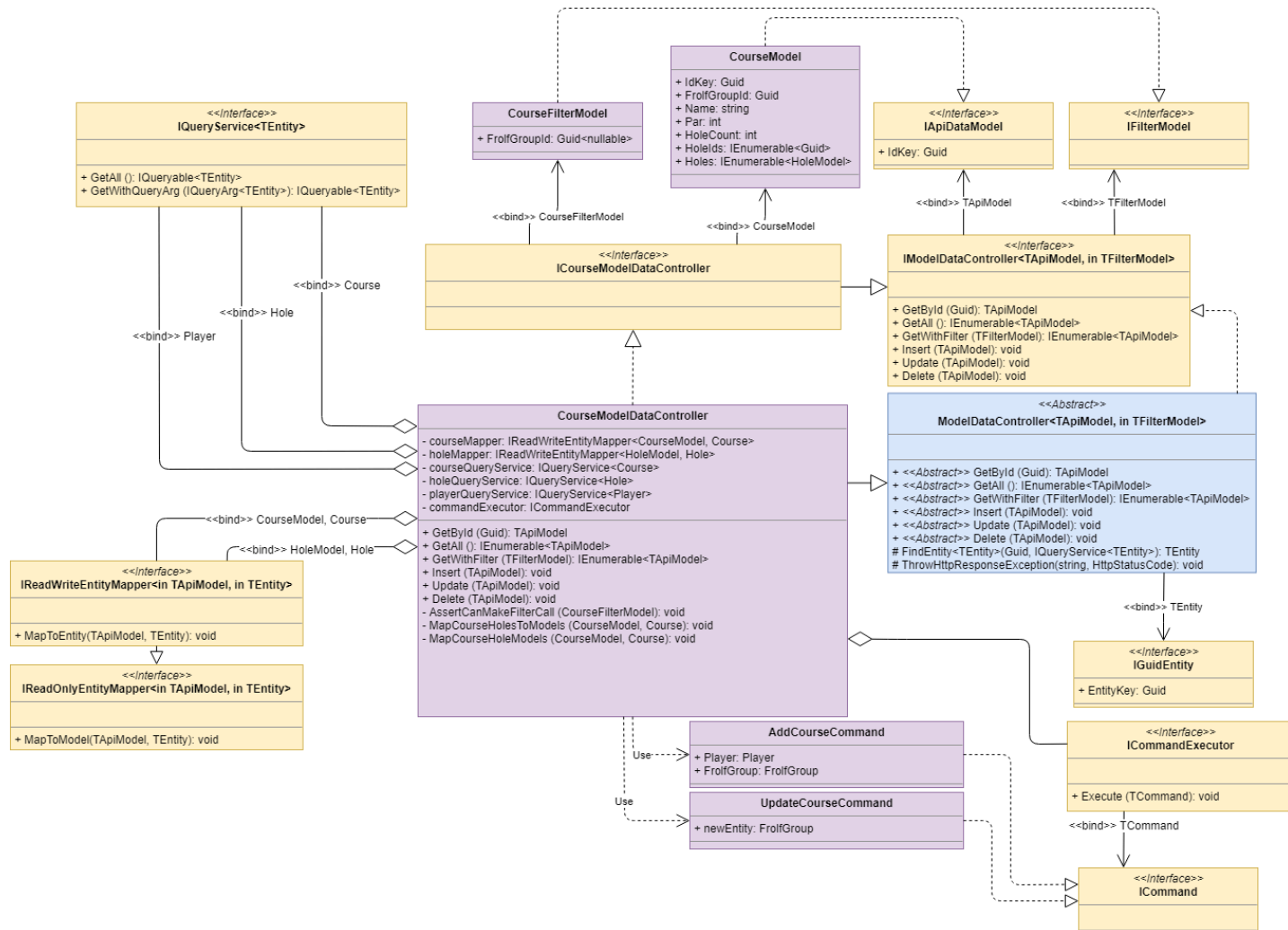
Holds values to filter hole scores by.

2.5.1.15.1.3 HoleScoreModelDataController

Coordinates the HoleScore entity. Provides methods to add/update/retrieve the entity. The HoleScoreController uses the model data controller as its entry point to the entity.

2.5.1.16 Course

2.5.1.16.1 Model Data Controller



2.5.1.16.1.1 CourseModel

Holds data mapped from a course entity.

2.5.1.16.1.2 CourseFilterModel

Holds values to filter courses by.

2.5.1.16.1.3 CourseModelDataController

Coordinates the Course entity. Provides methods to add/update/retrieve the entity. The Course Controller uses the model data controller as its entry point to the entity.

2.5.1.16.1.4 AddCourseCommand

Holds data required to add a course.

2.5.1.16.1.5 UpdateCourseCommand

Holds data required to update a course.

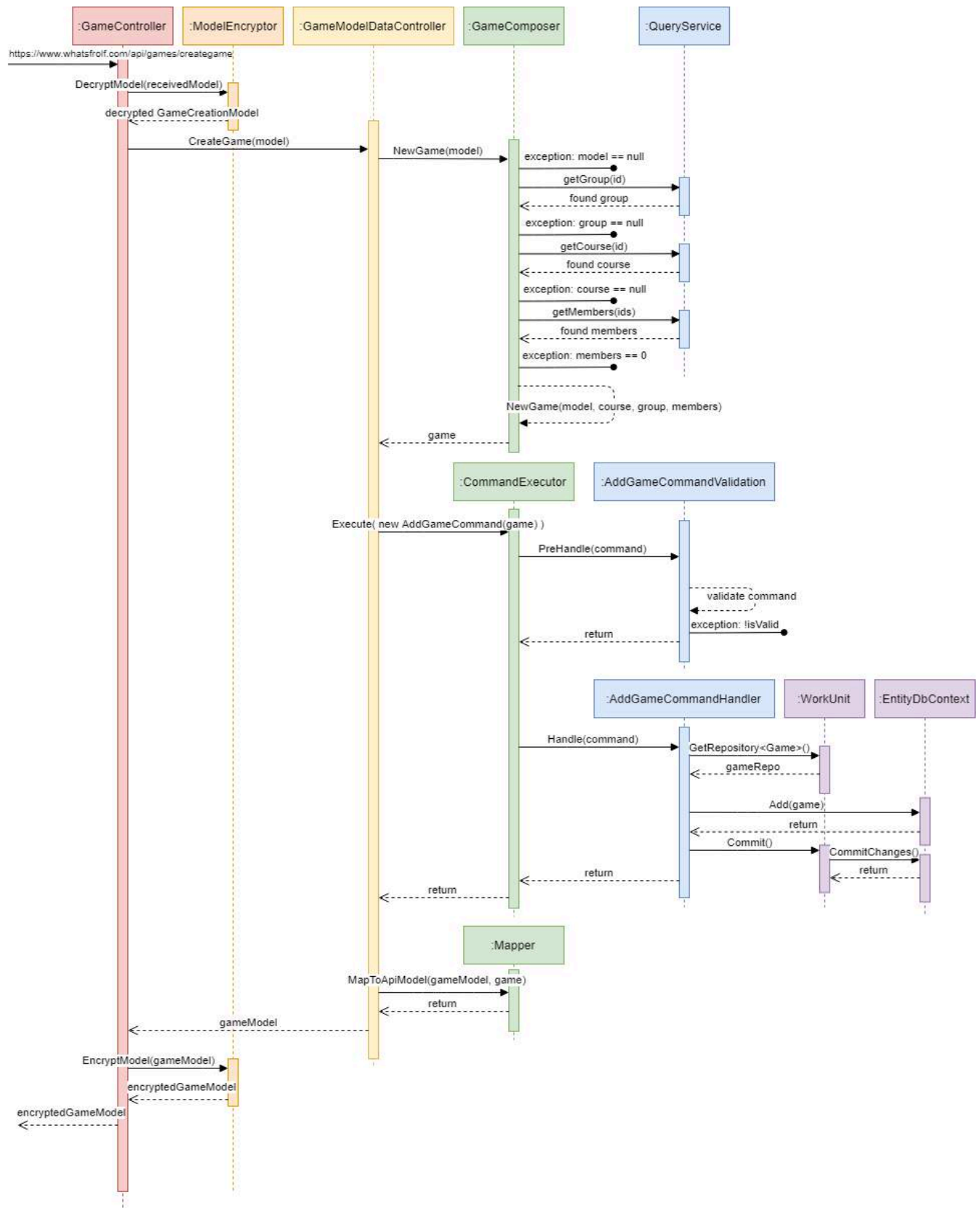
2.5.2 Sequence Diagrams

2.5.2.1 *Executing an API call that receives/returns data as well as executes a command.*

The most complete API request is one that saves data to the database using a command and sends data back.

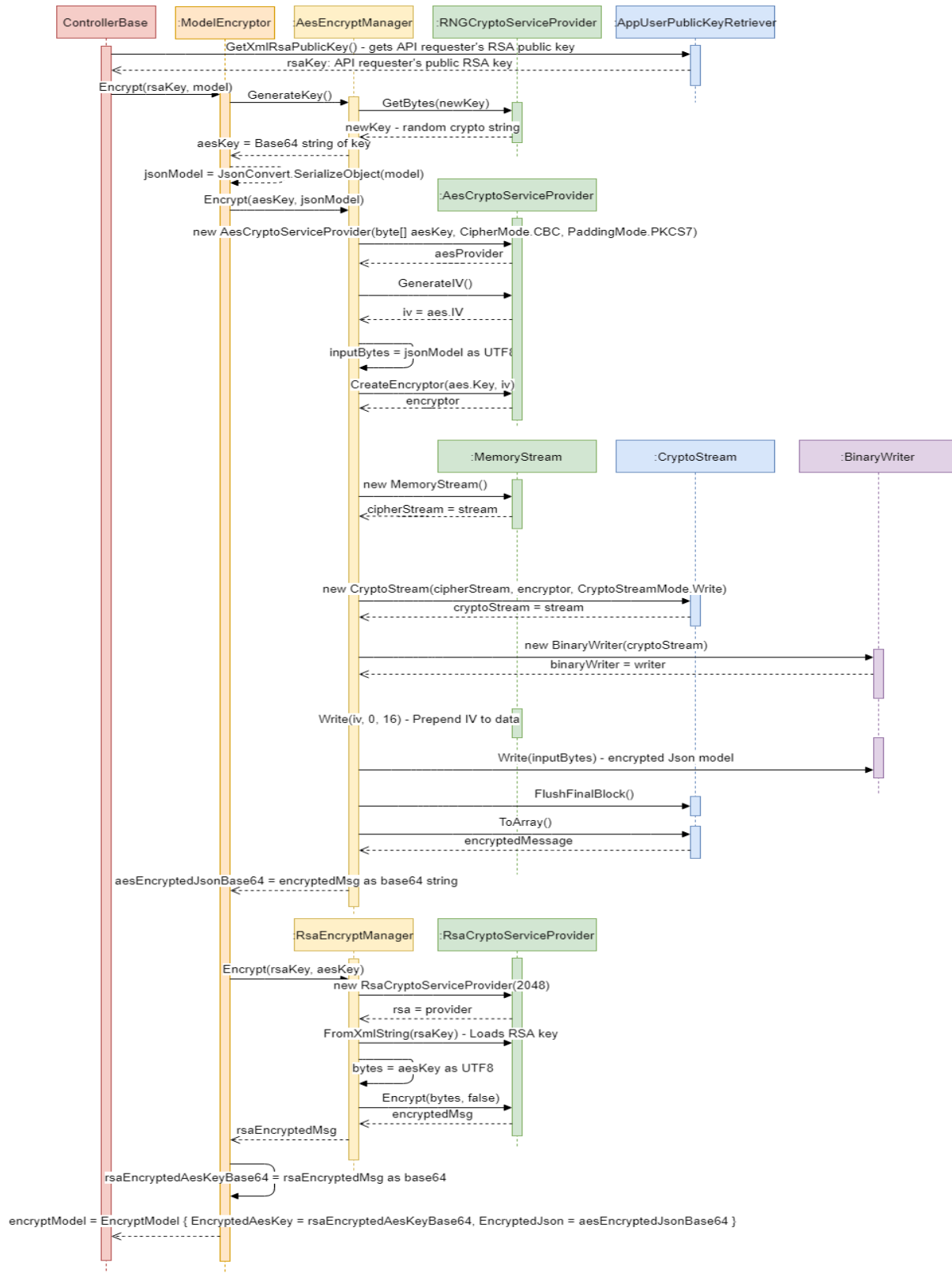
2.5.2.1.1 Example: Creating a Game

An API request made to create a game.

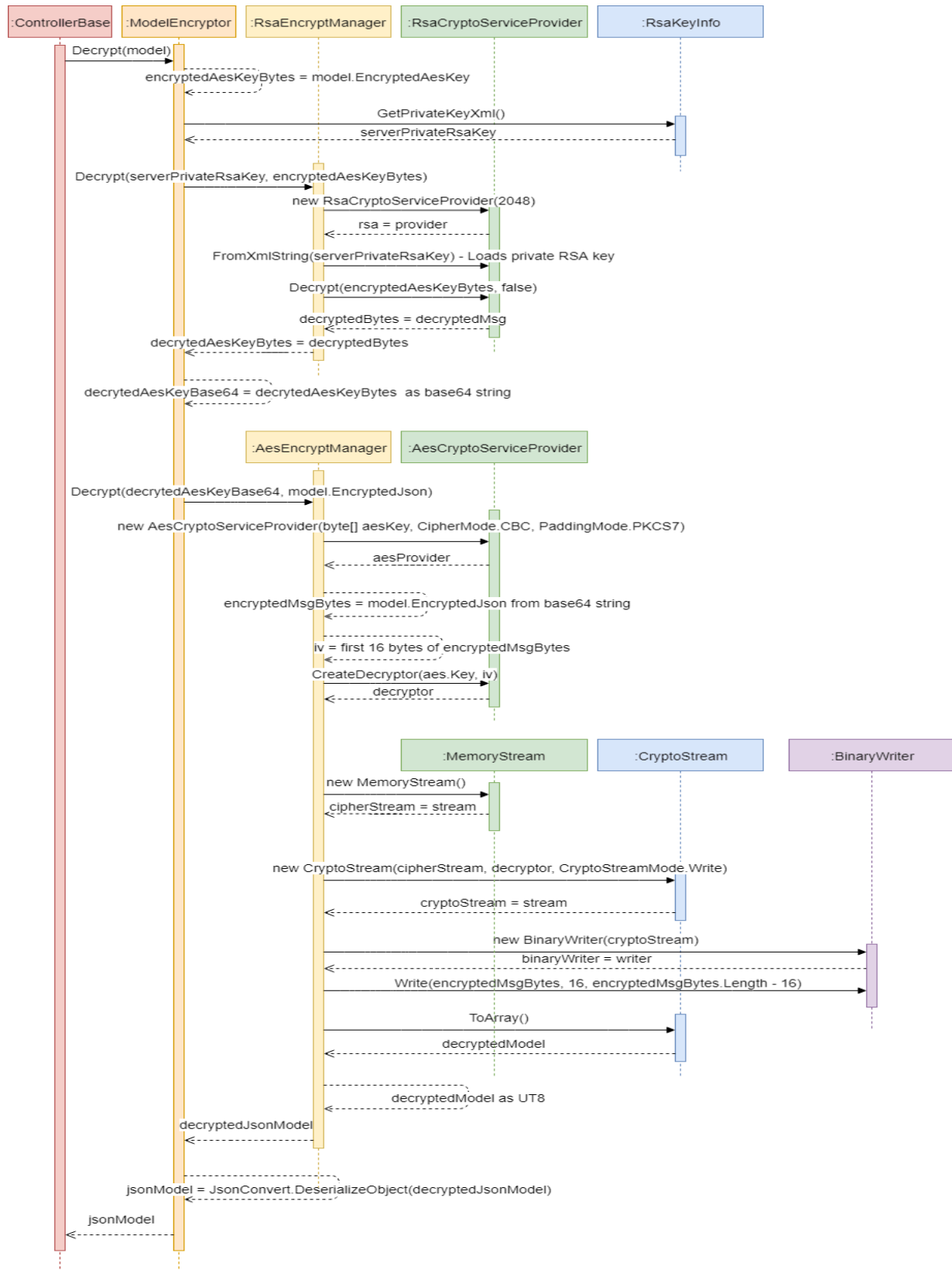


2.5.2.2 Model Encryption

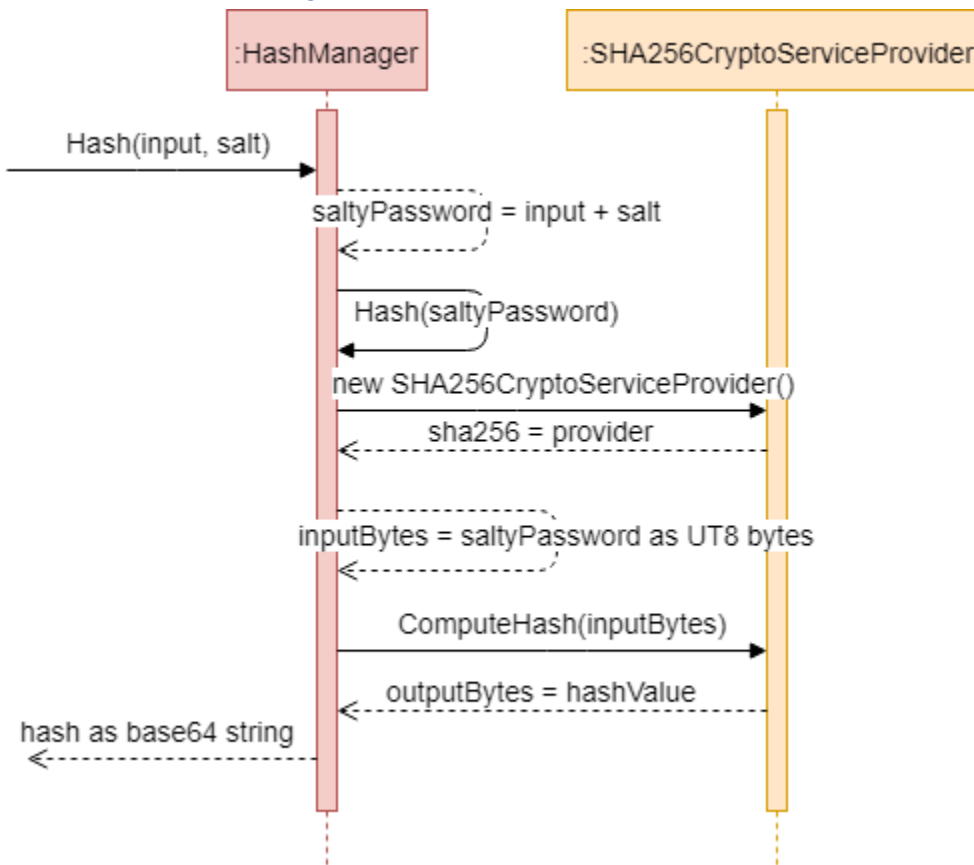
2.5.2.2.1 Encryption



2.5.2.2.2 Decryption



2.5.2.3 Password Hashing



2.5.3 API Calls

All data sent to or received from the API is expected to be encrypted using the RSA/AES encryption process and sent as an `EncryptModel`. The `EncryptModel` will provide the encrypted AES key needed to decrypt the encrypted JSON model provided. Each API description outlines what models will be expected to be in the `EncryptModel` for the receives and returns.

There are two exceptions for the model encryption model expectation which are the OAuth API calls that are required to use the URL form encoded to send data. In this case, user data is only RSA encrypted.

2.5.3.1 OAuth

Login	
Route	oauth2/token
Http Method	Post
Rules	- Login name is for an existing user - Password matches the one for the user found with the requested login name
Received Model	RSA encrypted login name and password
Returned Model	TokenModel
Notes	- Contains the refresh token - FromUrlEncoded: granttype=password&username=<login name>&password=<password for user> - Login name and password are RSA encrypted

Refresh Token	
Route	oauth2/token
Http Method	Post
Rules	- An app user exists for the given Guid
Received Model	IdModel
Returned Model	TokenModel
Description	Refreshes the token based on the refresh token given
Notes	- FromUrlEncoded: granttype=refresh_token&refresh_token=<user refresh token>

2.5.3.2 AppUserController

Base Route: api/appusers

Get Current User Info	
Route	info
Http Method	Get
Rules	- An app user exists for the given Guid
Received Model	IdModel
Returned Model	AppUserModel
Description	Retrieves the current user's information
Notes	

Get Current User Info Internal	
Route	info/internal
Http Method	Get
Rules	- An app user exists for the given Guid
Received Model	n/a
Returned Model	AppUserModel
Description	Retrieves an app user model for internal server use
Notes	- Data is encrypted using the server's public key

Create Account	
Route	create/account
Http Method	Post
Rules	- Friend code must be unique - Login name must be unique - Password and confirm password must match - No null fields in the AppUserCreationModel
Received Model	AppUserCreationModel
Returned Model	AppUserModel
Description	Creates a new account based on the parameters in the AppUserCreationModel
Notes	- Can be called anonymously

Set New Public Key	
Route	setPublicKey
Http Method	Post
Rules	- The key string is a valid XML Public RSA key
Received Model	PublicKeyModel
Returned Model	n/a
Description	Sets the given RSA public key for the currently logged in user.
Notes	

Update Account	
Route	update/account
Http Method	Post
Rules	- The account being updated must be for the calling user
Received Model	AppUserUpdateModel
Returned Model	n/a
Description	Updates the logged in user's account info.
Notes	- Login name can be updated - Handle can be updated - The handle is changed for every group the user is in

2.5.3.3 CourseController

Base Route: api/courses

Get By Id	
Route	getById
Http Method	Post
Rules	- The calling user must be in the group the course is for to retrieve it
Received Model	IdModel – Guid for the course to retrieve
Returned Model	CourseModel
Description	Gets the course requested by the calling user
Notes	

Get All	
Route	<i>Base post call</i>
Http Method	Post
Rules	- The calling user must be in the group the course is for to retrieve it
Received Model	n/a
Returned Model	IEnumerable<CourseModel>
Description	Gets all the courses that are in groups the calling user is in
Notes	

Get With Filter	
-----------------	--

Route	filter
Http Method	Post
Rules	- The calling user must be in the group the course is for to retrieve it
Received Model	n/a
Returned Model	IEnumerable<CourseModel>
Description	Gets all the courses for groups the calling user is in
Notes	- Can filter by a group the course is for

Create	
Route	insert
Http Method	Post
Rules	- Course name is not null or empty - The frof group, being associated with the course, exists - The calling user is in the group the course is for - All pars are set to 1 or more - Hole order is unique and in ascending order starting at 1 e.g 1, 2, 3, 4 where the course has 4 holes
Received Model	CourseModel
Returned Model	CourseModel
Description	Creates a course for a given group.
Notes	- The course model used to create the course is returned with its new ID.

Update	
Route	update
Http Method	Post
Rules	- Course name is not null or empty - The course exists - The calling user is in the group the course is for - All pars are set to 1 or more - Hole order is unique and in ascending order starting at 1 e.g 1, 2, 3, 4 where the course has 4 holes
Received Model	CourseModel
Returned Model	n/a
Description	Updates the course
Notes	- Can update the course name

2.5.3.4 FrolfGroupController

Base Route: api/frolfgroups

Get By Id	
Route	getById
Http Method	Post
Rules	- The calling user must be in the group being requested
Received Model	IdModel – Guid for the group requested

Returned Model	FrolfGroupModel
Description	Gets the frolf group requested
Notes	

Get All	
Route	getAll
Http Method	Post
Rules	- The calling user must be in the groups to be retrieved
Received Model	n/a
Returned Model	IEnumerable<FrolfGroupModel>
Description	Gets all the groups the calling user is in
Notes	

Create	
Route	insert
Http Method	Post
Rules	- Group name is not empty or null - Group has no invites associated with it - The calling user is the only member in the group
Received Model	FrolfGroupModel
Returned Model	FrolfGroupModel
Description	Creates a course for a given group.
Notes	- The course model used to create the course is returned with its new ID.

Update	
Route	update
Http Method	Post
Rules	- Group name is not null or empty - The frolf group exists - The calling user is in the group - Calling user is the creator of the group
Received Model	FrolfGroupModel
Returned Model	n/a
Description	Updates the group
Notes	- Can update the group name

Get Frolf Group Members	
Route	groupMembers
Http Method	Post
Rules	- The group exists - The calling user is in the group that members are being requested for
Received Model	IdModel – Guid to retrieve group members for
Returned Model	IEnumerable<PlayerModel>

Description	Retrieves all group members for the requested group
Notes	- Can update the group name

Leave	
Route	leave
Http Method	Post
Rules	- The group exists - The calling user is in the group - The calling user is not the group creator - The calling user is not part of any games in progress for the group being left
Received Model	IdModel – Guid for the group to leave
Returned Model	n/a
Description	The calling user is removed from the group
Notes	

Remove Player	
Route	removePlayer
Http Method	Post
Rules	- The group exists - The calling user is the group creator - Player being removed is in the group - The calling user is not part of any games in progress for the group being left
Received Model	RemovePlayerModel
Returned Model	n/a
Description	The requested player is removed from the group
Notes	

2.5.3.5 FrolfGroupInviteController

Base Route: api/frolfgroupinvites

Get By Id	
Route	getById
Http Method	Post
Rules	- The invite must be for the calling user
Received Model	IdModel – Guid for the group requested
Returned Model	FrolfGroupInviteModel
Description	Gets the invite requested
Notes	

Get All	
Route	getAll
Http Method	Post
Rules	- The invites must be for the calling user

Received Model	n/a
Returned Model	IEnumerable<FrolfGroupInviteModel>
Description	Gets all the invites for the calling user
Notes	

Send	
Route	send
Http Method	Post
Rules	- Friend code must be for an existing user - Inviter is the calling user - Invite has not been sent already - Frolf group the invite is for exists - The calling user is a group admin - The invitee is not already in the group
Received Model	InviteCreationModel
Returned Model	n/a
Description	Creates a group invite
Notes	

Accept	
Route	accept
Http Method	Post
Rules	- Invite exists - Only the invitee can accept the invite
Received Model	IdModel – Guid of the invite being accepted
Returned Model	n/a
Description	Accepts the group invite.
Notes	- Accepting the invite will remove it from the database

Remove	
Route	delete
Http Method	Post
Rules	- Invite exists - Only the invitee can decline the invite
Received Model	IdModel – Guid of the invite to decline
Returned Model	n/a
Description	Declines the group invite.
Notes	- Declining the invite will remove it from the database

2.5.3.6 GameController

Base Route: api/games

Get By Id

Route	getById
Http Method	Post
Rules	- The calling user must be in the group the game being requested is for
Received Model	IdModel – Guid for the game requested
Returned Model	GameModel
Description	Gets the game requested
Notes	

Get All	
Route	getAll
Http Method	Post
Rules	- The games must be to a group the calling user belongs to
Received Model	n/a
Returned Model	IEnumerable<GameModel>
Description	Gets all the games for the calling user if the game is in a group the user belongs to
Notes	

Get With Filter	
Route	filter
Http Method	Post
Rules	- The calling user must be in the group the game is for to retrieve it
Received Model	GameFilter
Returned Model	IEnumerable<GameModel>
Description	Gets all the courses for groups the calling user is in
Notes	- Can filter by the state the game is in (InProgress or Completed)

Create Game	
Route	createGame
Http Method	Post
Rules	- The group the game is for must exist - The course the game is for must exist - There must be at least one player for the game - Game name is not null or empty - The calling user is in the group the game is for - The course is in the group the game is for
Received Model	GameCreationModel
Returned Model	GameModel
Description	Creates a new game based on the creation model
Notes	- Contains the newly created game with its new id

Update Game Holes

Route	holeScores
Http Method	Post
Rules	- The game must be in the InProgress state - The holes must be part of a game in a group that the calling user is part of - The holes being updated must exist - All holes being updated must be for the same game - All holes must be for the game the calling user specified - The calling user must be the one that created the game - No stroke counts can be below 1
Received Model	HoleScoreSetUpdateModel
Returned Model	n/a
Description	Updates the stroke count for each hole in the hole score update set.
Notes	

Get Game Results	
Route	results
Http Method	Post
Rules	- The calling user must be in the group the game is for to retrieve the game's results
Received Model	IdModel – Guid to retrieve results for
Returned Model	GameResultModel
Description	Retrieves results for all players that participated in the game
Notes	- Rankings included

Can Spectate Game	
Route	spectate
Http Method	Post
Rules	- The calling user must be in the group the game is for to be eligible to spectate it
Received Model	IdModel – Guid requesting to be spectated
Returned Model	BoolModel – True if the calling user can spectate, false otherwise
Description	Determines if the calling user is eligible to spectate the game requested
Notes	

Can Announce Game	
Route	announce
Http Method	Post
Rules	- The calling user must be the creator of the game
Received Model	IdModel – Guid requesting to be announce
Returned Model	BoolModel – True if the calling user can announce, false otherwise
Description	Determines if the calling user is eligible to announce the game requested
Notes	

Complete Game	
Route	complete
Http Method	Post
Rules	- The calling user must be the creator of the game - The game must be in the InProgress state
Received Model	IdModel – Guid of the game to complete
Returned Model	n/a
Description	Completes the requested game.
Notes	- GameState changes from InProgress -> Completed - Games cannot have their holescores modified.

2.5.3.7 HoleScoreController

Base Route: api/holescores

Get By Id	
Route	getById
Http Method	Post
Rules	- The calling user must be in the group the hole score is for to retrieve it
Received Model	IdModel – Guid for the hole score to retrieve
Returned Model	HoleScoreModel
Description	Gets the hole score requested
Notes	

Get All	
Route	getAll
Http Method	Post
Rules	- The calling user must be in the group the hole score is for to retrieve it
Received Model	n/a
Returned Model	IEnumerable<HoleScoreModel>
Description	Gets all the hole scores for the calling user
Notes	

Get With Filter	
Route	filter
Http Method	Post
Rules	- The calling user must be in the group the hole score is for to retrieve it
Received Model	HoleScoreFilter
Returned Model	IEnumerable<HoleScoreModel>
Description	Gets all the hole scores for the calling user
Notes	- Can filter by the game - Can filter by the tee

2.6 Broadcast Server Design

The broadcast server handles sending game updates from the client announcing to the clients spectating. Clients make requests to the server to join as either an Announcer (a client that sends game updates) or Spectator (a client that receives game updates). Client requests are validated before they can communicate with the server.

The server is multithreaded allowing client requests to be handled asynchronously:

- 1 thread listening for TCP connection requests
- A new thread for every connection being validated
- A new thread for every announcer connected
- A new thread for every game update sent to spectators

2.6.1 Classes

2.6.1.1 Broadcaster

The broadcaster listens for incoming TCP requests. It reads a configuration file to setup the listening IP and port. Each accepted request is passed off to the ClientHandler on a new thread to verify/handle the request. This keeps the listening port free to listen without interruption.

2.6.1.2 ClientHandler

This class handles a client request. Before handling the request, the request is validated for the following:

- The GetCurrentUserInfoInternal API call made to the API server, using the token provided from the client, successfully returns the client's relative AppUserModel
- The command the client is requesting to be handled is valid. The choices are:
 - SPECTATE – The client is requesting to watch a game
 - ANNOUNCE – The client is requesting to broadcast a game
- The command can be handled for the game the client is requesting the command for.

Any exception encountered during the validation process is caught and ignored so the server doesn't crash due to an invalid request.

If a client request is valid the command is handled.

Command Handling

Before any handling is done, the announcer list is locked for use.

- **Announce:** When the game is not being announced, a new AnnouncerClient is added for the requesting client making this command request. Announcing a game allows spectators to watch the game. In the case a game is being announced, the current AnnouncerClient accepts the requesting client as the new announcer.
- **Spectate:** Only games being announced can be watched. If the game the client is requesting to watch is being announced, a new SpectatorClient is created for the requesting client and it is added to the Announcer's list of spectators.

2.6.1.3 *AnnouncerClient*

Listens to clients that made Announce requests successfully. When a new AnnounceClient is created for a client, it triggers a new thread that listens to game updates sent by the client. The game updates are read, decrypted, then given to each spectator (SpectatorClient) in the announcer's spectator list. The spectator handles sending the update to its respective client.

2.6.1.4 *SpectatorClient*

Sends game updates to the client watching a game. When an announcer receives a game update, it broadcasts it to any listening spectators. Game updates to spectators are sent on new threads, so the announcer does not need to wait for a spectator to send its update before being able to broadcast to the next spectator.

Game updates are encrypted using the receiving clients RSA public key before being sent to the client.

2.6.1.5 *Messages*

Reads messages received from a NetworkStream.

2.6.1.6 *ApiContactor*

Makes calls to the API server. The calls made to help validate the requesting client is valid.

API Calls to the server:

- GetCurrentUserInfoInternal – To get the requesting client's user info
- CanSpectate – To determine if the requesting client can spectate the game
- CanAnnounce – To determine if the requesting client can announce the game

2.6.1.7 *ServerConfiguration*

Holds configurations for the server that are read in from a server configuration file. The configuration file allows the server to be setup without making code changes.

Configuration:

- ApiUrl – The API server's URL
- Ip – The IP the broadcaster server will listen on for client requests on
- Port – The Port the broadcaster is open on

Example Server Configuration File

```
{
  "apiUrl": "https://www.whatsfrolf.com/api/",
  "ip":     "174.31.4.4",
  "port":   45000
}
```

2.6.1.8 *ModelEncryptor*

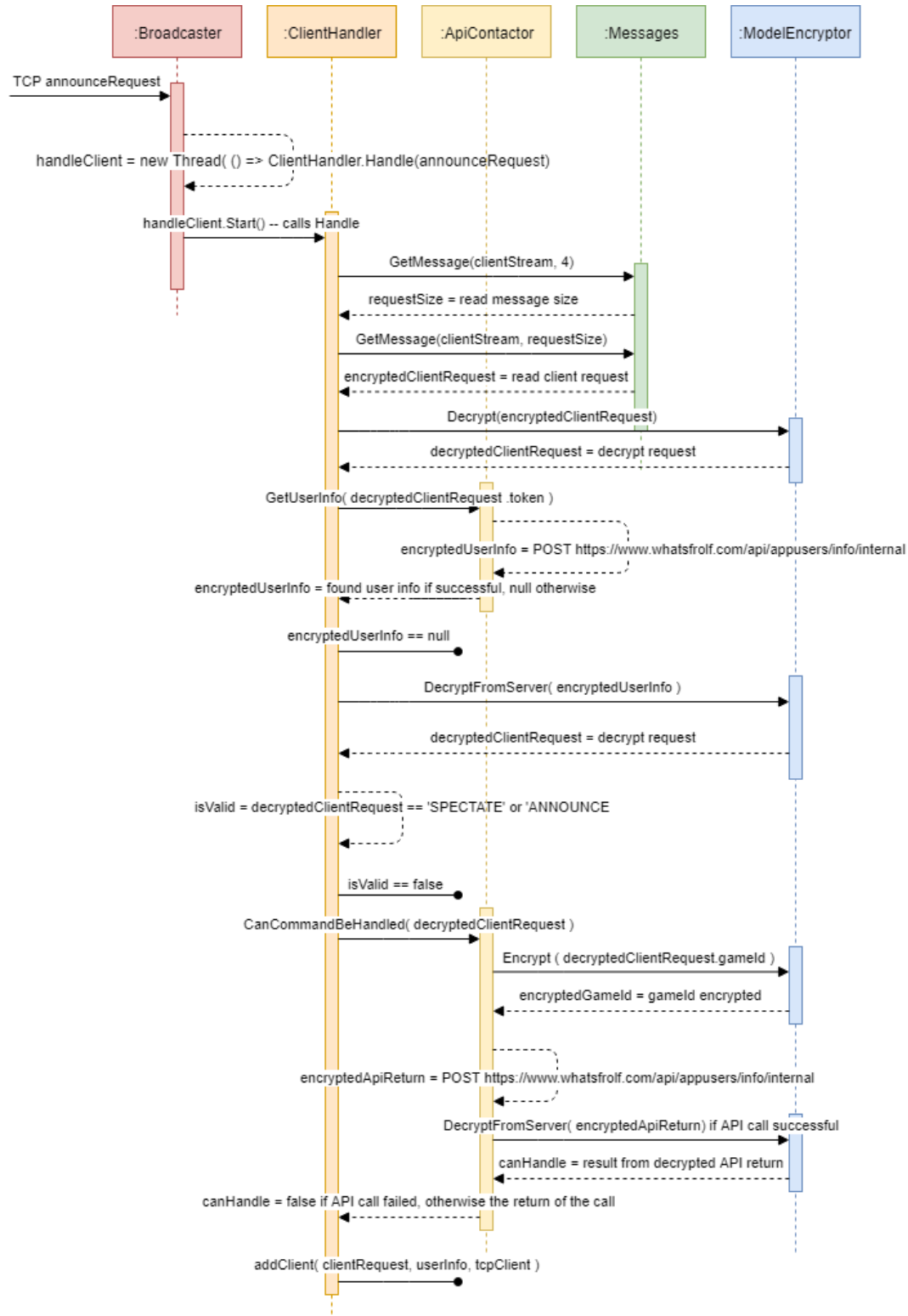
Encrypts models using the RSA/AES algorithm. This is a copy of the one on the API server and is explained in the API section in detail.

2.6.1.9 *Program*

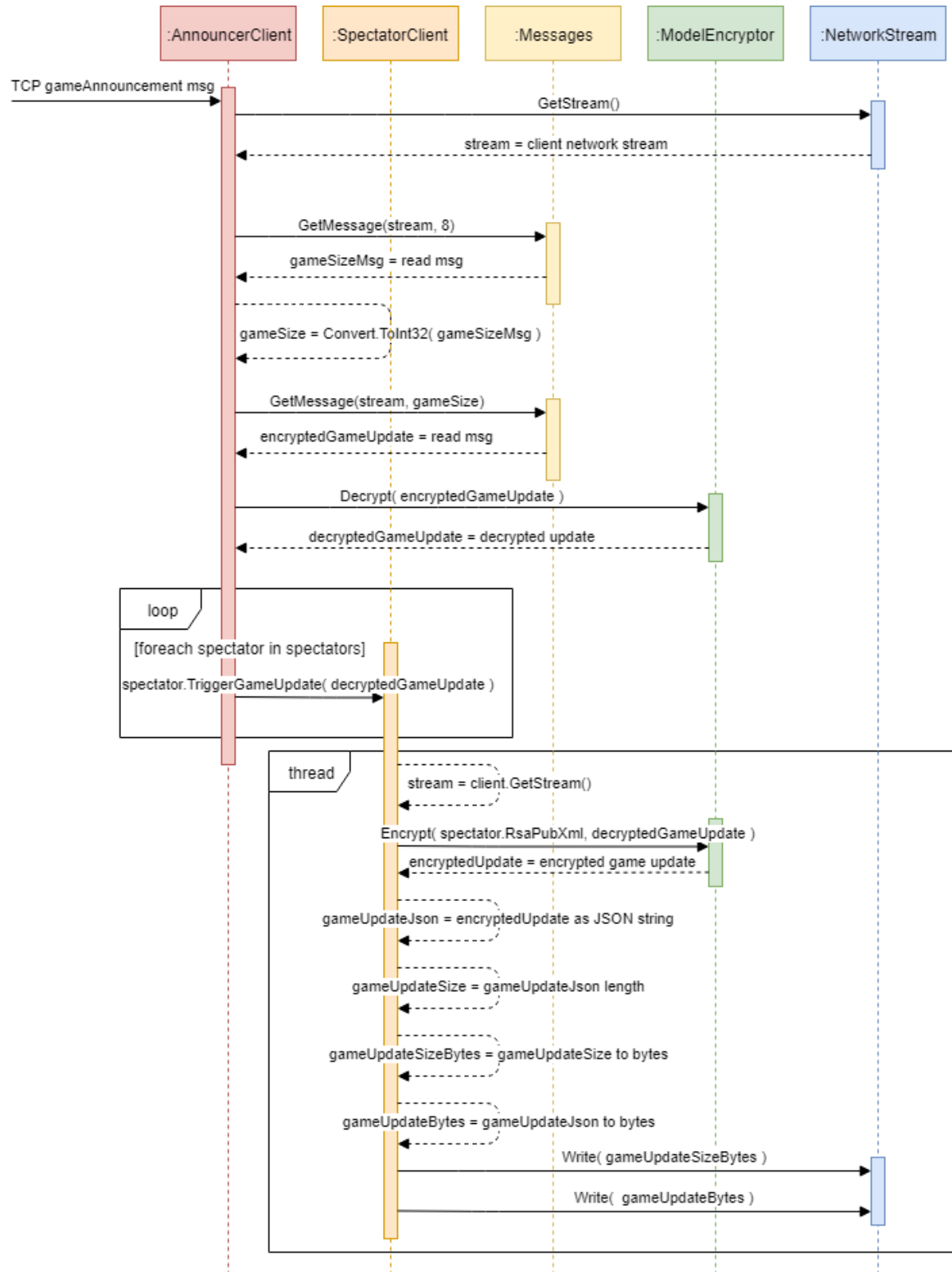
Entry point. Starts the broadcast server on a thread when the program is executed.

2.6.2 Sequence Diagrams

2.6.2.1 Client Validation Request

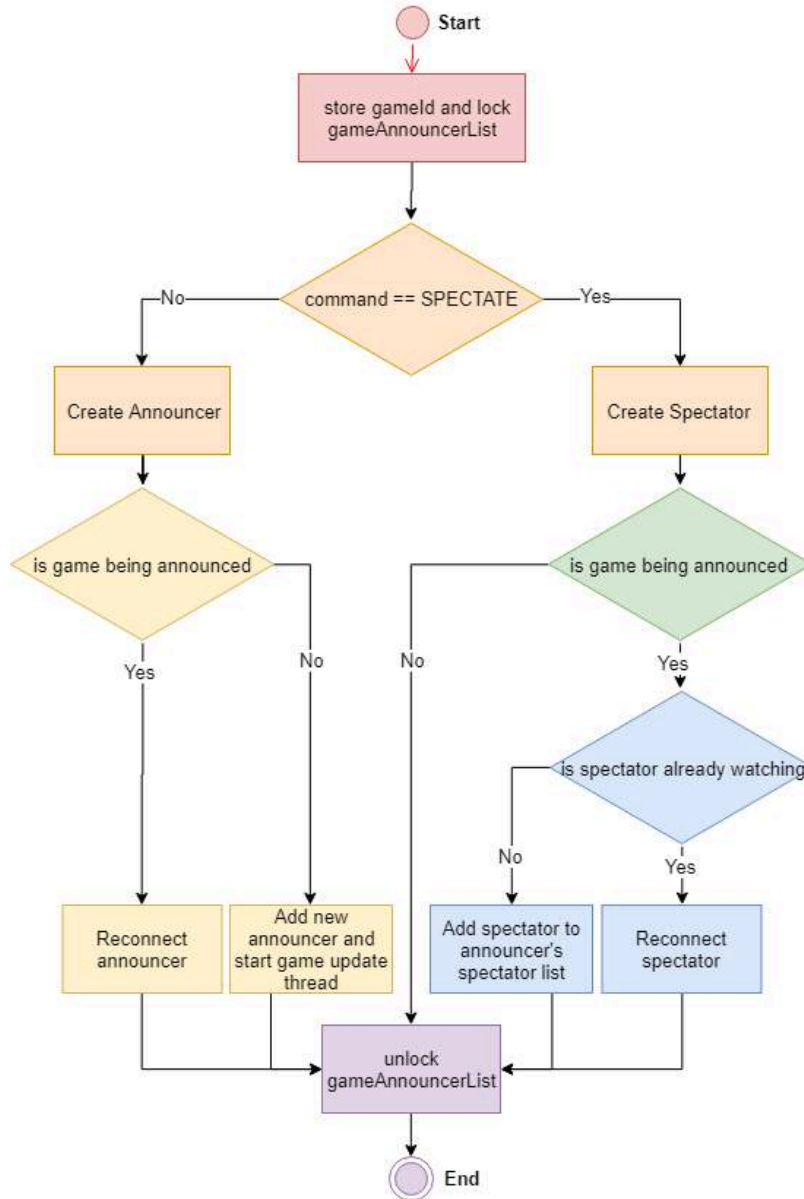


2.6.2.2 Game Broadcast



2.6.3 Flowcharts

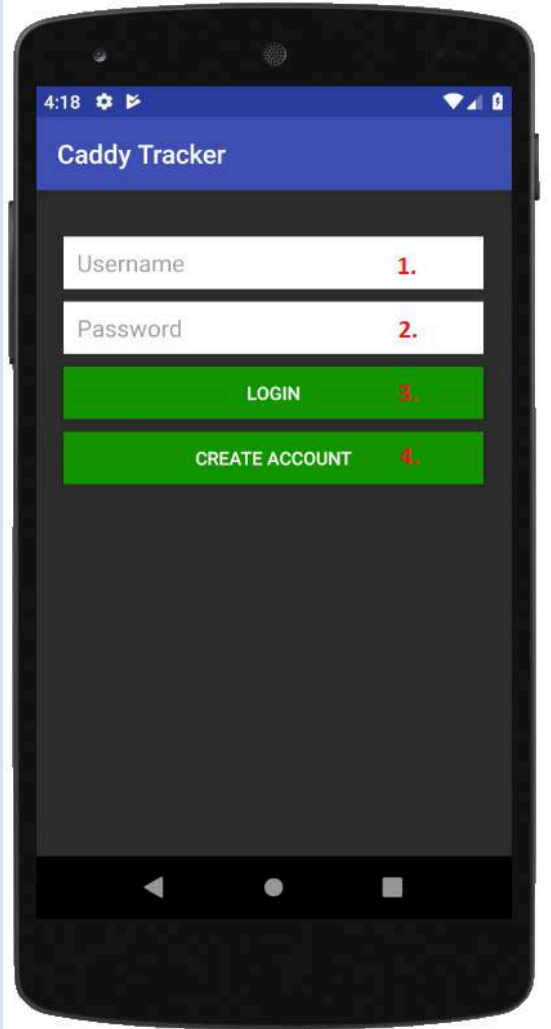
2.6.3.1 Add Announcer / Spectator



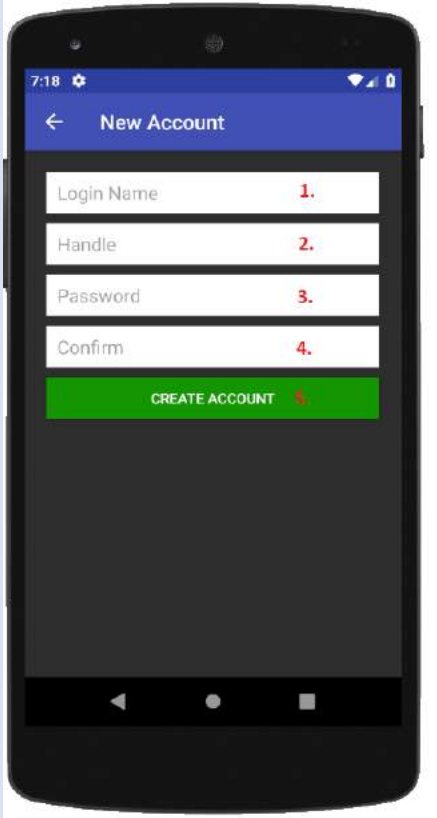
2.7 UI Design

2.7.1 Screens

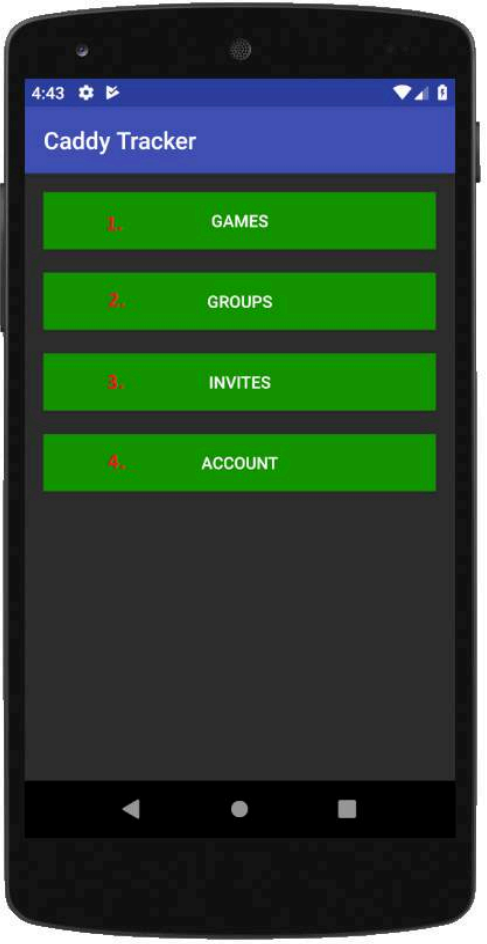
2.7.1.1 Login

Description	Screen
Allows the user to login or navigate to the account creation screen. The login attempt will fail if either the login name or password are invalid. 1. – Login name of the user you want to login as. 2. – Password of the user you want to login as. 3. – Button – Attempts login. Successful attempts bring the user to the main menu screen. 4. – Button – Brings the user to the account creation screen.	

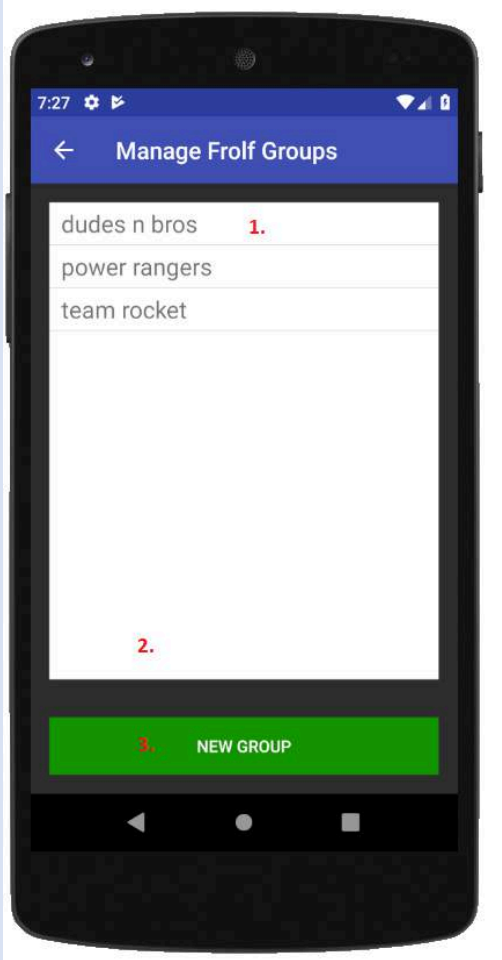
2.7.1.2 New Account

Description	Screen
Allows the user to create an account. The account attempt will not be complete if: - Login name is taken - Any input fields are too long - Password/Confirm input fields don't match Back button goes to the login screen. 1. – Login name input field 2. – Handle input field (the name shown to other user's in your groups) 3. – The password to login with 4. – The password retyped 5. – Button – Attempt to create the account. Successful attempts bring the user back to the login screen.	

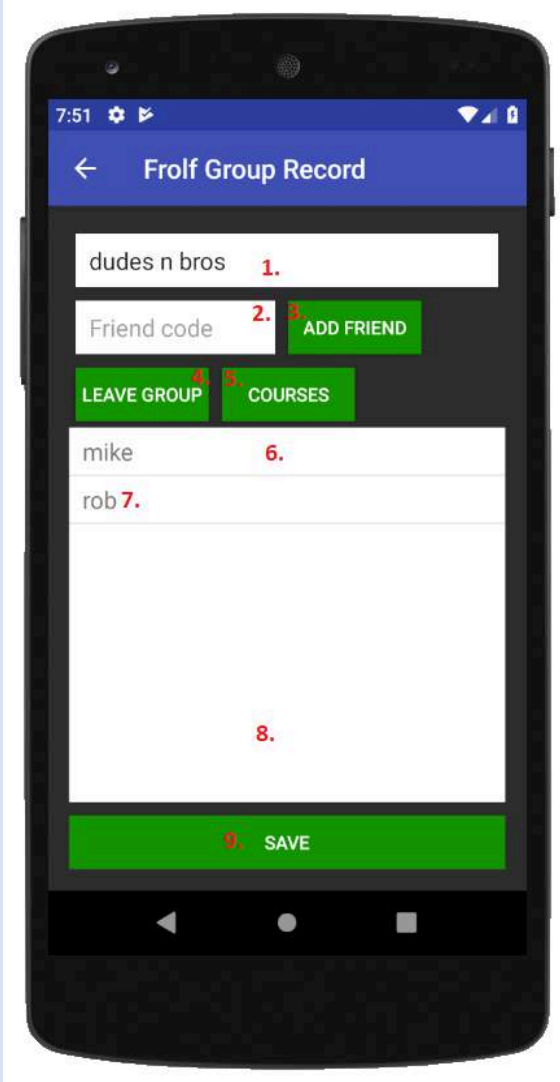
2.7.1.3 Main Menu

Description	Screen
Allows the user to access the main areas of the application. 1. – Button – Where a user goes to create, score, watch games as well as view game results. 2. – Button – Where a user goes to manage groups that they are in. 3. – Button – Where a user goes to accept/decline invites. 4. – Button – Where a user goes to view their account info.	

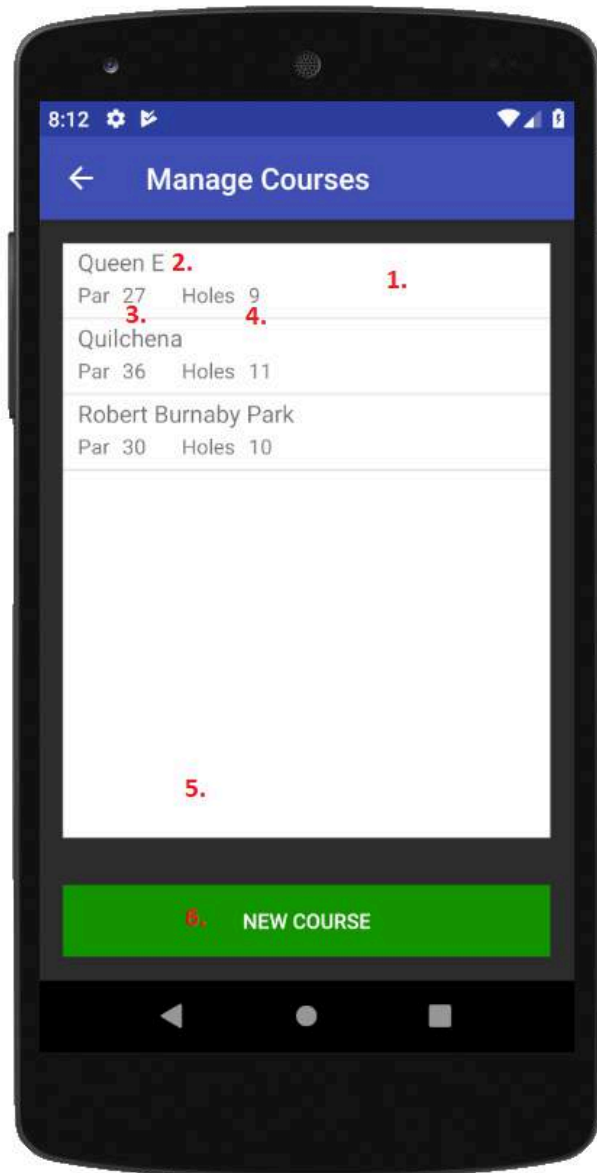
2.7.1.4 Groups

Description	Screen
Allows the user to navigate to a group they belong to or to navigate to group creation. Back button goes to main menu screen. 1. A group row in the group list fragment. Displays the name of the group. Clicking a row will navigate to the group record screen for the group. 2. Group list is ordered alphabetically. 3. – Button – Navigates to the group creation screen.	

2.7.1.5 Frolf Group Record

Description	Screen
Allows the user to manage a frolf group if they are the creator of the group. All frolf group members can leave the group and make courses for it. Back button goes to manage frolf group screen. 1. Name of the frolf group. Can be edited. 2. Friend code for the user a group invite will be sent to. 3. – Button – Attempts to add the friend associated with the friend code input. 4. – Button – Attempts to remove you from the group. This is not possible when you are the creator of the group. 5. – Button – Navigates to the manage course screen. 6. Member list fragment row. Click a row will attempt to remove the player from the group. Only the creator can do this, and they cannot remove themselves. 7. Name of the member the row is for. 8. Member list fragment. 9. Attempts to save the group. If the group is new, it will be created, otherwise updated. The frolf group name can be updated.	

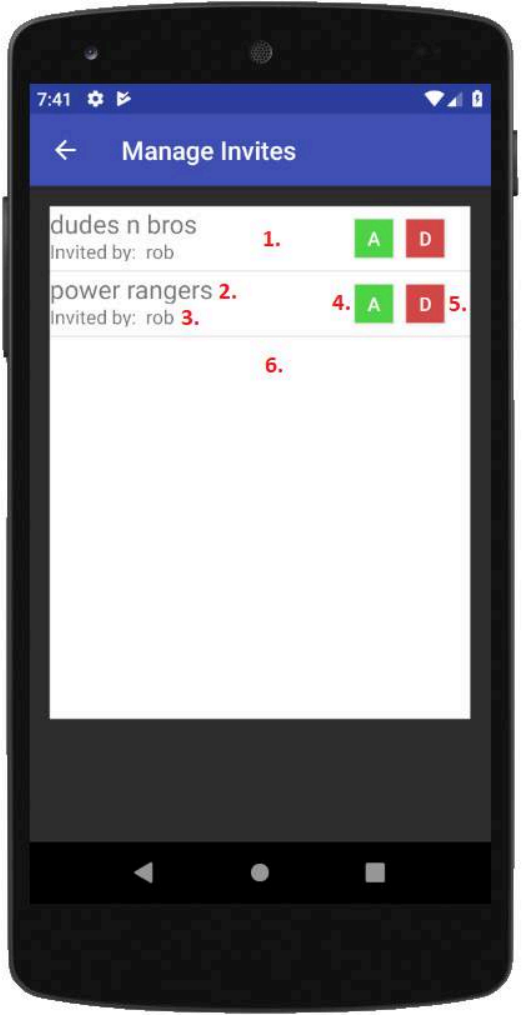
2.7.1.6 Manage Courses

Description	Screen												
Allows the user to manage courses in the frof group they accessed this screen from. All frof group members can create and edit courses that anyone in the group created. Back button goes to the frof group record screen. 1. Course fragment list row. Clicking opens the course record screen. 2. Name of the course. 3. Par for the course. 4. Number of holes for the course. 5. Course fragment list. 6. Navigates to the course record screen to create a course.	 The screenshot shows a mobile application interface titled "Manage Courses". At the top, there is a blue header bar with a back arrow and the title. Below the header, a list of golf courses is displayed. Each course entry includes the course name, its par value, and the number of holes. A green button at the bottom is labeled "NEW COURSE". <table><tr><th>Course Name</th><th>Par</th><th>Holes</th></tr><tr><td>Queen E</td><td>27</td><td>9</td></tr><tr><td>Quilchena</td><td>36</td><td>11</td></tr><tr><td>Robert Burnaby Park</td><td>30</td><td>10</td></tr></table>	Course Name	Par	Holes	Queen E	27	9	Quilchena	36	11	Robert Burnaby Park	30	10
Course Name	Par	Holes											
Queen E	27	9											
Quilchena	36	11											
Robert Burnaby Park	30	10											

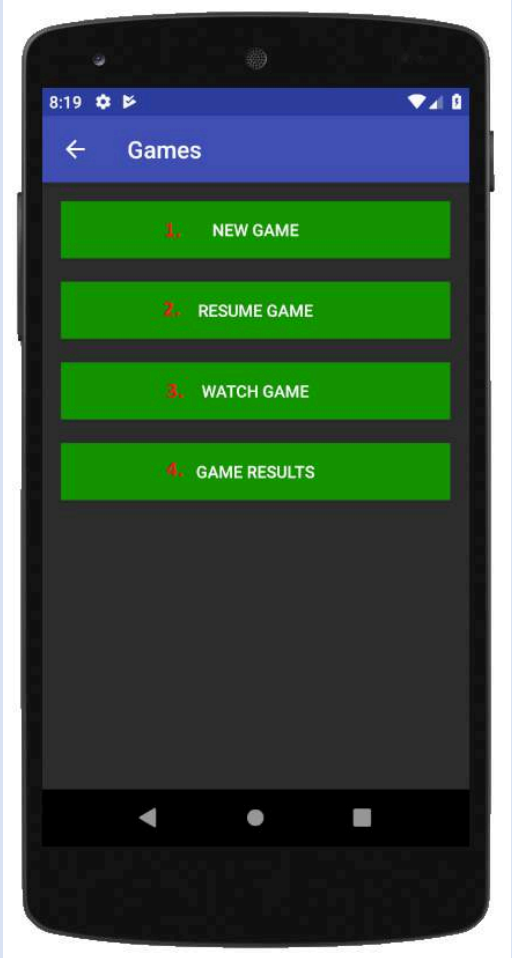
2.7.1.7 Course Record

Description	Screen
Allows the user to create a course that will reside within the frof group it is created from. Back button goes to the frof group record screen. 1. Name of the course. Can be edited. 2. – Button – Increase the hole count. Adds a row to the end of the hole list. 3. – Button – Decrease the hole count. Removes a row from the end of the hole list. 4. Hole fragment list row. Clicking doesn't do anything. 5. Tee number for the hole. 6. Par for the hole. 7. Increase the hole par. 8. Decrease the hole par. 9. Hole fragment list. 10. – Button – Saves the course. If it's new, it is created. Once created, only the course name can be modified.	

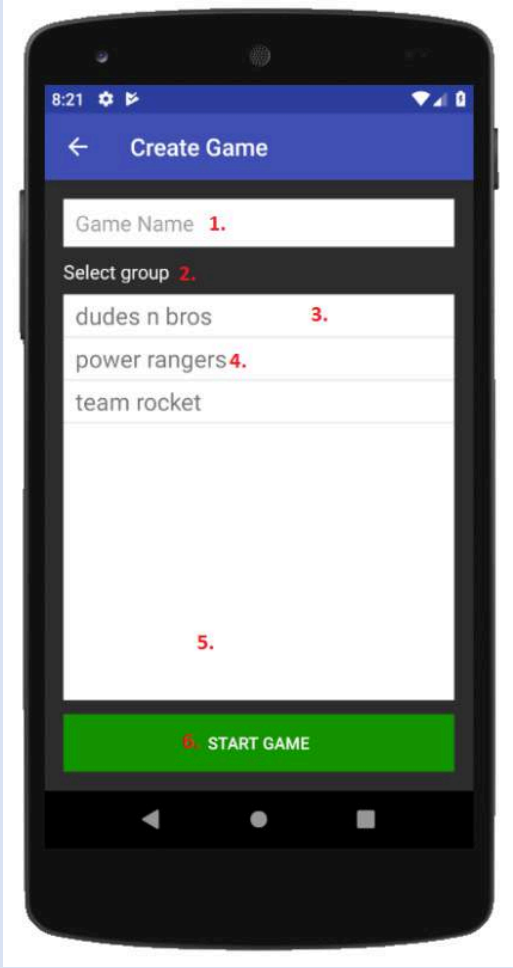
2.7.1.8 Manage Invites

Description	Screen
Allows the user to view, accept, and decline invites sent to them. Back button goes to the main menu screen. 1. Invite fragment list row. Clicking the row does nothing. 2. Name of the group the invite is to. 3. Name of the group member that invited you. 4. – Button – Attempts to accept the group invite. If successful, the invite is removed from the list and the user is added to the group. 5. – Button – Attempts to decline the group invite. If Successful, the invite is removed from the list and the user is not added to the group. 6. Invite fragment list.	

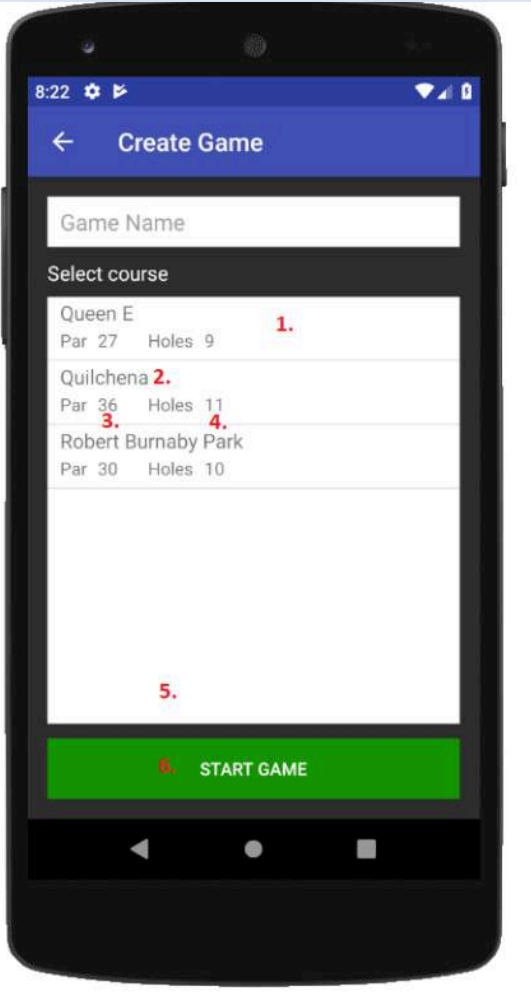
2.7.1.9 Manage Games

Description	Screen
Allows the user to access the game areas of the app. Back button goes to the main menu screen. 1. – Button – Navigates to the game creation screen. 2. – Button – Navigates to resume the game screen. 3. – Button – Navigates to the watch game screen. 4. – Button – Navigates to the game results screen.	

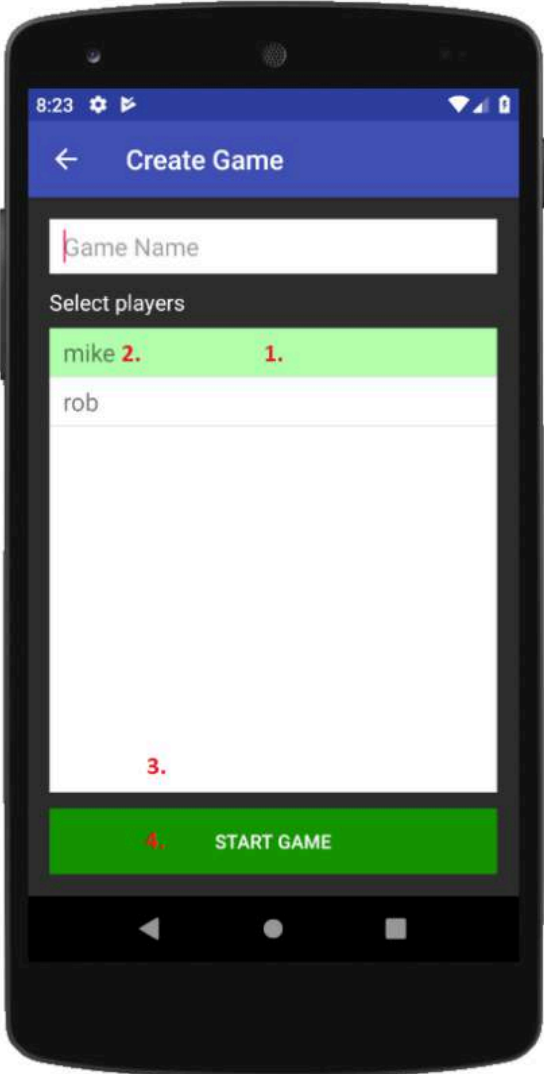
2.7.1.10 Create Game Screen – 01. Choose Group

Description	Screen
Allows the user to create a game. This is the 1st of 3 screens involved in the game creation process: Selecting a group the game is for. Back button goes to the manage games screen. 1. Name of the game being created. 2. The current action to take when creating a game. Currently – selecting a group the game is for. 3. Group fragment list row. Clicking it selects the group the game will be for. Selecting the group switches to the course selection. 4. Name of the game 5. Group list fragment. Only displays groups the user is part of. The list is alphabetically ordered. 6. – Button – Attempts to start the game. Cannot be pressed until the last screen involved in the game creation process is reached.	

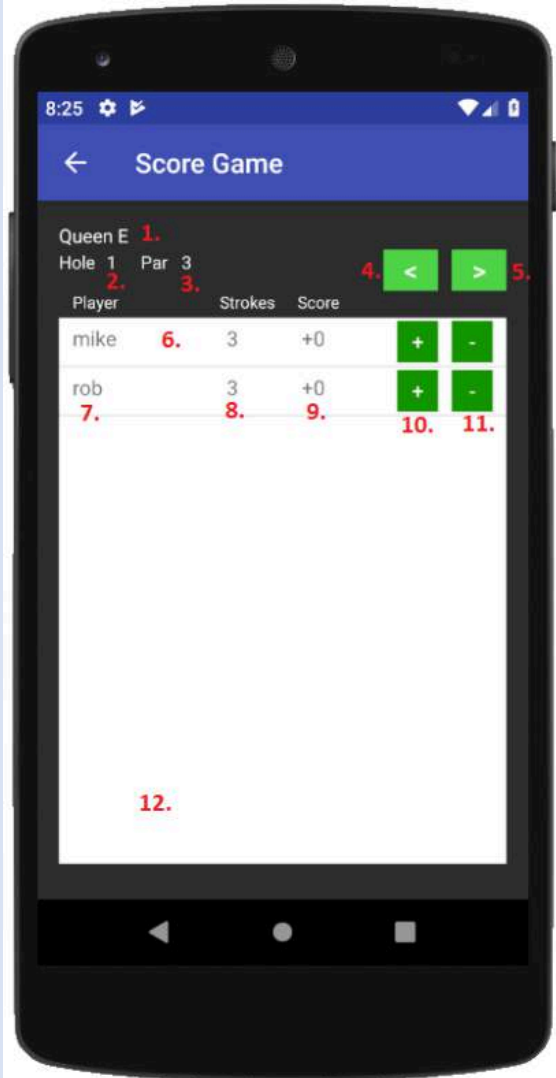
2.7.1.11 Create Game Screen – 02. Choose Course

Description	Screen
Allows the user to select a course for the game. This is the 2nd of 3 screens involved in the game creation process. Back button goes to the manage games screen. 1. Course fragment list row. Clicking selects the course for the game. Selecting the course switches to the player selection. 2. Name of the course. 3. Par for the course. 4. Hole count of the course. 5. Course fragment list. Only displays courses associated with the selected group. The list is alphabetically ordered. 6. – Button – Attempts to start the game. Cannot be pressed until the last screen involved in the game creation process is reached.	 The screenshot shows a mobile application interface titled 'Create Game'. At the top is a blue header bar with a back arrow and the title. Below the header is a text input field for 'Game Name'. Underneath is a section titled 'Select course' containing a list of golf courses. The list items are: 'Queen E' (with a red '1.' next to it), 'Par 27 Holes 9', 'Quilchena' (with a red '2.' next to it), 'Par 36 Holes 11' (with a red '3.' next to 'Par 36'), 'Robert Burnaby Park' (with a red '4.' next to it), and 'Par 30 Holes 10'. Below the list is a red '5.' indicating the next step. At the bottom of the screen is a green button with a red '6.' and the text 'START GAME'. The phone's status bar at the very top shows the time as 8:22 and various icons.

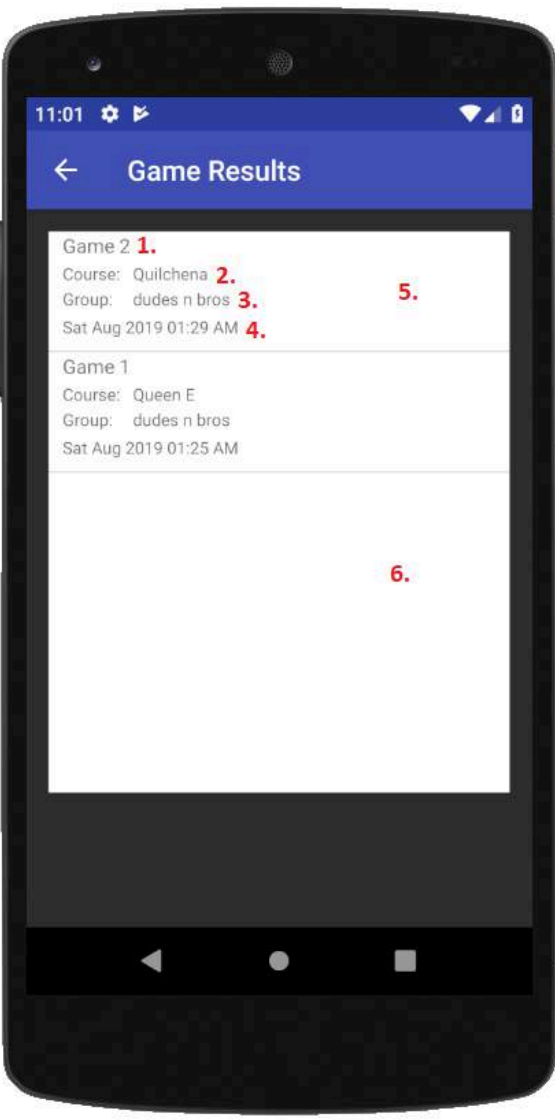
2.7.1.12 Create Game Screen – 03. Choose Players

Description	Screen
Allows the user to select players for the game. This is the 3rd of 3 screens involved in the game creation process. Back button goes to the manage games screen. 1. Member fragment list row. Clicking selects the member for the game. Players can be unselected. Green signifies they are currently selected. 2. Name of the member. 3. Member fragment list. The list is alphabetically ordered. 4. – Button – Attempts to start the game. The game won't start unless a minimum of one player is selected. If successful, the app will navigate to the score screen.	 The screenshot shows a mobile application interface titled 'Create Game'. At the top, there is a blue header bar with a back arrow and the title. Below the header is a text input field labeled 'Game Name'. Underneath is a section titled 'Select players'. It contains a list of player names: 'mike' and 'rob'. The 'mike' row is highlighted in green, indicating it is selected. To the right of the names are red numbers: '2.' next to 'mike' and '1.' next to 'rob'. At the bottom of the screen is a large green button labeled 'START GAME' with a red number '4.' to its left. The status bar at the very top shows the time as 8:23 and various system icons.

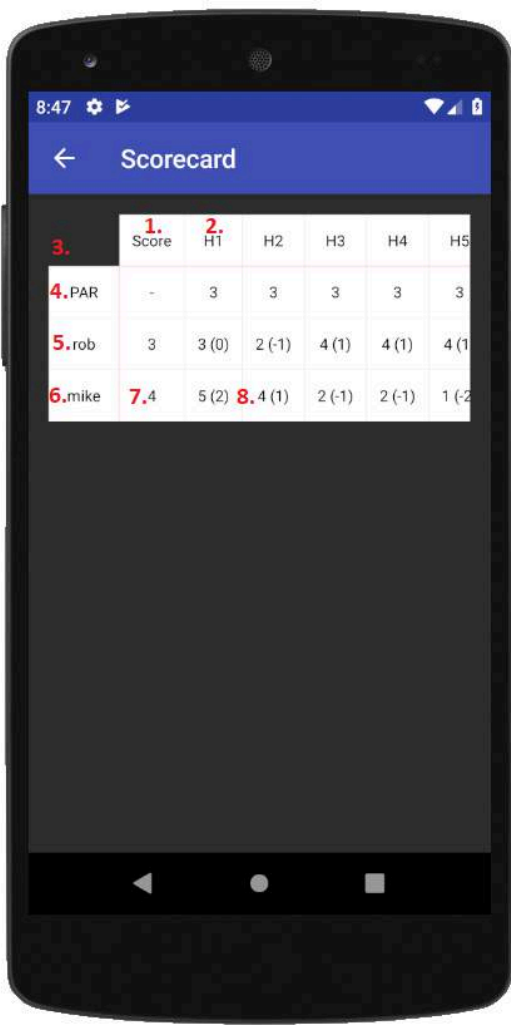
2.7.1.13 Score Game

Description	Screen
Allows to score the game hole by hole. Back button goes to the create game screen. 1. Name of the course being played. 2. The current tee being shown. 3. Par for the current hole. 4. – Button – Goes to the previous hole, if it is the first hole, does nothing. 5. – Button – Goes to the next hole. If it is the last hole, clicking this button will ask if the scorer wants to complete the game. 6. Hole score fragment list row. Clicking it does nothing. 7. Name of the player the hole score is for. 8. Strokes the player has on the current hole. 9. Score the player has on the current hole. 10. – Button – Increase the stroke count for the player. 11. – Button – Decreases the stroke count for the player. 12. Hole score fragment list. The list is displayed in alphabetical order.	

2.7.1.14 Game Results

Description	Screen
Allows the user to view games that have been completed and select one to view results for. Back button goes to the manage game. 1. Name of the game. 2. Course the game was played on. 3. Group the game was played with. 4. Date the game was created. 5. Game fragment list row. Clicking navigates to the game results screen for the selected game. 6. Game fragment list. Displays games that are completed and associated with any group the user is part of. The list is ordered from the newest to oldest created.	

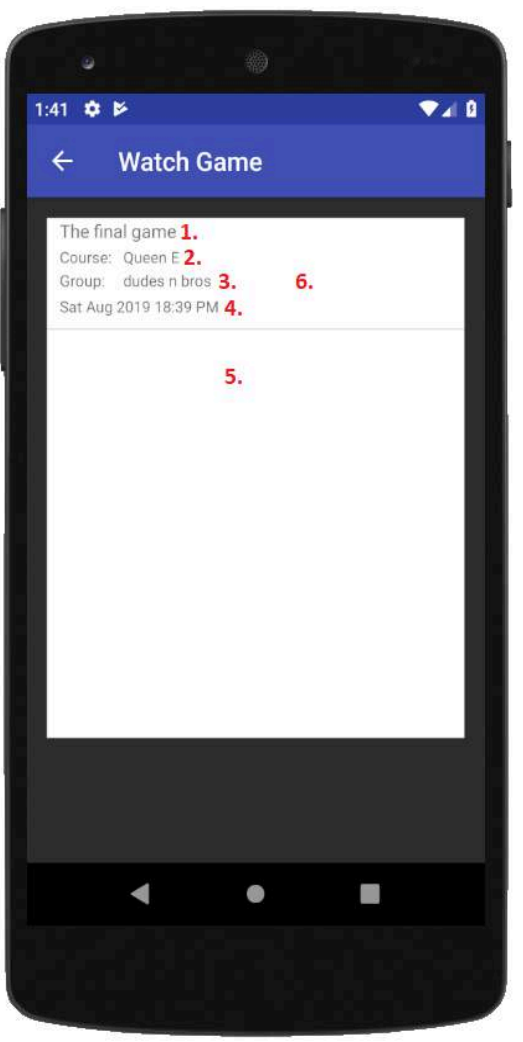
2.7.1.15 Scorecard

Description	Screen
Allows the user to view the game results for a game. Back button goes to the game results screen. - Score for the players. - Hole scores for each player H1...Hn. - Scorecard table. Can be scrolled up/down, left/right. The order goes from first to last. - Hole pars. - Name of the player. - Name of the player. - The player's total score. - Hole score for the player.	

2.7.1.16 Resume Game

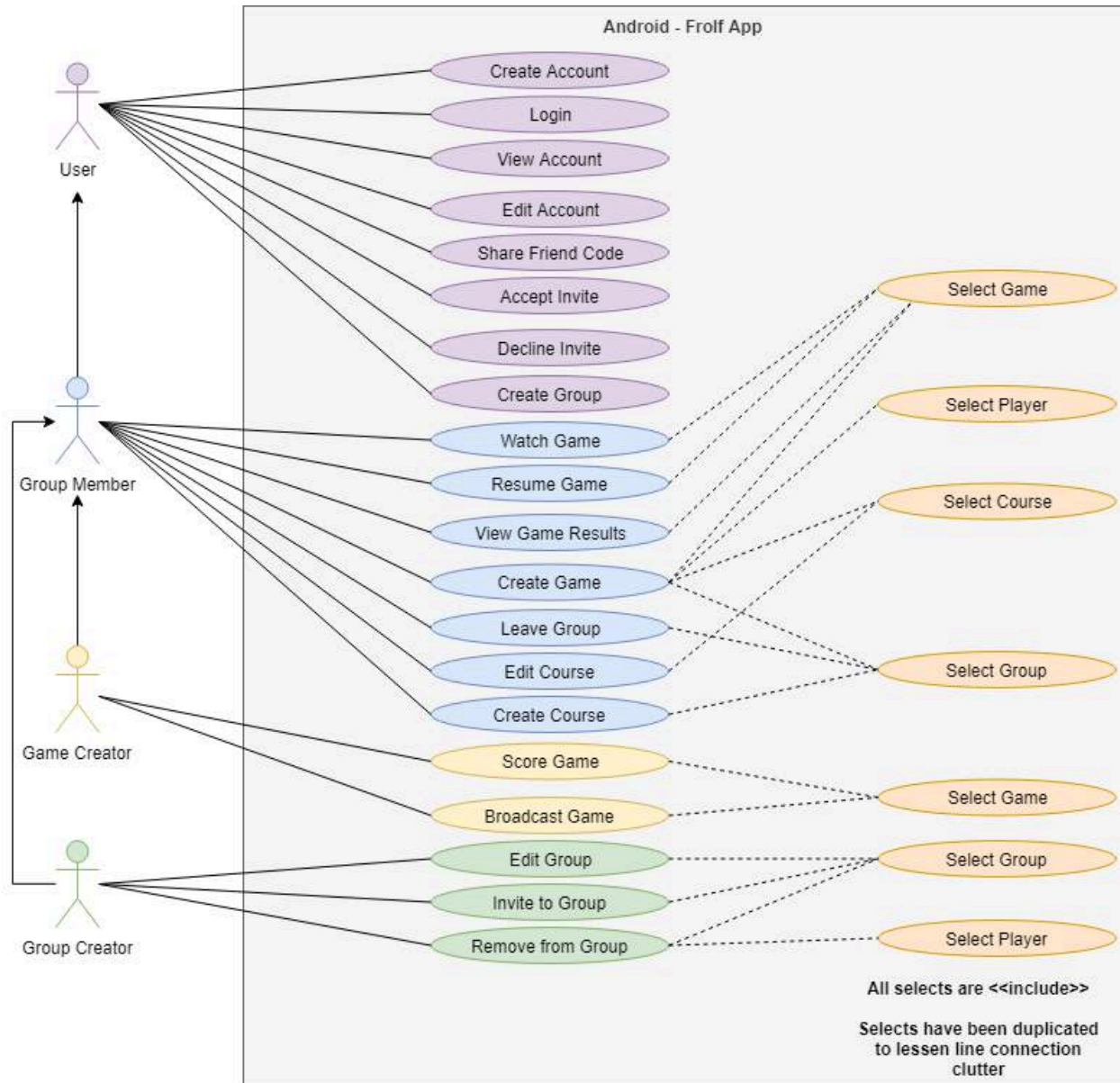
Description	Screen
Allows the user to resume a game that is not completed. Back button goes to the manage game screen. 1. Name of the game. 2. Course the game is played on. 3. Group the game is played with. 4. Date the game was created. 5. Game fragment list. Only displays games that have not been completed and is in a group the user belongs to. Games are displayed from newest to oldest created. 6. Game fragment list row. Clicking opens the score screen for the selected game.	

2.7.1.17 Watch Game

Description	Screen
Allows the user to watch a game that is not completed. Back button goes to the manage game screen. 1. Name of the game. 2. Course the game is played on. 3. Group the game is played with. 4. Date the game was created. 5. Game fragment list. Only displays games that have not been completed and is in a group the user belongs to. Games are displayed from newest to oldest created. 6. Game fragment list row. Clicking opens the game watch screen.	

2.7.2 Use Cases

2.7.2.1 Diagram



2.7.2.2 Analysis

2.7.2.2.1 User

A basic user who is using the application.

2.7.2.2.1.1 Create Account

2.7.2.2.1.1.1 Introduction

As a user, I want to create an account so that I can log into the application.

2.7.2.2.1.1.2 Inputs

- Login name
- Handle

- Password
- Confirm Password

2.7.2.2.1.1.3 *Processing*

1. User shall open the app and navigate to the account creation screen.
2. User shall enter valid inputs.
3. User shall click the create account button.
4. Data inputs are saved to the database.

2.7.2.2.1.1.4 *Outputs*

- If successful, the user shall be brought to the login screen.

2.7.2.2.1.1.5 *Error Handling*

- Errors result in the account being unable to be created
- Possible errors:
 - Login name is taken
 - Password does not match the confirm password

2.7.2.2.1.2 *Login*

2.7.2.2.1.2.1 *Introduction*

As a user, I want to log into the application.

2.7.2.2.1.2.2 *Inputs*

- Login name
- Password

2.7.2.2.1.2.3 *Processing*

1. User shall open the app to the login screen.
2. User shall enter a valid login name and password combination.
3. User shall click the login button.

2.7.2.2.1.2.4 *Outputs*

- If successful, the user shall be brought to the main menu screen.

2.7.2.2.1.2.5 *Error Handling*

- Errors result in a failed login
- Possible errors:
 - Invalid login name and password combination

2.7.2.2.1.3 *View Account*

2.7.2.2.1.3.1 *Introduction*

As a user, I want to view my account information.

2.7.2.2.1.3.2 *Inputs*

- n/a

2.7.2.2.1.3.3 *Processing*

1. User shall navigate to the account info screen.

2.7.2.2.1.3.4 *Outputs*

- User shall see their:
 - Login name
 - Handle
 - Friend code

2.7.2.2.1.3.5 *Error Handling*

- n/a

2.7.2.2.1.4 *Edit Account*

2.7.2.2.1.4.1 *Introduction*

As a user, I want to be able to edit my account information.

2.7.2.2.1.4.2 *Inputs*

- Login name
- Handle

2.7.2.2.1.4.3 *Processing*

1. User shall open the app to the account info screen.
2. User shall edit their login name and handle.
3. User shall click the save button.

2.7.2.2.1.4.4 *Outputs*

- A success message will be displayed.

2.7.2.2.1.4.5 *Error Handling*

- Blank name or handle will result in a failed account edit.

2.7.2.2.1.5 *Share Friend Code*

2.7.2.2.1.5.1 *Introduction*

As a user, I want to share my friend code with others without manually typing it.

2.7.2.2.1.5.2 *Inputs*

- n/a

2.7.2.2.1.5.3 *Processing*

1. User shall navigate to their account info screen.
2. User shall click the share button.
3. User shall select the application of their choice to send the friend code (WhatsApp, Facebook Messenger, etc.)

2.7.2.2.1.5.4 *Outputs*

- The friend code is sent using the application the user chose.

2.7.2.2.1.5.5 *Error Handling*

- n/a

2.7.2.2.1.6 Accept Invite

2.7.2.2.1.6.1 Introduction

As a user, I want to accept a group invite so that I can join the group.

2.7.2.2.1.6.2 Inputs

- n/a

2.7.2.2.1.6.3 Processing

1. User shall navigate to the manage invite screen.
2. User shall click the accept “A” button.

2.7.2.2.1.6.4 Outputs

- If successful, the user is put into the group and the invite is removed.

2.7.2.2.1.6.5 Error Handling

- n/a

2.7.2.2.1.7 Decline Invite

2.7.2.2.1.7.1 Introduction

As a user, I want to decline a group invite so that I am not put into the group.

2.7.2.2.1.7.2 Inputs

- n/a

2.7.2.2.1.7.3 Processing

1. User shall navigate to the manage invites screen.
2. User shall click the decline “D” button.

2.7.2.2.1.7.4 Outputs

- If successful, the user is not put into the group and the invite is removed.

2.7.2.2.1.7.5 Error Handling

- n/a

2.7.2.2.1.8 Create Group

2.7.2.2.1.8.1 Introduction

As a user, I want to create a group so that I can add friends to the group.

2.7.2.2.1.8.2 Inputs

- Group name

2.7.2.2.1.8.3 Processing

1. User shall navigate to the group creation screen.
2. User shall input a group name.
3. User shall click the save button.

2.7.2.2.1.8.4 Outputs

- If successful, the group is created. When the group is created, the user who created the group becomes the group creator and is added to the group.

2.7.2.2.1.8.5 *Error Handling*

- Errors result in the group not being able to be created.
- Errors include:
 - A group name that is empty

2.7.2.2.2 *Group Member*

A group member is a user that belongs to a group. They can operate within the group.

2.7.2.2.2.1 *Watch Game*

2.7.2.2.2.1.1 *Introduction*

As a group member, I want to watch a game be scored.

2.7.2.2.2.1.2 *Inputs*

- n/a

2.7.2.2.2.1.3 *Processing*

1. User shall navigate to the watch game screen.
2. User shall select a game they want to watch be scored.

2.7.2.2.2.1.4 *Outputs*

- User is brought to the scorecard screen for the selected game. A connection is made to the broadcast server. If successful, the user will automatically receive updates after the scorer updates a hole.

2.7.2.2.2.1.5 *Error Handling*

- Errors result in the game being viewed to not receive score updates
- Errors include:
 - The game not being broadcasted

2.7.2.2.2.2 *Resume Game*

2.7.2.2.2.2.1 *Introduction*

As a group member, I want to resume a game so I can see the game hole by hole.

2.7.2.2.2.2.2 *Inputs*

- n/a

2.7.2.2.2.2.3 *Processing*

1. User shall navigate to the resume game screen.
2. User shall select a game to resume.
 - a. If the user is the game creator, they will be able to score the game.

2.7.2.2.2.2.4 *Outputs*

- User is brought to the score game screen for the selected game.

2.7.2.2.2.2.5 *Error Handling*

- n/a

2.7.2.2.2.3 View Game Results

2.7.2.2.2.3.1 Introduction

As a group member, I want to view the scorecard of a completed game.

2.7.2.2.2.3.2 Inputs

- n/a

2.7.2.2.2.3.3 Processing

1. User shall navigate to the game results screen.
2. User shall select a game to view the results for.

2.7.2.2.2.3.4 Outputs

- User is brought to the scorecard screen for the game selected and the scorecard for the selected game is displayed.

2.7.2.2.2.3.5 Error Handling

- n/a

2.7.2.2.2.4 Create Game

2.7.2.2.2.4.1 Introduction

As a group member, I want to create a game so that I can score it.

2.7.2.2.2.4.2 Inputs

- Game name
- Selected group
- Selected course
- Selected group members

2.7.2.2.2.4.3 Processing

1. User shall navigate to the create game screen.
2. User shall input a game name.
3. User shall select a group.
4. User shall select a course.
5. User shall select members that are playing the game.

2.7.2.2.2.4.4 Outputs

- If successful, the user is brought to the score game screen.

2.7.2.2.2.4.5 Error Handling

- Errors result in the game not being created.
- Errors include
 - A game name not being provided

2.7.2.2.2.5 Leave Group

2.7.2.2.2.5.1 Introduction

As a group member, I want to leave a group that I am in.

2.7.2.2.2.5.2 Inputs

- n/a

2.7.2.2.2.5.3 *Processing*

1. User shall navigate to the group the wish to leave.
2. User shall click the leave button.

2.7.2.2.2.5.4 *Outputs*

- If successful, the user is removed from the group and all their group data is deleted.

2.7.2.2.2.5.5 *Error Handling*

- Errors result in the user not being able to leave the group
- Errors include:
 - The user is currently in a game being scored.

2.7.2.2.2.6 *Edit Course*

2.7.2.2.2.6.1 *Introduction*

As a group member, I want to edit course details.

2.7.2.2.2.6.2 *Inputs*

- Course name

2.7.2.2.2.6.3 *Processing*

1. User shall navigate to the course they want to edit.
2. User shall input a new course name.
3. User shall click save.

2.7.2.2.2.6.4 *Outputs*

- The course's name is changed.

2.7.2.2.2.6.5 *Error Handling*

- Errors result in the name not being updated.
- Errors include:
 - The name being empty.

2.7.2.2.2.7 *Create Course*

2.7.2.2.2.7.1 *Introduction*

As a group member, I want to create a course in a group I'm in.

2.7.2.2.2.7.2 *Inputs*

- Course name
- Hole count selection (add/remove) – can't go below 1.
- Hole par selection (increase/decrease) – can't go below 1.

2.7.2.2.2.7.3 *Processing*

1. User shall navigate to the course creation screen.
2. User shall enter a name for the course.
3. User shall set the hole count for the course.
4. User shall set the hole par for the course.

2.7.2.2.2.7.4 *Outputs*

- If successful, the new course is created which removes the ability to modify the hole count and par count. The course name is still editable.

2.7.2.2.2.7.5 *Error Handling*

- Errors result in the course not being created.
- Errors include:
 - The name being empty

2.7.2.2.3 *Game Creator*

A group member who has created a game.

2.7.2.2.3.1 *Score Game*

2.7.2.2.3.1.1 *Introduction*

As the game creator, I want to score a game I created.

2.7.2.2.3.1.2 *Inputs*

- Player score updates – cannot be below 1 stroke.

2.7.2.2.3.1.3 *Processing*

1. User shall create a new game or resume a game in progress.
2. User shall update player scores hole by hole.
3. User shall complete a game by clicking the next button on the last hole of the game.
4. User shall confirm they want to complete the game.

2.7.2.2.3.1.4 *Outputs*

- As the user progresses through the holes, they are saved.

2.7.2.2.3.1.5 *Error Handling*

- n/a

2.7.2.2.3.2 *Broadcast Game*

2.7.2.2.3.2.1 *Introduction*

As a game creator, I want to broadcast my game so that others in the group can receive live updates.

2.7.2.2.3.2.2 *Inputs*

- n/a

2.7.2.2.3.2.3 *Processing*

- User shall create a new game or resume a game in progress.

2.7.2.2.3.2.4 *Outputs*

- Behind the scenes, a connection to the broadcast server is established and data is transmitted to it after each hole is scored.

2.7.2.2.3.2.5 *Error Handling*

- Errors result in the game being unable to be broadcasted.
 - Errors include the connection to the broadcast server failing. This will not stop the game from being able to be scored.

2.7.2.2.4 Group Creator

The creator of a group.

2.7.2.2.4.1 Edit Group

2.7.2.2.4.1.1 Introduction

As a group creator, I want to edit the group details.

2.7.2.2.4.1.2 Inputs

- Group name

2.7.2.2.4.1.3 Processing

1. User shall navigate to the group they want to edit.
2. User shall modify the group name.
3. User shall click save.

2.7.2.2.4.1.4 Outputs

- If successful, the group name will be changed.

2.7.2.2.4.1.5 Error Handling

- Errors result in the game name not being changed.
- Errors include:
 - The group name being empty

2.7.2.2.4.2 Invite to Group

2.7.2.2.4.2.1 Introduction

As a group creator, I want to invite a user to my group so I can play games with them.

2.7.2.2.4.2.2 Inputs

- The friend code of the user to invite

2.7.2.2.4.2.3 Processing

1. User shall navigate to the group the want to invite to.
2. User shall enter the friend code of the user to invite.
3. User shall click the add friend button to send the invite.

2.7.2.2.4.2.4 Outputs

- If successful, the invite will be sent to the user.

2.7.2.2.4.2.5 Error Handling

- Errors result in the invite not sending.
- Errors include:
 - A friend code that doesn't exist
 - A friend code for a user already in the group
 - A friend code for a user that already has a pending invite

2.7.2.2.4.3 Remove from Group

2.7.2.2.4.3.1 Introduction

As a group creator, I want to remove a user from the group, so they no longer belong to the group.

2.7.2.2.4.3.2 *Inputs*

- n/a

2.7.2.2.4.3.3 *Processing*

1. User shall navigate to the group they want to remove a user from.
2. User shall click the user they want to remove.
3. User shall confirm they want to remove the user.

2.7.2.2.4.3.4 *Outputs*

- If successful, the user is removed from the group and their group data is deleted.

2.7.2.2.4.3.5 *Error Handling*

- Errors result in the user not being removed from the group.
- Errors include:
 - The user is the creator of the group.

2.7.3 Encryption

The android application encrypts/decrypts just as the model encryptor on the API server and broadcast server does only when it's written in Java.

RSA Public/Private Key Setting

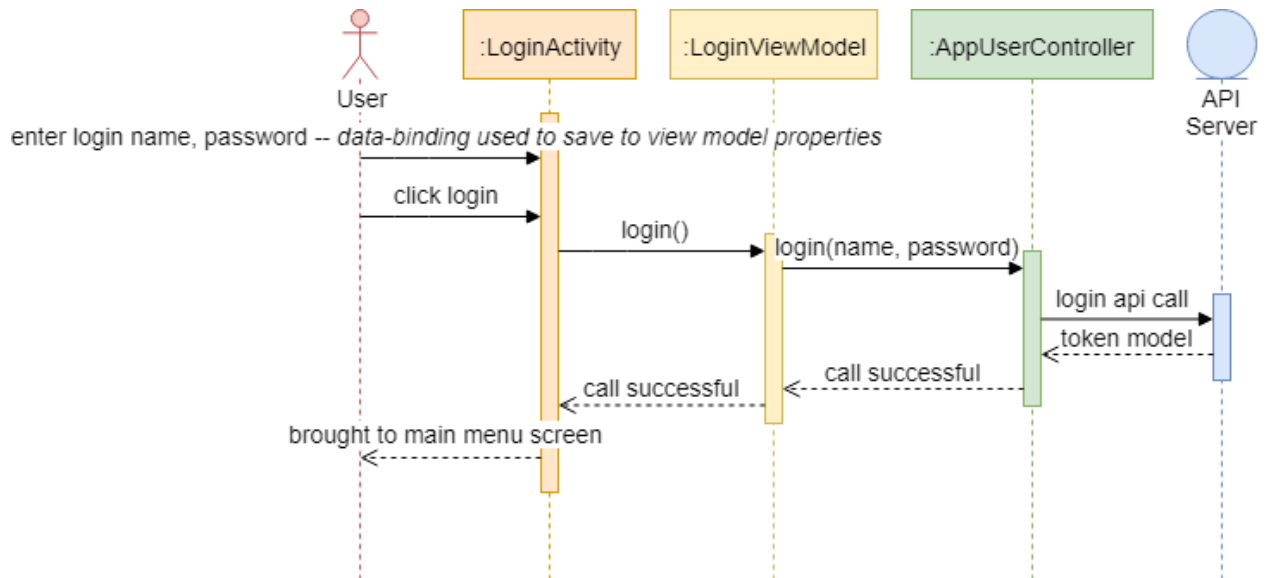
There is an API call to set a user's RSA public key. This API call is made every time the user logs in. It is up to the Client applications how often they want to update their RSA keys.

2.7.4 UI Code Structure

2.7.4.1 *Model-View-ViewModel Pattern*

The frontend application uses a model-view-viewmodel pattern (MVVM). The model holds the data, the view allows interaction with and display of the model, and the view model performs actions to send/retrieve data from the API. All android activities/fragments have a specific view model that acts on a model. The view models use various API controllers to access models which are retrieved from the API server.

Example MVVM for a user login



Data stored on the view model that is bound to the view (activity) is stored in MutableLiveData objects, an android lifecycle class that notifies when data held changes. Using these objects allowed me to have user inputs be directly stored in the view model as the user is inputting the data.

2.7.4.2 API Calls

To make API calls, Retrofit and OkHttp were used in combination. First, an HTTP client is created which is setup in the **HttpClientConfig** file. The http configuration sets up read/connection timeouts and has potential for additional setup, but that was not used. The **RetrofitConfig** file then builds the API requester using the http client built as well as adding on the base URL for the API calls.

2.7.4.3 User Information Storage

User info is stored in shared preferences. The shared preferences are created as private so that the application is the only one that can access the data.

The user information stored in shared preferences is:

- User token
- User refresh token
- User ID
- RSA public / private keys.

The shared preferences can be accessed globally allowing any activity to access the data.

2.7.4.4 Live Broadcasting

Both announcing and spectating are done on separate threads from the UI thread. Using separate threads allows the UI thread to continue while data is being sent/received from the broadcast server.

2.7.4.4.1 Announcing

To announce a game, the application sends an announce request which includes the game to announce and the token of the user that wants to announce. If the request is valid, the TCP connection made to the broadcast server remains open and listens for game updates from the application. A user can trigger

a game update by saving a set of scores for a hole while scoring a game. Any listening spectators will be sent the game update.

2.7.4.4.2 Spectating

To spectate a game, the application sends a spectate request which includes the game to spectate and the token of the user that wants to spectate. If the request is valid, the TCP connection made to the broadcast server remains open. When a game update is received from an announcer, the broadcaster server will send out the update to the listening spectator. The broadcast server does not handle any messages sent from a spectator after the validation request is handled. Data is only sent from the broadcast server to the spectator.

2.8 Testing

Three types of testing were done for this project. Unit testing, smoke testing, and manual testing. Unit tests were written for the API server, smoke tests for the android application, and manual testing for both the live broadcasting server and android application.

2.8.1 Android App – Client

A Smoke test was written to cover valid use of the android application. It is a single smoke test that navigates through the app, interacting with the interface as a user would and validating that the expected behavior can be seen. It does not cover complex situations like viewing game results of a scored game or validating live scoring results received.

2.8.1.1 Coverage

The smoke test covers the following scenarios:

- Creating an account
- Creating a group
- Creating a course
- Creating a game
- Editing a course
- Editing a group
- Displaying account info
- Displaying group info
- Displaying course info
- Displaying game info
- Inviting to a group
- Accepting a group invite
- Declining a group invite
- Scoring a game

2.8.1.2 Setup

Each part of the smoke test has a comment explaining what the section is doing. The names of each method further explain the process. Sleeps were used for two reason:

1. Allow time for the API calls to finish before continuing the test.
2. Allow time for me to view the test in the case of a failure so that I can better see exactly how the failure occurred.

Example Section

```
// User B logs in, the friend code is copied. Returns to the login screen.
Login(loginNameB, passwordB);
goToAccountInfo();
validateAccountInfo(loginNameB, handleB);
friendCodeB = getFriendCode();
pressBack();
pressBack();
```

safeSleep(4000);
safeSleep(1000);

safeSleep(500);
safeSleep(500);

2.8.2 ASP.NET Web API – API server

Unit tests were written to confirm the API's commands, mappers, conditions, composers and encryption classes work as expected. The combination of these classes makes up the rules of each API call. Currently, there are 184 unit tests, and all are passing.

A combination of the nugets NUnit and Moq were used to write the unit tests.

Unit tests all passing

Application.Command.Tests (150)	729 ms
Application.Command.Tests.AppUsers.Commands (10)	565 ms
Application.Command.Tests.AppUsers.Conditions (8)	5 ms
Application.Command.Tests.Courses.Commands (21)	60 ms
Application.Command.Tests.Courses.Conditions (9)	1 ms
Application.Command.Tests.FrolfGroupInvites.Commands (25)	29 ms
Application.Command.Tests.FrolfGroupInvites.Conditions (6)	< 1 ms
Application.Command.Tests.FrolfGroups.Commands (17)	21 ms
Application.Command.Tests.FrolfGroups.Conditions (15)	< 1 ms
Application.Command.Tests.Games.Commands (10)	25 ms
Application.Command.Tests.Games.Conditions (9)	1 ms
Application.Command.Tests.HoleScores.Commands (6)	18 ms
Application.Command.Tests.HoleScores.Conditions (2)	< 1 ms
Application.Command.Tests.Players (6)	4 ms
Application.Command.Tests.Players.Conditions (6)	< 1 ms
Frolf.Api.Tests (30)	523 ms
Frolf.Api.Tests.Composers.Games (6)	476 ms
Frolf.Api.Tests.Composers.Users (10)	27 ms
Frolf.Api.Tests.Mappers.Courses (2)	5 ms
Frolf.Api.Tests.Mappers.FrolfGroups (2)	4 ms
Frolf.Api.Tests.Mappers.Games (2)	5 ms
Frolf.Api.Tests.Mappers.HoleScoreScores (2)	2 ms
Frolf.Api.Tests.Mappers.Holes (2)	1 ms
Frolf.Api.Tests.Mappers.Players (2)	2 ms
Frolf.Api.Tests.Mappers.Users (2)	1 ms
Security.Tests (4)	670 ms
Security.Tests.Encryption (4)	670 ms
AesEncryptionTests (3)	57 ms
RsaEncryptionTests (1)	613 ms

2.8.2.1 Setup

2.8.2.1.1 Naming format

The format of each unit test method is: <class name>_<method name>_<expectation>_<when>

```
[Test]
public void AddAppUserCommandValidation_PreHandle_ExpectException_WhenFriendCodeInUse()
```

2.8.2.1.2 Setup function

There is a setup method in which the valid case is setup making it easier to make a single change to test failure:

```
[SetUp]
public void Setup()
{
    SetupUsers();
    SetupMockAppUserQuery();
    SetupWorkUnit();

    command = new AddAppUserCommand { NewUser = newAppUser };
    handler = new AddAppUserCommandValidation(mockWorkUnit.Object, mockAppUserQuery.Object);
}
```

2.8.2.1.3 Example Test

In the following test example, the test validates that the command validation fails when adding a new user with a friend code that already exists.

```
[Test]
public void AddAppUserCommandValidation_PreHandle_ExpectException_WhenFriendCodeInUse()
{
    command.NewUser.FriendCode = existingAppUser.FriendCode;

    Assert.Throws<Exception>( () => handler.PreHandle(command) );
}
```

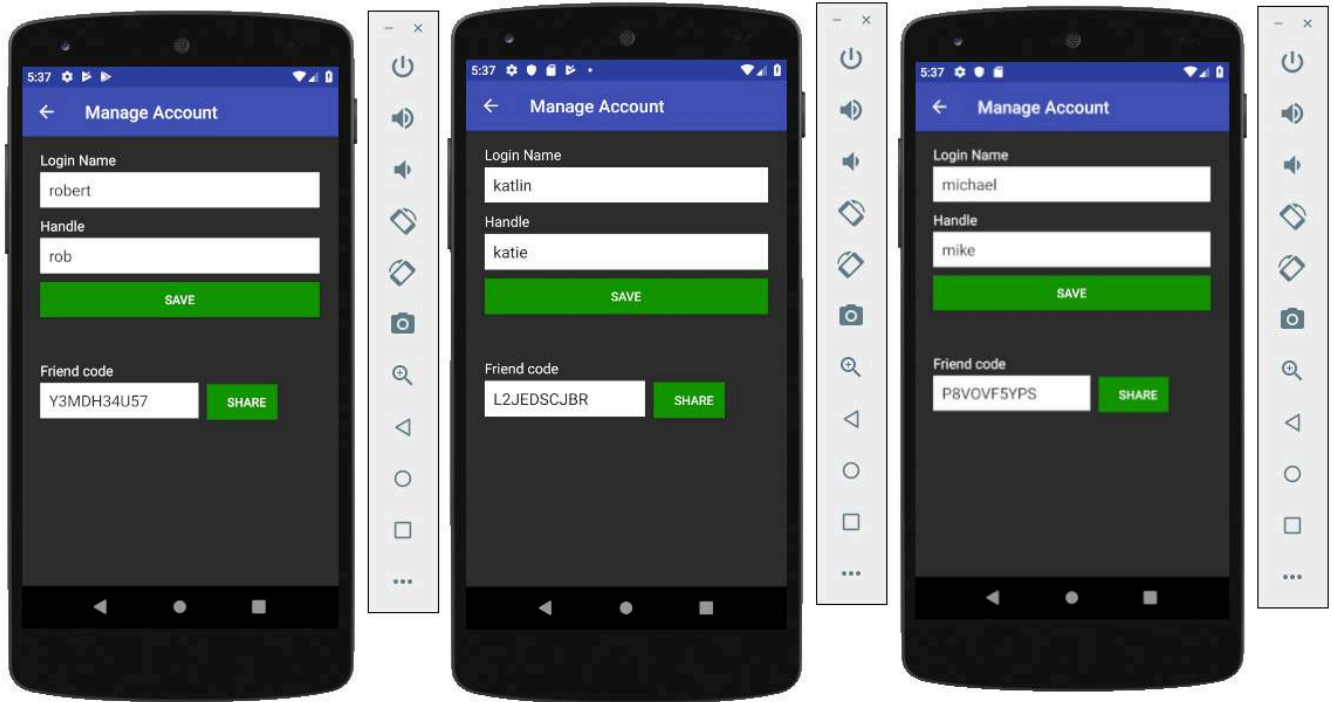
2.8.3 ASP.NET Core App – Score Broadcaster Server

To test the score broadcasting server, manual tests were performed that verify the following:

- A client can connect to the broadcast server as an announcer. An announcer is a client that send data to the server so that it can be broadcasted to clients who are listening. Clients that are listening are known as spectators.
- A client can connect to the broadcast server as a spectator. This allows them to receive game updates from the broadcast server.
- As an announcer, saving the scores of a hole will broadcast the data to each spectator listening.

The testing environment used 3 android emulators running on my PC that were in contact with my API and broadcast server being hosted on AWS.

Clients part of this test – showing their account info for clarity on who is who



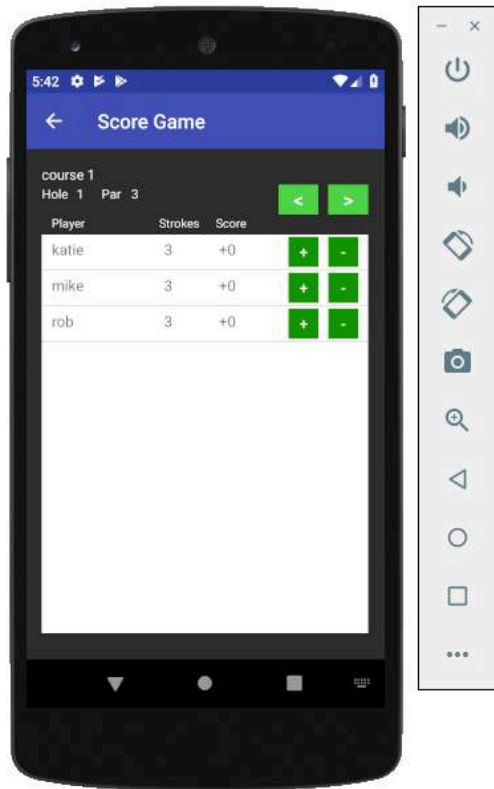
2.8.3.1 A Client can Connect as an Announcer

When a client starts to score a game they are connected to the broadcaster as an AnnouncerClient.

1. Broadcast server is started

```
C:\>cd "frolf app broadcaster 2"
C:\frolf app broadcaster 2>FrolfAppScoreBroadcaster.exe
Broadcaster starting...
Server started. Listening to TCP clients on 172.31.7.8:45000
Waiting for client...
```

2. The game is created on the android client with three players: katie, mike and rob



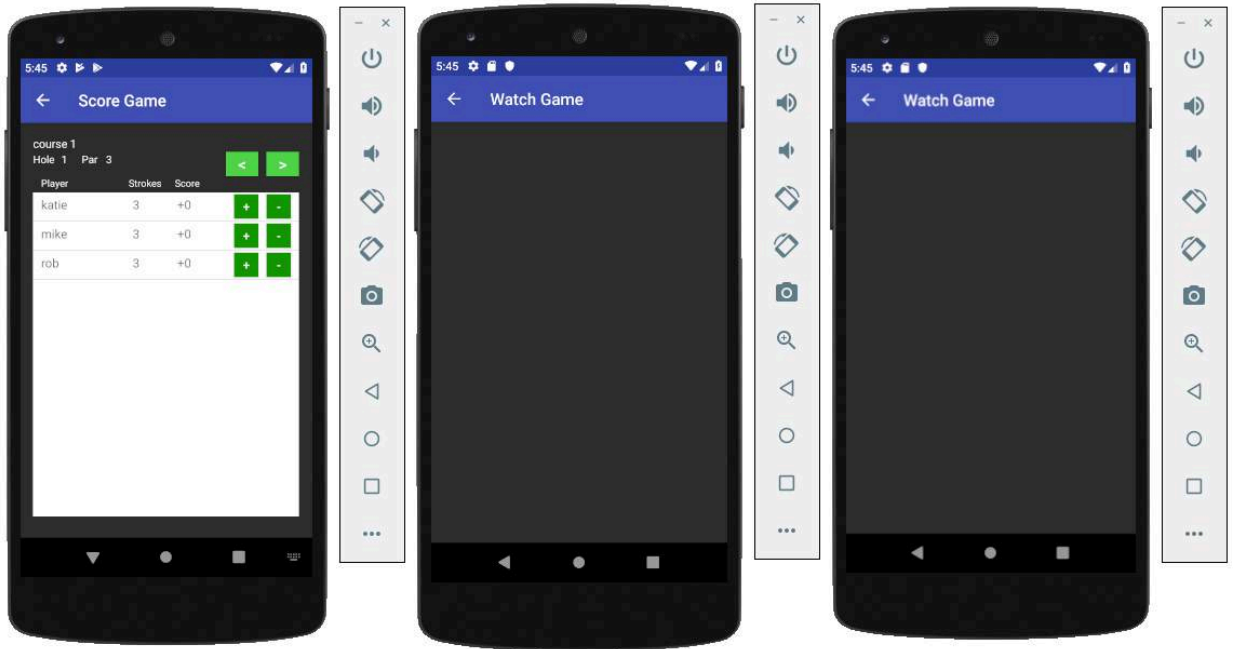
3. The broadcaster receives the client request and accepts the client as the announcer

```
Client attempting to connect...  
Waiting for client...  
Client request received...  
Announcer connected...
```

4. The test is completed.

2.8.3.2 A Client can Connect as a Spectator

1. Two clients connect to a game being broadcasted



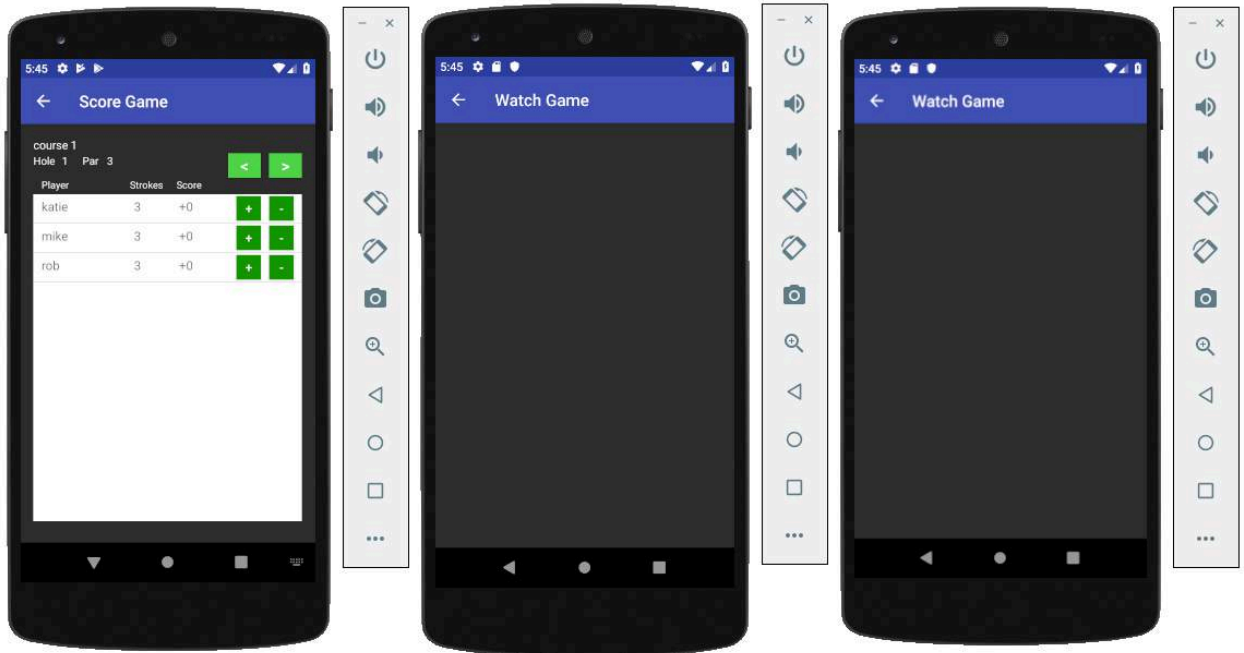
2. The broadcaster accepts the two clients as spectators

```
Client attempting to connect...
Waiting for client...
Client request received...
Spectator connected...
Client attempting to connect...
Waiting for client...
Client request received...
Spectator connected...
```

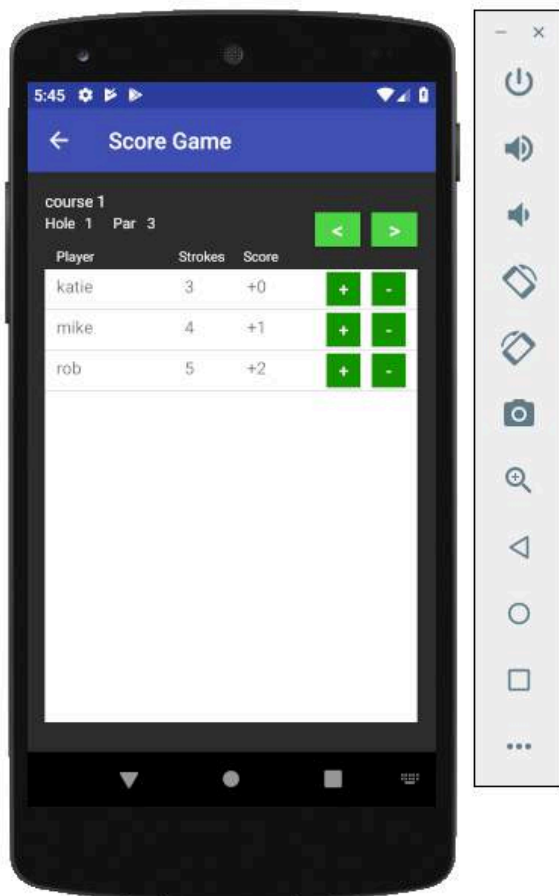
3. The test is completed.

2.8.3.3 *Holes Saved by an Announcer are Broadcasted to all Spectators*

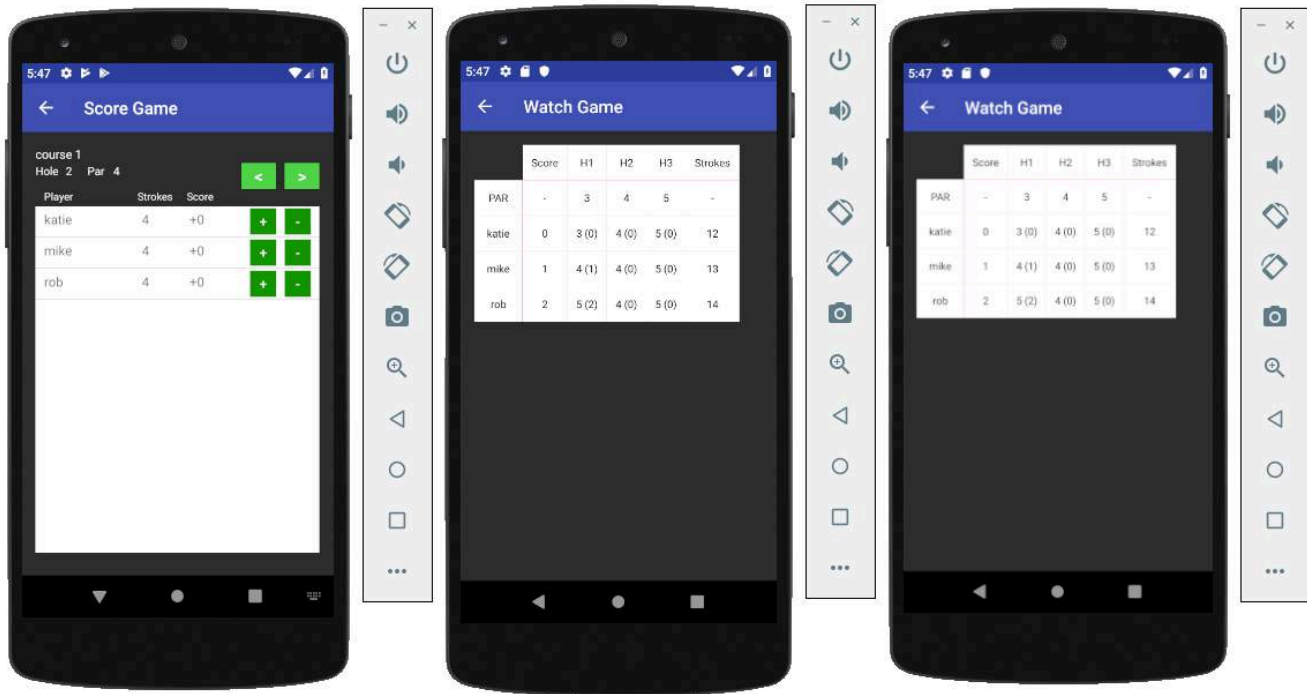
1. Katie and mike watch the game as spectators. They are brought to the watch game screen. The screen is blank until the first broadcast is sent out.



2. The announcer sets the first hole scores to 3, 4, and 5



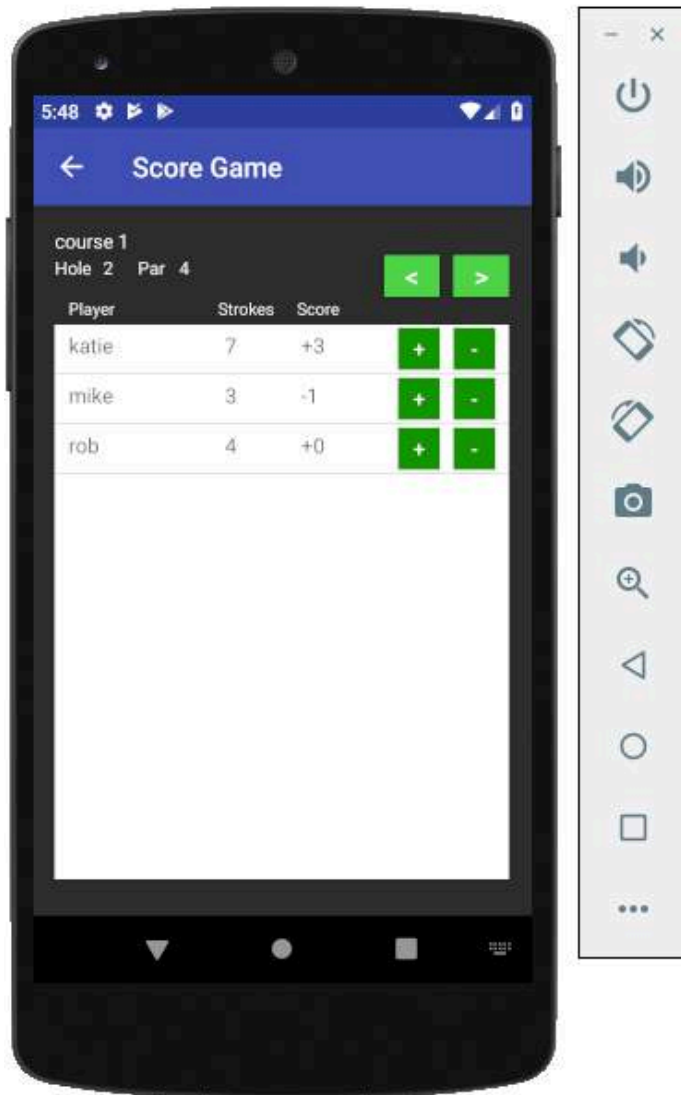
- The announcer saves the first hole which triggers the two spectators to receive the game update and the announcer to be brought to the second hole



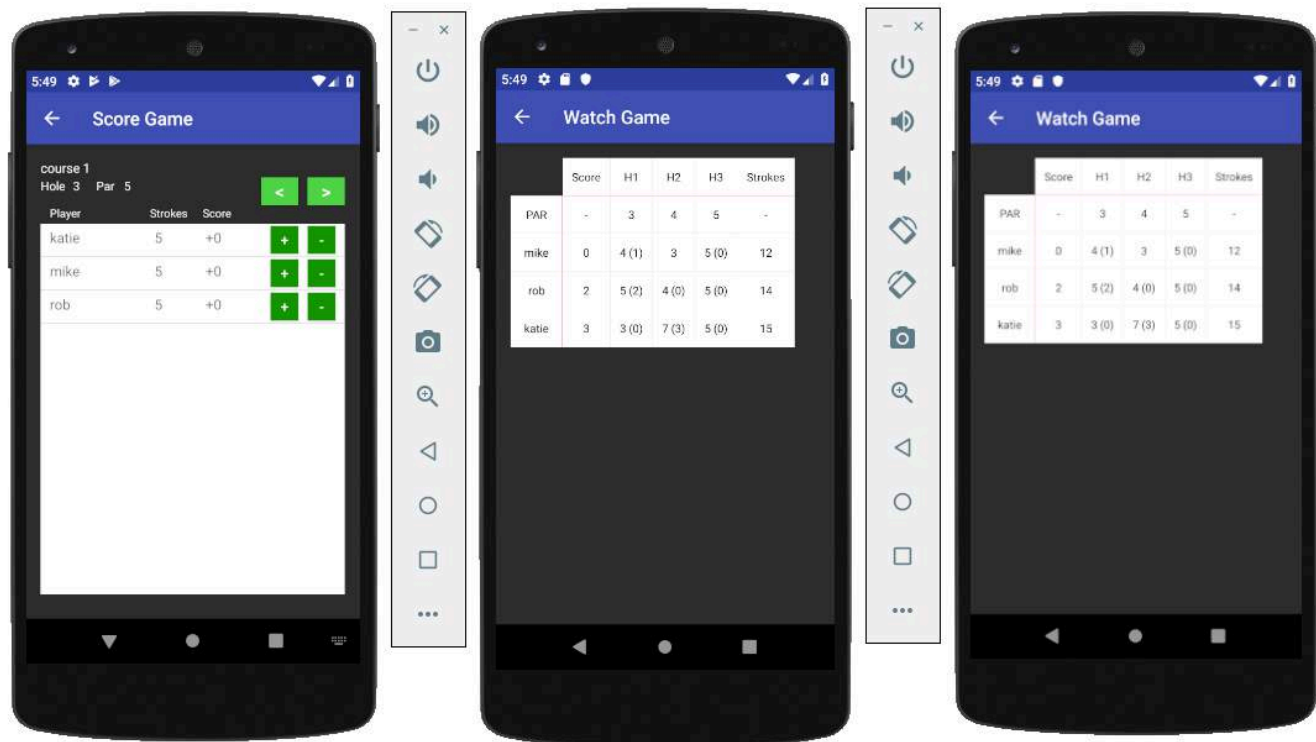
The broadcaster server receiving the game update and sending it out to the two spectators

```
Game Update Size: 10208
Game update sent...
Game update sent...
```

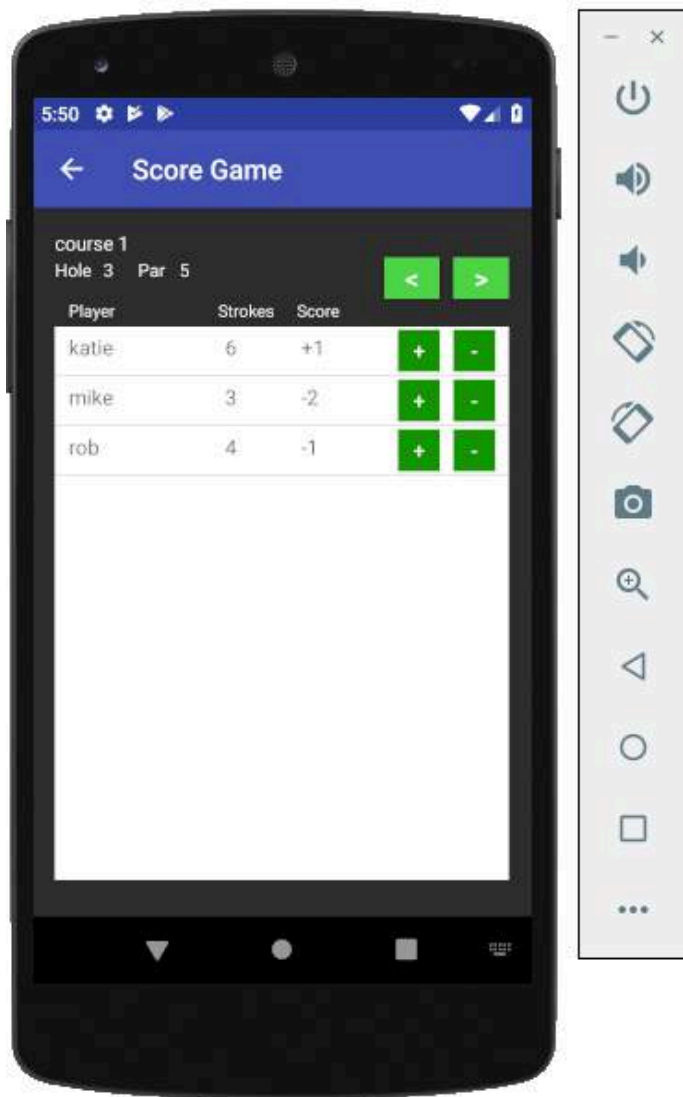
- The second hole is scored 7, 3, and 4 then saved



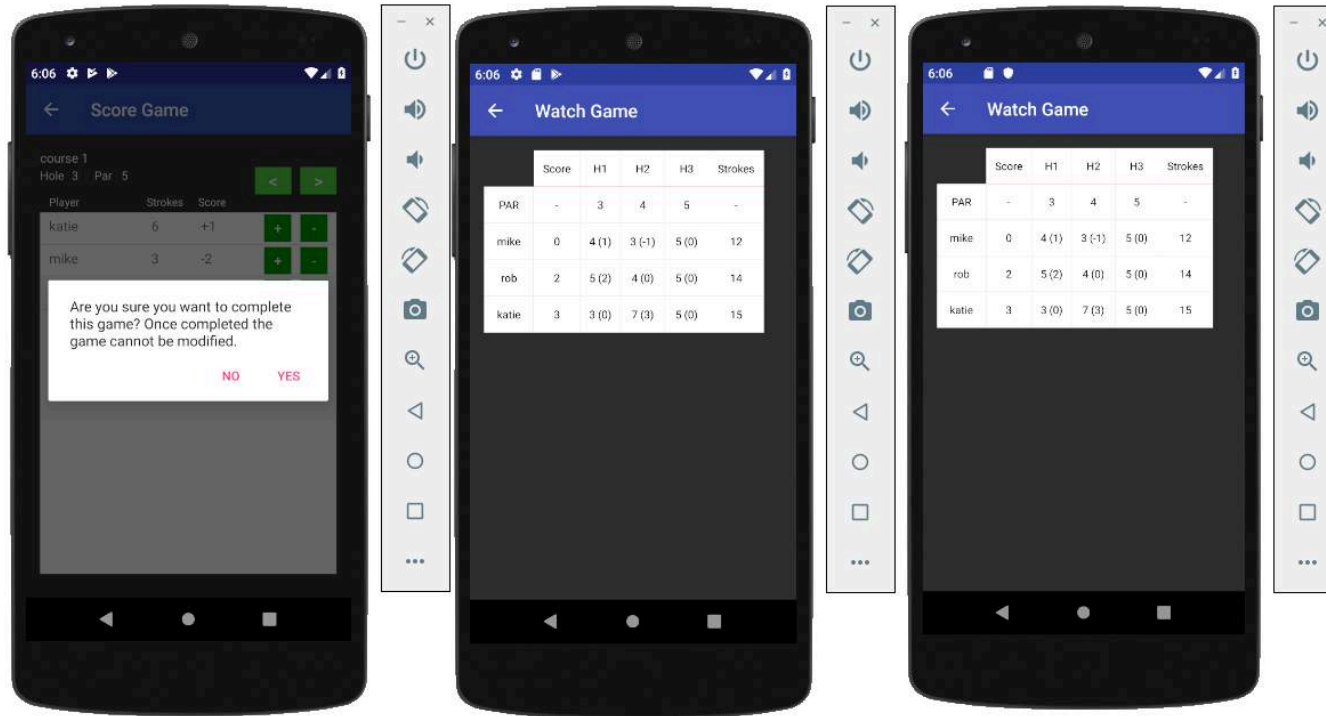
5. The announcer arrives at the third and final hole and the spectators receive the updates. Also, the order of results changes for the spectators. It always shows from 1st to last.



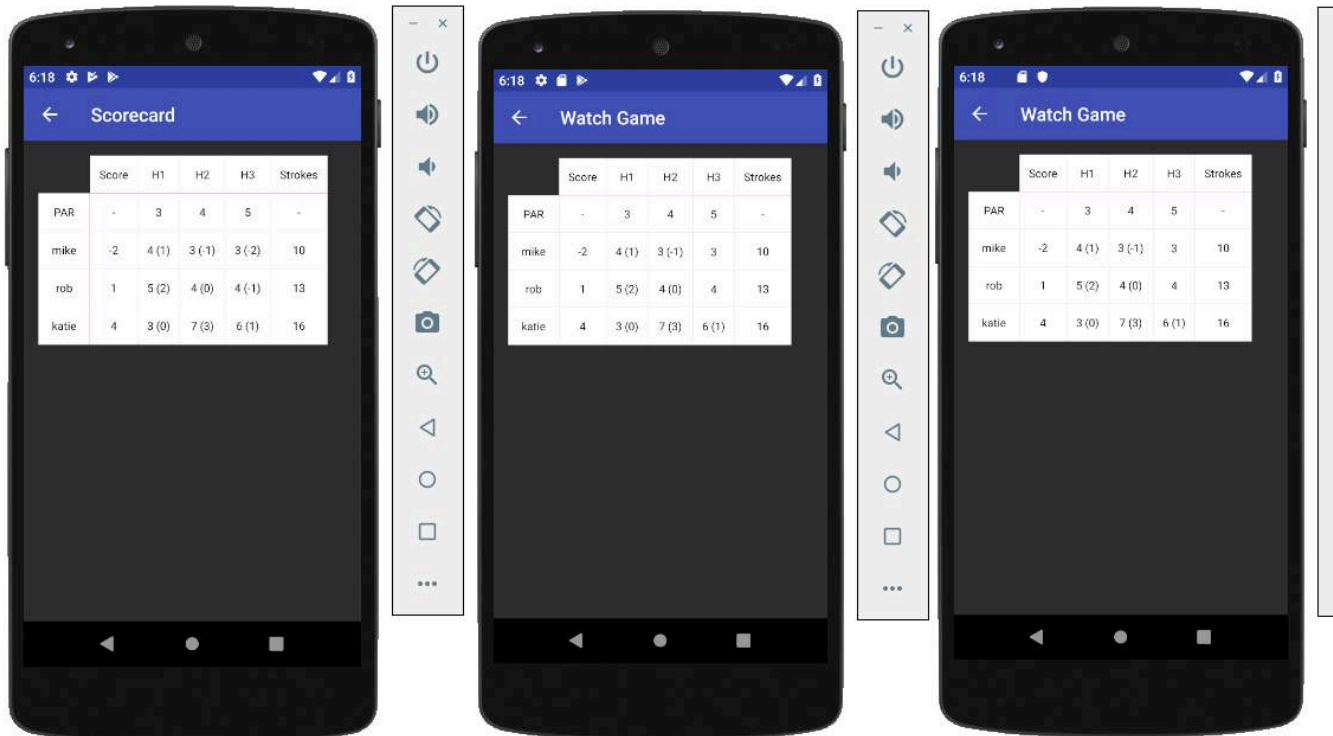
6. The announcer scores the third hole is scored 6, 3, 4



7. The announcer clicks next on the last hole triggering the app to ask if they want to complete the game



8. The announcer clicks “YES” triggering the game to complete, bringing the announcer to the score card for the game and sending the last game update to the spectators



9. The test is completed.

2.9 Implications

2.9.1 Performance

The performance for the applications is quite good. API calls are returning in 0.5-1.5s seconds depending on the size of the data being sent in the API call. In terms of the broadcaster, it can handle 3-4 live games being broadcasted but after that the CPU usage and RAM are maxed, making things slow down. This is less of a code implementation issue and more of a hardware issue.

Currently the server is hosted on a very cheap AWS EC2 instance. The instance only has 1 CPU and 1GB of ram and the live score broadcaster alone can chew threw that quickly. I don't have any load balancing in place, so all broadcasts go through a single server.

To increase performance and ignoring cost, I would set up the broadcast server on a much stronger EC2 instance type meant for constant communication.

2.9.2 Functionality

The application functions well.

2.9.3 Portability

The portability of my application is pretty good. The server contains all business logic meaning creating a new type of client such as an IOS client would only require a new UI to be coded. Also, the API server and the broadcast server do not rely on being on the same server. They could easily be separated into two different servers. Doing something like this may be desirable if I wanted to dedicate a single server to the broadcast server to increase performance.

2.9.4 Maintainability

Overall, I believe the project has achieved a good level of maintainability based on the amount of testing in place and design patterns used. Dependency injection reduces a lot of boilerplate code that needs to be maintained. Also the command pattern used makes it very quick to make new ways to interact with entities and the database.

2.9.4.1 *Frolf API Server*

On the server I coded everything with the SOLID principles in mind. Classes were kept to single responsibilities as can be seen in the class diagrams.

I kept parent classes closed for modification but open for extensions. This can be seen in the command classes I made such as CommandValidationHandler. The child classes inheriting this class cannot change the process of validation but can extend what is validated.

The liskov substitution principle was also followed and can be seen in the command handler pattern used. The CommandValidationHandler can be replaced with subtypes and still won't change how the handler process runs.

I have used small interfaces to stop classes from being forced to over implement.

Finally, I have followed the dependency inversion principle very well. All constructors take interfaces, this means no objects passed into constructors are bound to any one class.

2.9.4.2 *Android Client Application*

The android follows a standard MVVM pattern. Using this pattern keeps the POJO object, UI logic, and backend access logic separate from each other. If I need to add backend access logic, I can put it in the view model, leaving the view and model untouched. When less code needs to be changed, there is a higher chance less things can break unexpectedly.

The folder structure in my project organizes files so it is easy to navigate to the files that need to be changed. Also, my file names are well named making it easy to track them down when they need to be edited.

2.10 New Technologies

2.11 Future Enhancements

I found one of the most difficult parts of working on this project was working on it alone. I always had ideas of new things to add that were not part of my proposal or would take too many hours of additional work. I have an endless amount of enhancements planned for this project, so I will focus on a few features that I believe are the most important.

2.11.1 Automatic Course Creation

Currently, the only way to create courses is manually using the course creation workflow the application provides. This works well when the user is familiar with the course needing to be created or they want to make a custom course. However, this workflow is less favourable when you don't know the details of the course you need to add.

My idea is to add functionality that would allow a user to add a course by lookup. The user would enter the name into the input field and if the course can be found, the user can select it, and hit save. This would trigger the application to build the course for the user without them requiring knowledge of the course details.

2.11.2 Detailed Statistics

My application can score games of frolf, but there is no way to view statistical data in a meaningful way. My idea is to provide users with quick access to such data. I would like to display stats to the user such as game score averages and hole score averages. I would also like to show graphs that plot their scores over a duration of games.

2.11.3 More Communication between the Client and Broadcast Server

The broadcaster works great and communicates live data from the person scoring the game to each person watching the game. I would like to make it more user friendly by sending additional pieces of information to the announcer and spectators of the games regarding their connection to the broadcast server.

For both the announcer and the spectator, I would like to show an icon that shows whether they are connected to the broadcast server or not. Green for connected and red for not connected.

For the spectator I would also like to display when the game has been completed. This will notify them the game will no longer be able to be spectated.

2.11.4 Permissions Set by Users

Right now, a group has two roles a player can be: Member and Administrator. An Admin has full control over the group and a member can only create courses, create games, and score the games they created. I would like to add a third role, Manager. Along with what a member can do, a manager will also be able to invite and remove players to/from groups. On top of adding the third role, I want administrators to be able to choose who can be a manager.

2.11.5 Different Game Types

The only game type right now is a regular game where each player is scored individually. I would like to make it possible to play Match Play and Skins Game.

Match Play is a head-to-head style where each hole is worth a point. The winner of the hole gets the point. Whoever has the most points at the end wins.

Skins is a multiplayer game where each hole is worth a skin. The winner of the hole earns the skin. The player with the most skins at the end of the game wins. This game type is like Match Play.

2.12 Timeline/Milestones

My initial timeline was not very close to the actual date. I believe I estimated dates in a picture perfect scenario where life wasn't going to get in the way. The time estimates were closer except for the implementation/testing portion of my server. The underestimation was caused because of an oversight on how I could implement TCP communication for the broadcast server. Originally, I thought this would be able to be done using my API server, but this was not the case. My miscalculation required me to make an additional server to handle the live broadcasting portion of the application.

Core functionality development – waterfall approach				
Timeline	Tasks	Breakdown	Estimated Hours	Actual Hours
Requirement Analysis June 1 st – June 8 th Actual: Aug 1 st – Aug 10 th	Gather requirements for: • Hosting the MySQL database • C# API call development • C# / MySQL integration • Parsing JSON in C#/Java	Requirement Analysis: 20h	20h	27h
System Design June 8 th – June 22 nd Actual: Aug 11 – Aug 25	Design the following core system implementations: • Database design (class diagrams, database access) • Client/Server API communication (sequence diagrams) • JSON parsing for client/server • Encryption/decryption processes for data transmission	System Design: 40h	40h	32h

Implementation June 22 nd – July 20 th Actual: Aug 26 – Jan 11	Implement the core functionality based on system designs: • CRUD implementation in C# to access data • API calls are in place to send/receive data to/from the server • Serializing/deserializing JSON data • End-to-end encryption in place	Implementation: 100h	100h	212h
Testing July 20 th – Aug 3 rd Actual: Jan 11 – March 20	Core functionality implementation testing: • CRUD operations work for all class entities • API calls can be made from the client to the server • Data sent to/from client and server is encrypted upon sending and decrypted upon receiving	Testing: 30h Fixing Bugs: 30h	60h	104h
Milestone: Core functionality has been completed			220h	359h
User Functionality development – agile approach				
Timeline		Breakdown	Total Hours	
Sprint #1 Aug 3 rd – Aug 17 th Actual: Apr 1 – Apr 29	Users can: • Create an account • Login to the created account • Create friend groups • Add/remove other users to/from their created friend groups	Planning: 8h Design: 10h Implementation: 18h Testing: 10h Build: 4h	50h	54h
Sprint #2 Aug 17 th – Aug 31 st Actual: Apr 30 – May 23	Users can: • Create courses • Create games • Score games	Planning: 8h Design: 10h Implementation: 18h Testing: 10h Build: 4h	50h	44h
Sprint #3 Aug 31 st – Sep 14 th Actual: May 23 – June 18	Users can: • View scorecard for a game that has been completed • Upload completed games, so they can be downloaded by anyone in the friend group • Download completed games, so they can be viewed	Planning: 8h Design: 10h Implementation: 18h Testing: 10h Build: 4h	50h	40h
Milestone: User functionality has been completed			150h	138h
Final Project Completion Requirements				

Final Requirements Sep 14th – Oct 5th Actual: June 19 – Aug 26	Prepare Demo: • Show the ability to create/login to an account • Show the ability to add other people to a created friend group • Show the ability to score a game and upload it • Show the ability to download the game (by an account other than the one that scored the game) Documentation: • Write final report • Write client application user guide • Write server setup guide	Prepare Demo: 15h Client user guide: 10h Server setup guide: 10h Final report: 40h	75h	58h
Milestone: Demo / Final Report completed			75h	58h
Milestone: Project completed			445h	571h

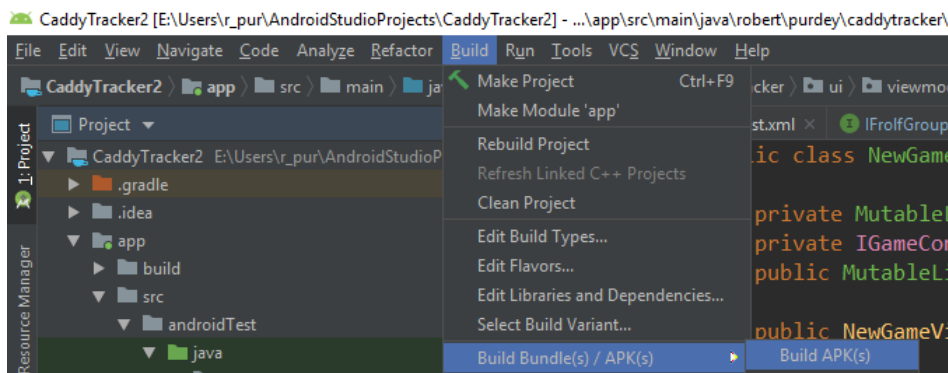
2.13 Installation Manual

2.13.1 Android Application

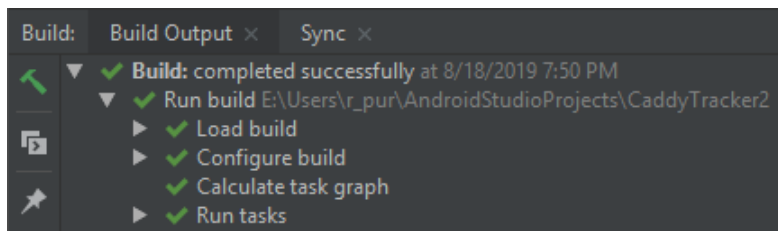
The application requires a minimum of SDK 26.

Covers generating the APK file for the application and loading it onto an android phone.

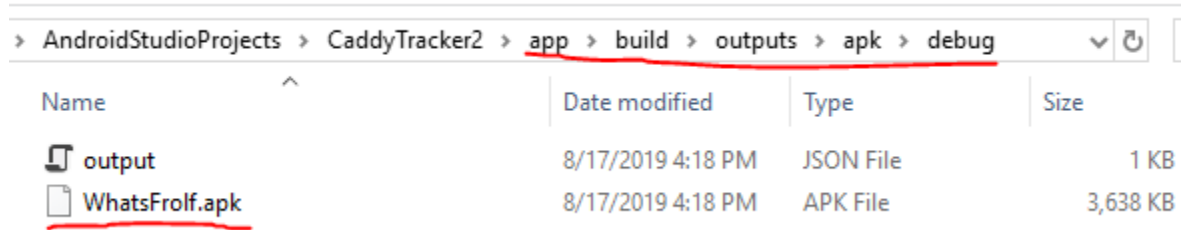
1. Build the APK file using Android Studio by clicking: **Build > Build Bundle(s) / APK(s) > Build APK(s)**



Note: Ensure the build succeeds



2. Navigate to <your android project folder>\app\build\outputs\apk\debug to find the APK file.



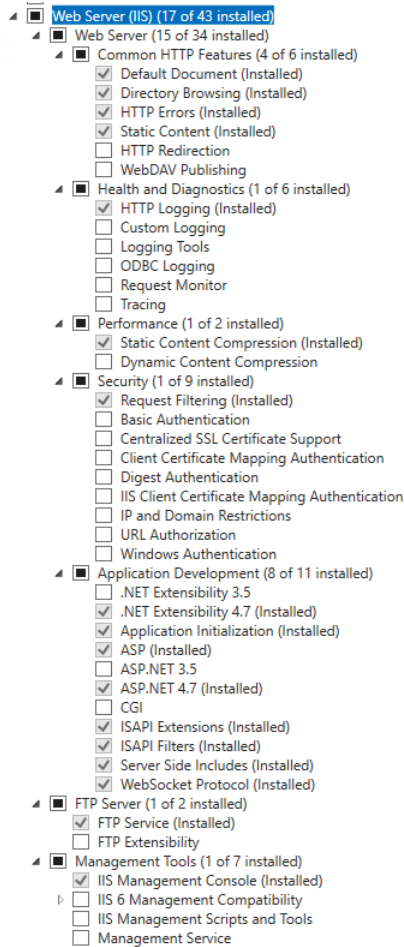
3. Connect the phone to the PC and transfer the file from the PC to the Android phone.
4. Locate the APK file on the phone using the file manager.
5. Click the found APK file and install it.

The application should now be installed on the Android device.

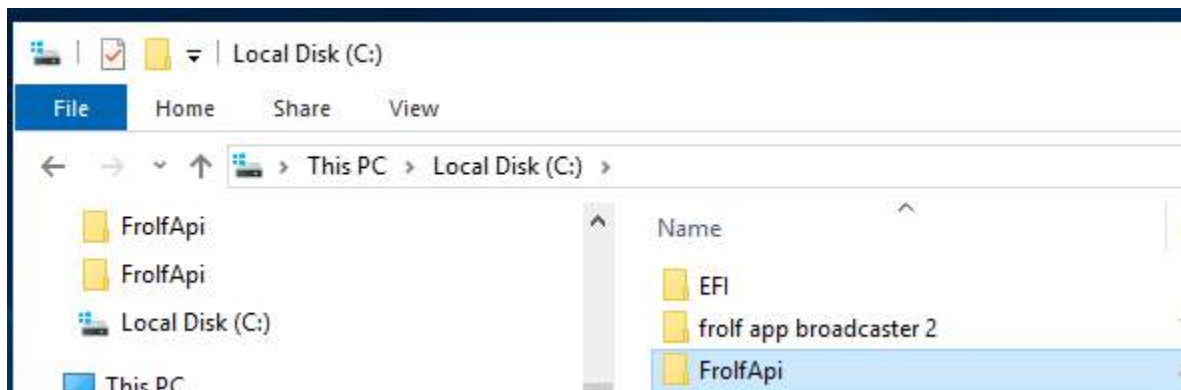
2.13.2 API Server

This installation covers hosting the API server using IIS. It does not cover setting up the AWS environment which is talked about in the deployment section.

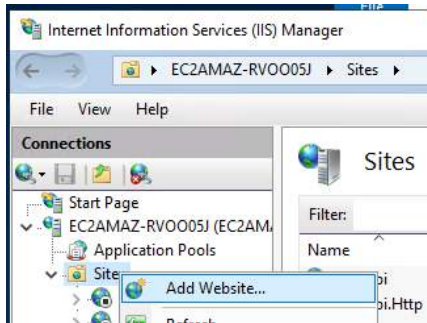
1. Enable IIS with the following settings on the EC2 server:



2. Copy the API project onto the EC2 server (Avoid user privilege issues by placing the project outside of the user folder). In my case, I used the C: drive:

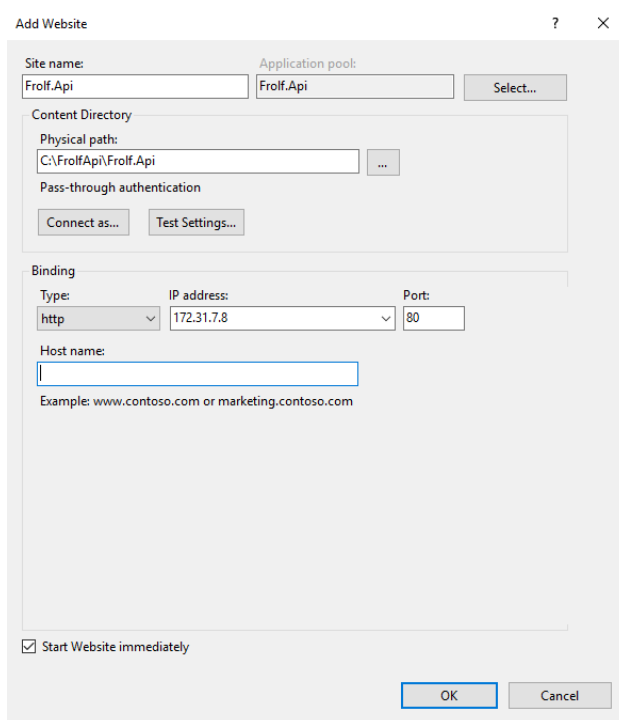


3. Open IIS and add a new website:



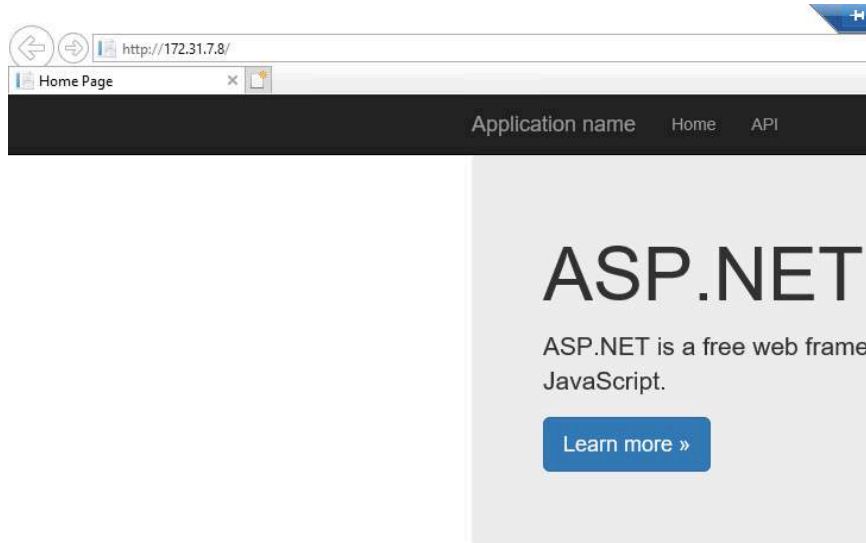
4. Give the website a name
5. Point the website to the Frof.Api folder inside the API project folder
6. Bind the site to HTTP and port 80 (This site is only accessed internally by the AWS load balancer so it is safe to use HTTP).

By now, the website setup should look like this:



7. Click "OK"

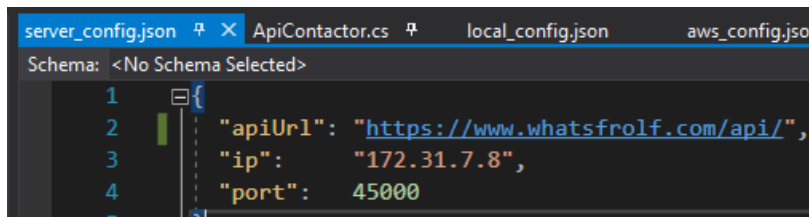
The website should now be hosted by IIS on the EC2 server. The site should display the API site if successful:



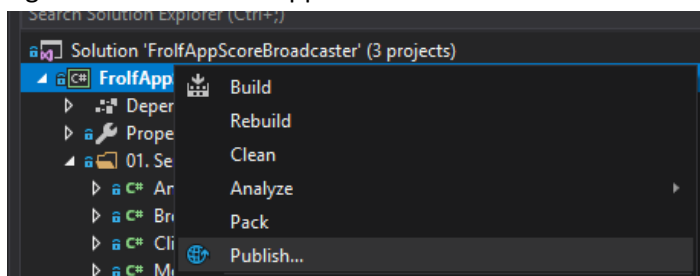
2.13.3 Broadcast Server

This installation covers hosting the broadcast server on the AWS EC2 server. It does not cover setting up the AWS environment I used.

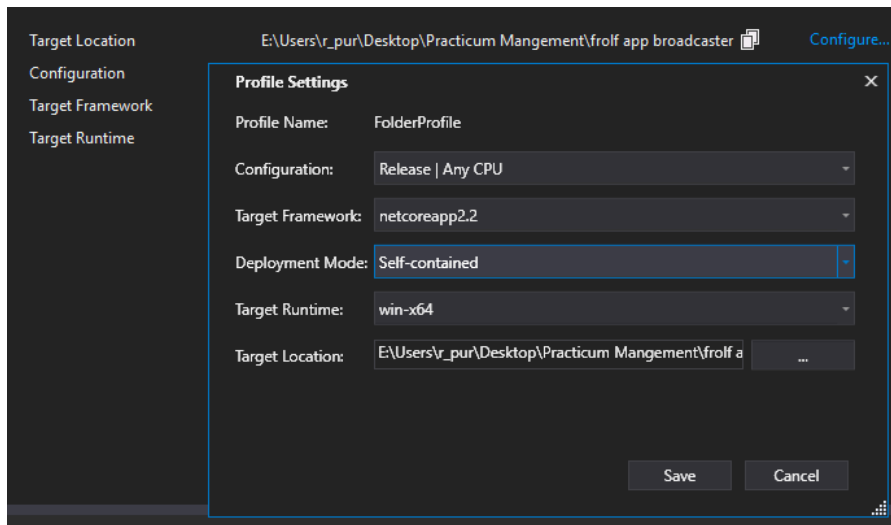
1. Setup the config file so (this can be done at any time):
 - a. apiUrl – set to the API project API URL
 - b. ip – set to the IP the server is listening on
 - c. port – set to the port the server is listening on



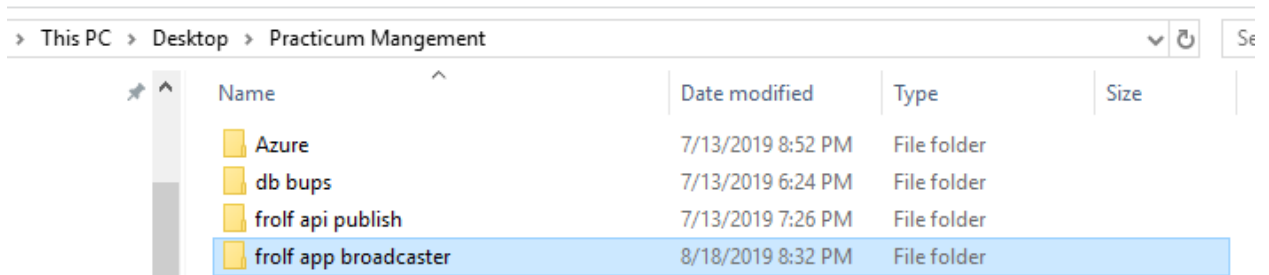
2. Right-click the console application and click Publish:



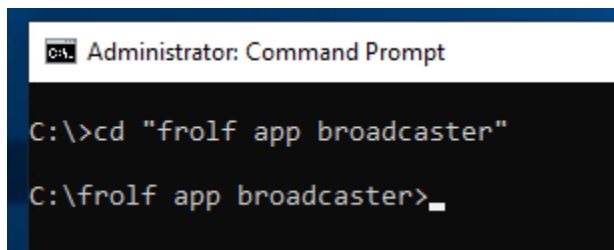
- Click 'Configure' and fill out the Profile Settings as follows and click 'Save':



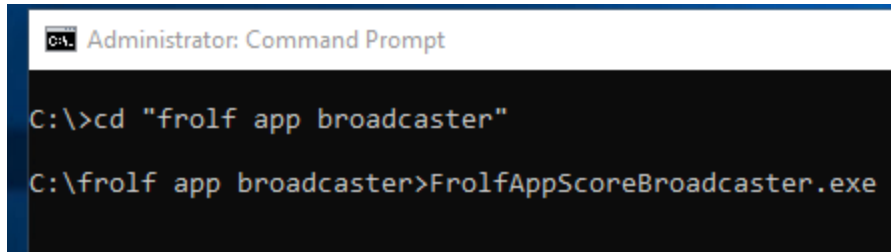
- Click 'Publish' and wait for it to complete.
- Navigate to the folder where the published files exist and copy it over to the EC2 server:



- Open the command line and navigate to the location where the broadcaster was copied to on the EC2 server:



- Run the broadcaster by entering: **FrolfAppScoreBroadcaster.exe**

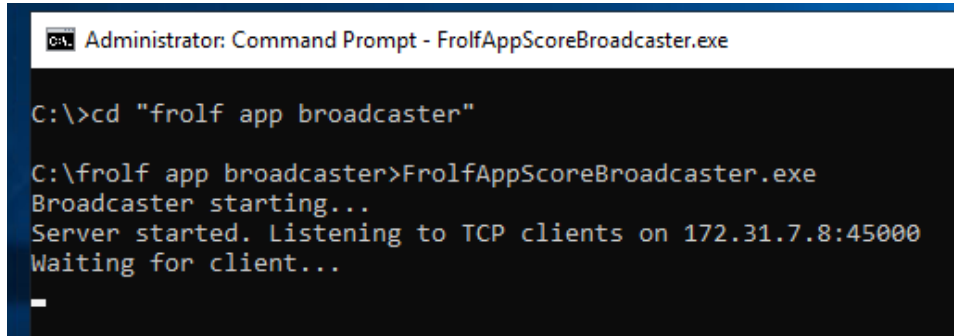


```
Administrator: Command Prompt

C:\>cd "frolf app broadcaster"

C:\frolf app broadcaster>FrolfAppScoreBroadcaster.exe
```

The broadcaster should now be listening. A successful starting of the server looks like this:



```
Administrator: Command Prompt - FrolfAppScoreBroadcaster.exe

C:\>cd "frolf app broadcaster"

C:\frolf app broadcaster>FrolfAppScoreBroadcaster.exe
Broadcaster starting...
Server started. Listening to TCP clients on 172.31.7.8:45000
Waiting for client...
_
```

2.14 User Manual

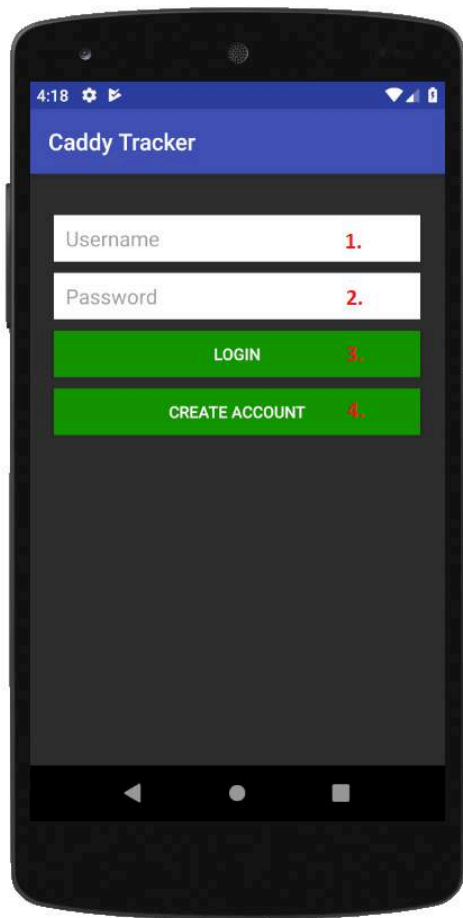
2.14.1 Android Application

2.14.1.1 *Creating an Account*

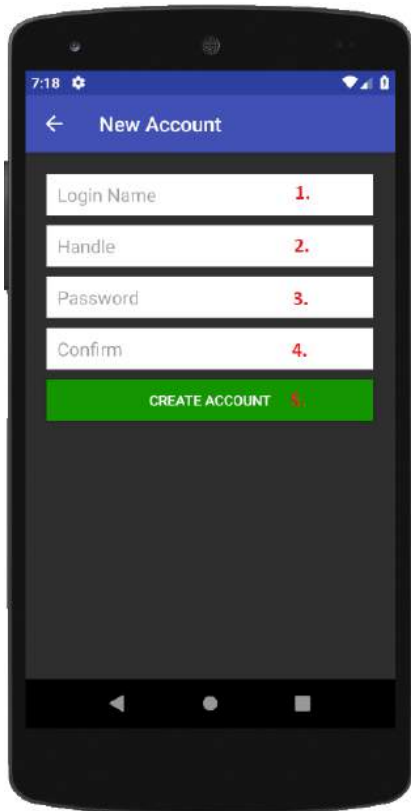
Creating an account will allow you to log into the application.

1. Open the application

2. Click 'Create Account'



3. Fill in each textbox on the screen



4. Click 'Create Account'

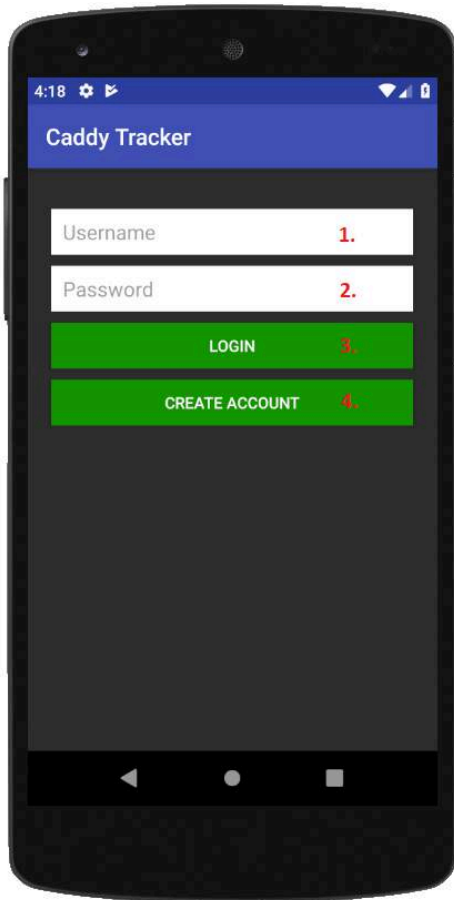
If successful, the application will bring you back to the login screen.

2.14.1.2 Logging In

Logging into the application will grant you access to the app's functionality.

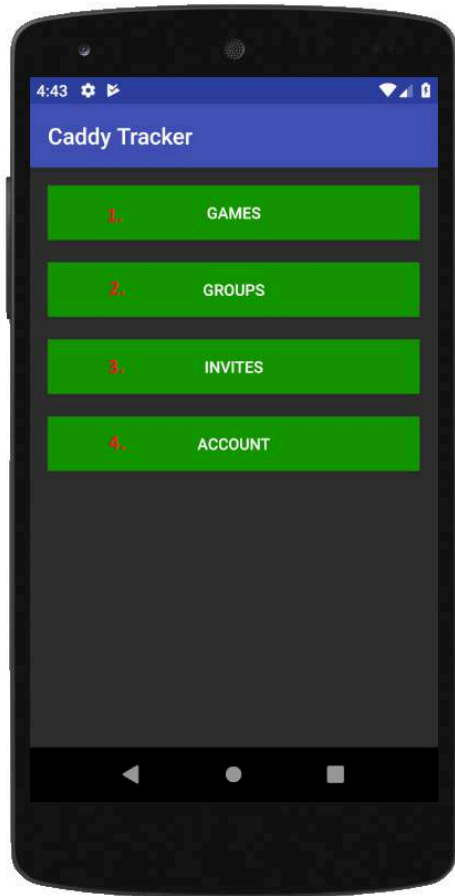
1. Open the application

2. Enter your username and password into their respective fields



3. Click 'Login'

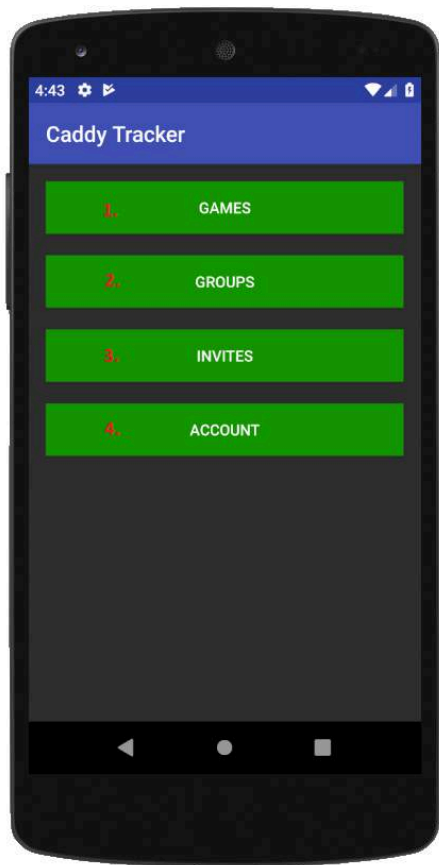
A successful login will bring you to the main menu of the application:



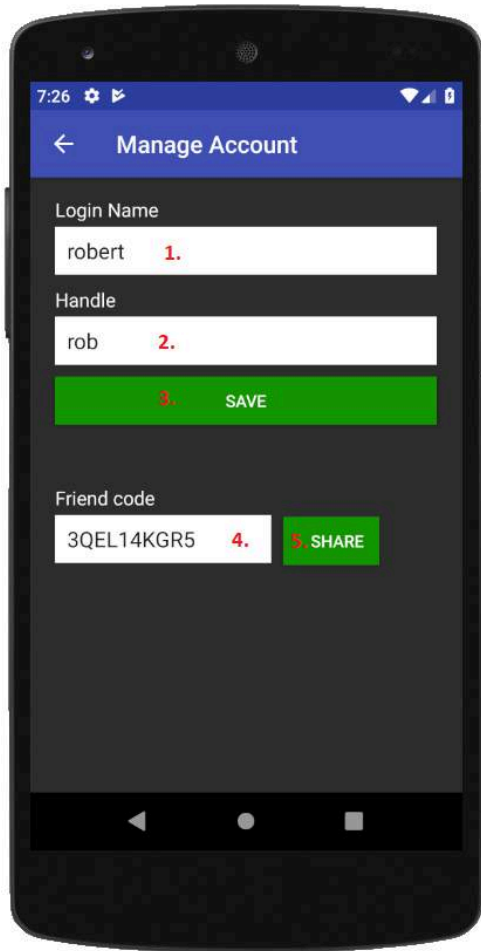
2.14.1.3 Sharing your Friend Code

1. Log into the app – See Logging In section.

2. Click 'Account'



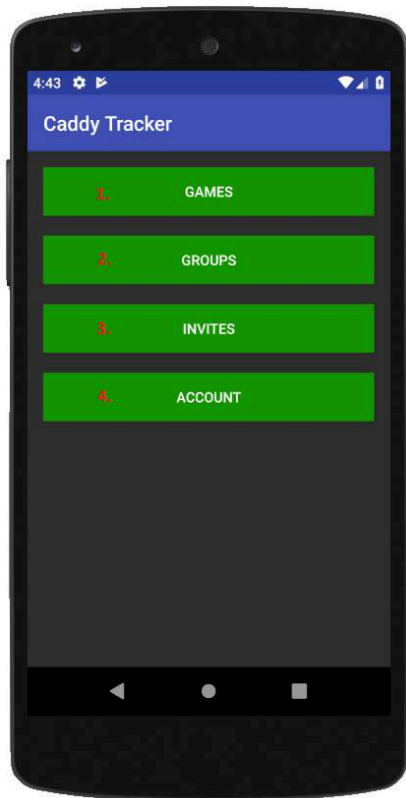
3. Click 'Share'. Clicking share allows you to send your friend code using the application of your choice.



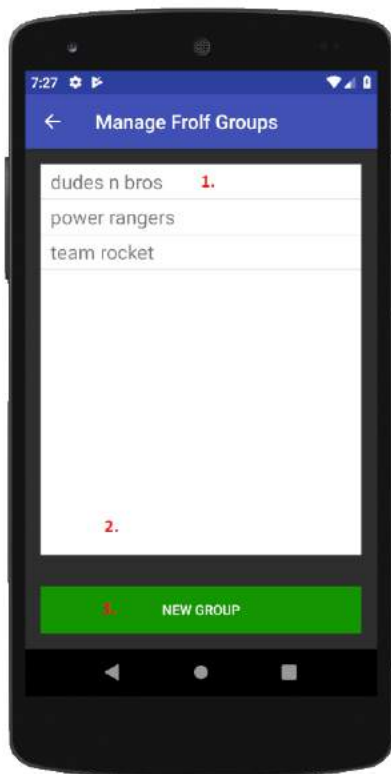
2.14.1.4 *Creating a Group*

Creating a group allows you to invite your friends to the group and to create courses the group can use when making a game.

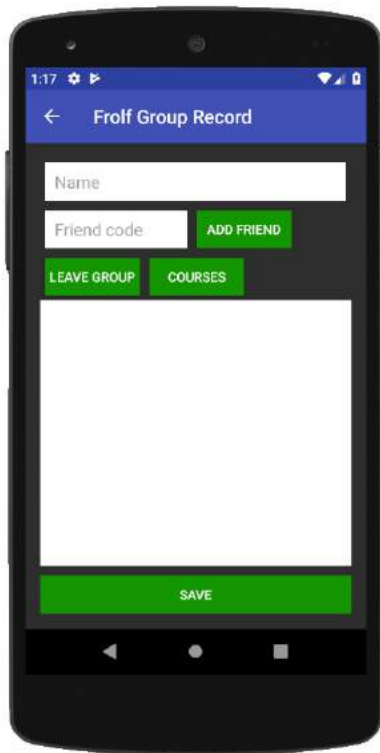
1. Log into the app – See Logging In section.
2. Click 'Groups'



3. Click 'New Group'

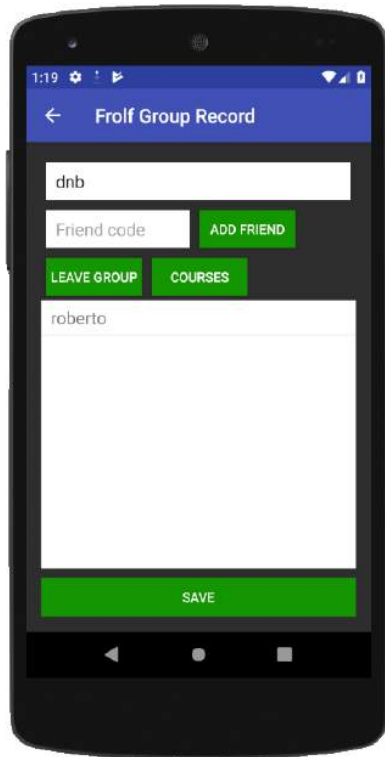


4. Enter a name for the group



5. Click 'Save'

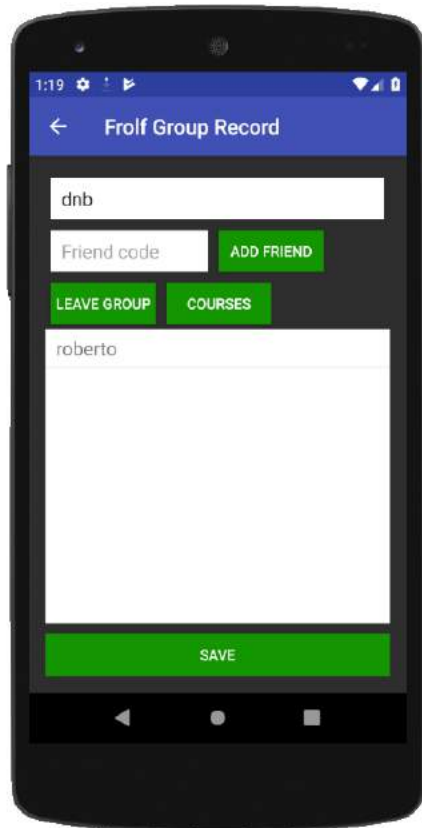
If successful, the group will display you as the only user in the group:



2.14.1.5 Adding Friends to a Group

Adding friends to the group will allow you to select them as players when creating a game.

1. Go to the group you want to invite to – See Creating a Group section to learn how to make a group.
2. Ask your friend for their friend code – See Sharing your Friend Code.
3. Enter your friend's code into the Friend code box



4. Click 'Add Friend'

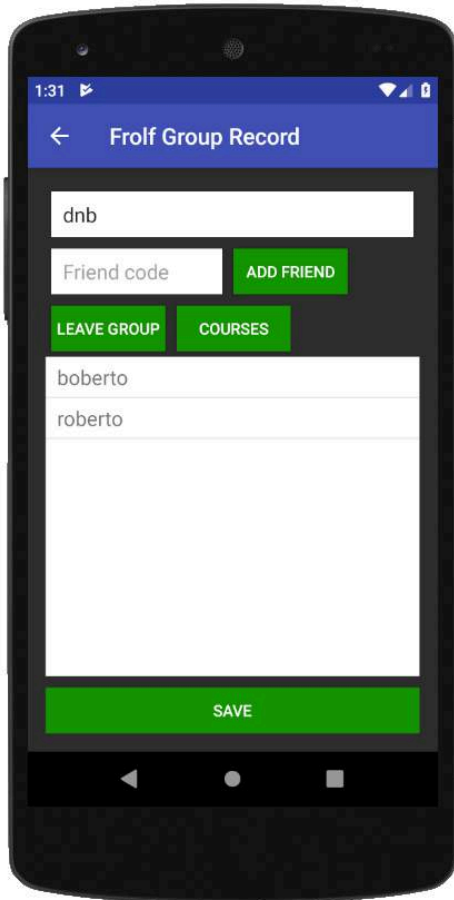
If successful, an invite will be sent to your friend. Your friend will be able to search their invites and accept the invite.

2.14.1.6 Removing Friends from a Group

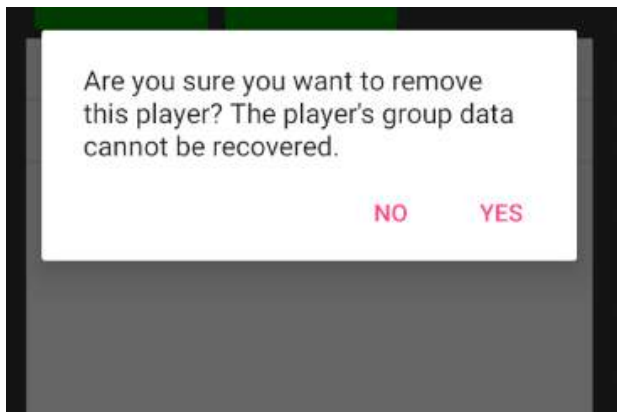
Removing a friend from the group will stop the friend from being able to participate in any group activities such as taking part in a game or viewing live games.

1. Go to the group you want to remove the friend from – See Creating a Group section to learn how to make a group.

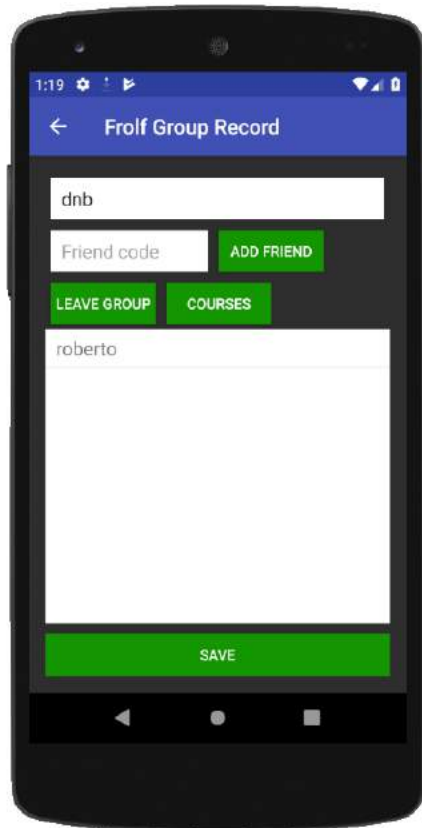
2. Click on the row containing the user you wish to remove. In this example, boberto will be removed.



3. Click 'Yes' to remove the user. Clicking 'No' will cancel the request.



If successful, the user will be removed from the group members list:

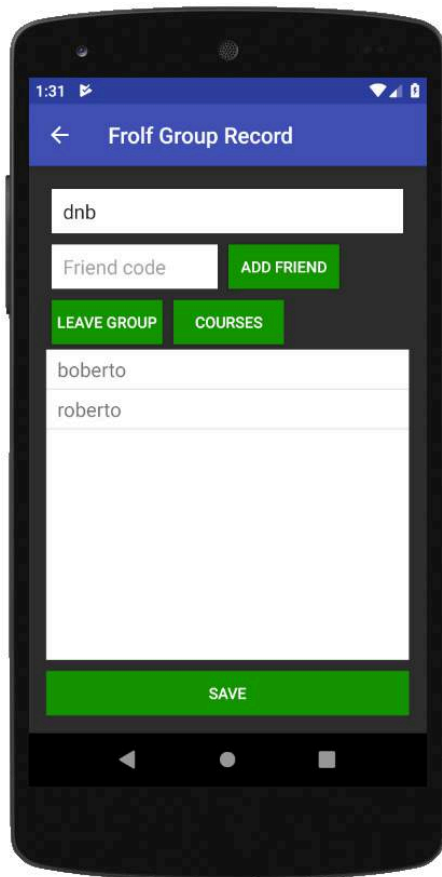


2.14.1.7 Leaving a Group

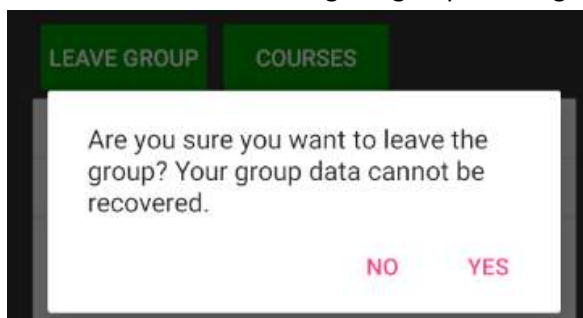
Leaving a group allows players to remove themselves from a group they don't want to be a part of.

1. Navigate to the group you don't want to be a part of anymore.

2. Click 'Leave Group'



3. Click 'Yes' to confirm leaving the group. Clicking 'No' will cancel the leave group request



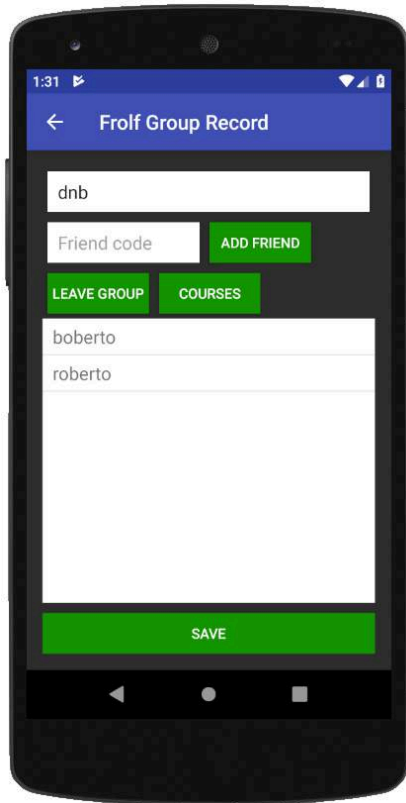
If successful, the application will navigate to the Manage Groups screen

2.14.1.8 Creating a Course

Creating a course will allow the course to be chosen during game creation given the group selected is the one the course was created for.

1. Navigate to the group you want to create the course for.

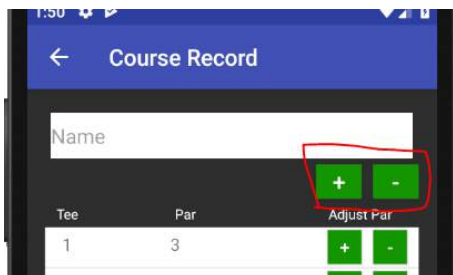
2. Click 'Courses'



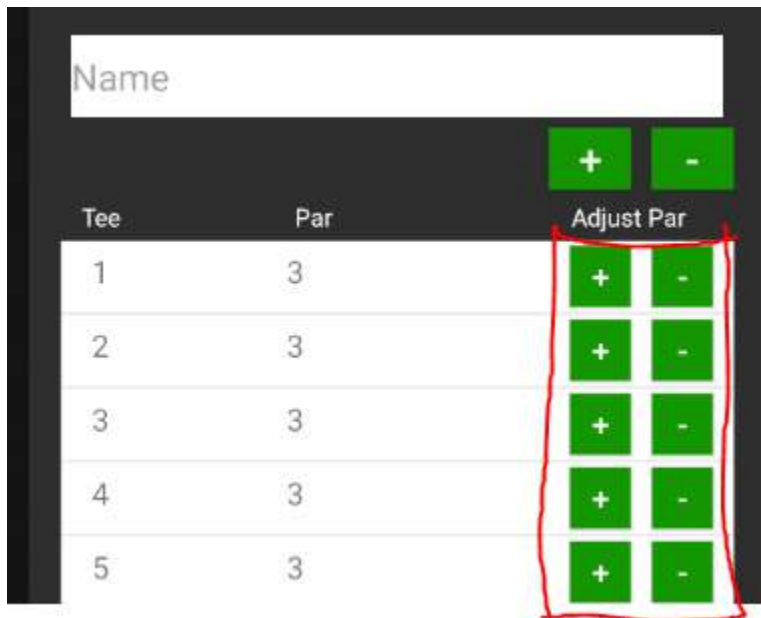
3. Click 'New Course'



4. Give the course a name
5. Use the +/- buttons outside of the hole list to add/remove holes

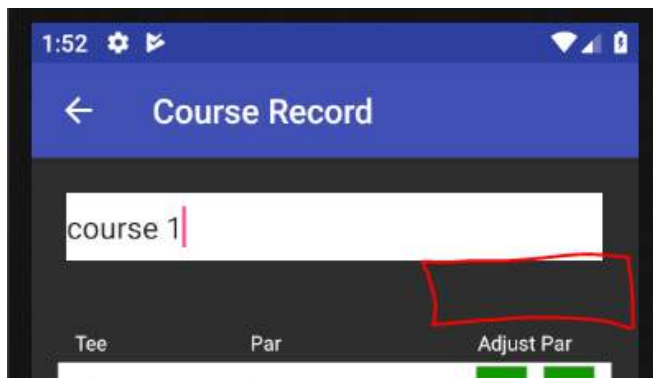


6. Use the +/- buttons inside the hole list to increase/decrease the par for the hole



7. Click 'Save' once the course has been setup

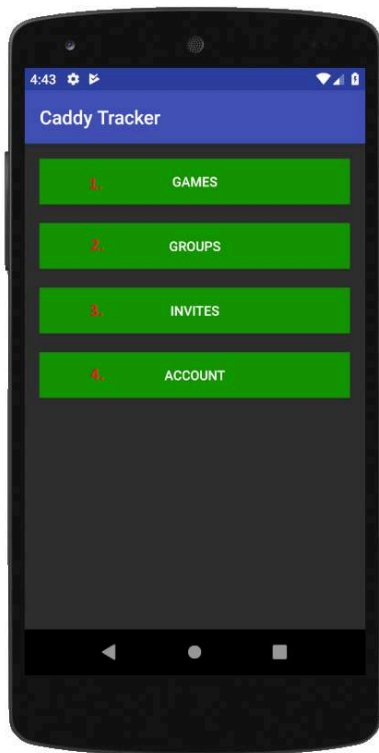
If successful, the hole count adjustment will be hidden. Adjusting the holes after the course has been created is not possible:



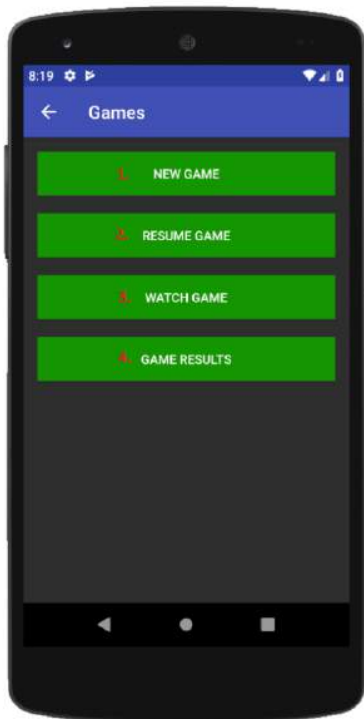
2.14.1.9 Creating a Game

Creating a game will allow you to score it and for others to watch it be scored. Before creating a game, you must be part of at least one group that has at least one course in it.

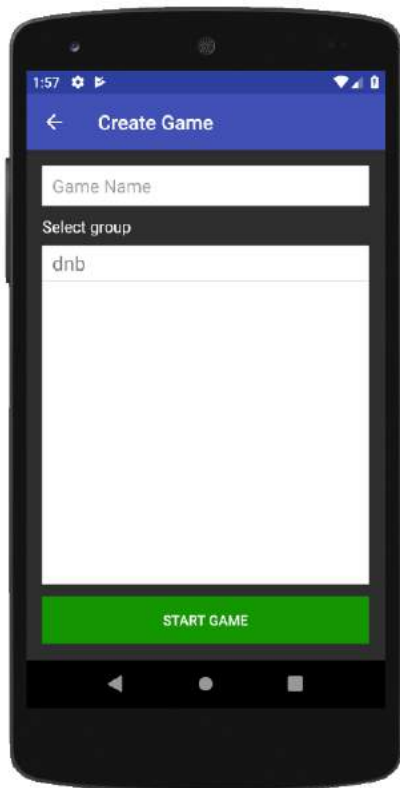
1. Go to the main menu and Click 'Games'



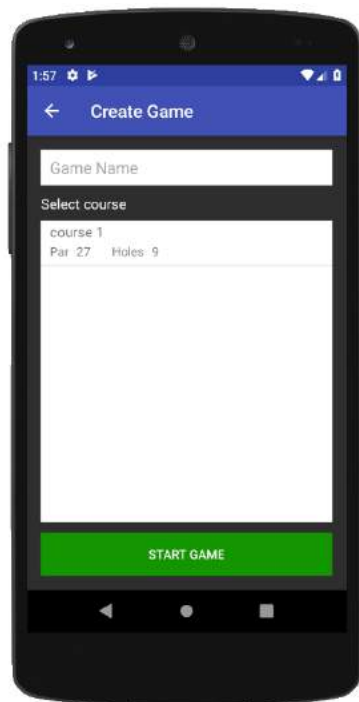
2. Click 'New Game'



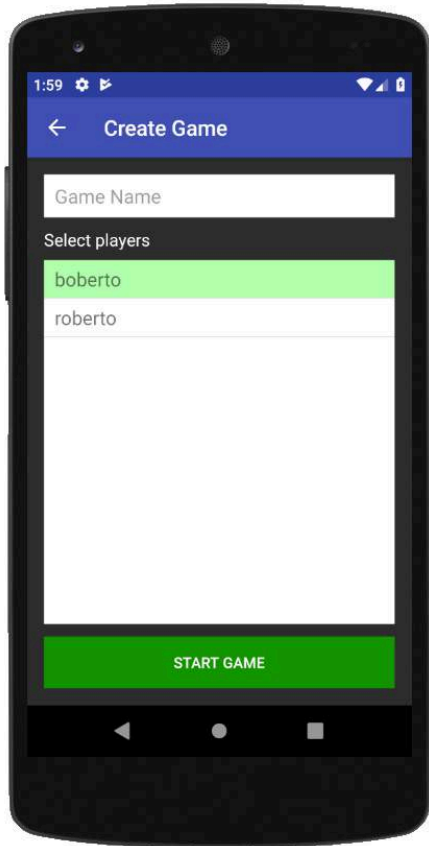
3. Select a group the game will be for by clicking on the group's row.



4. Select a course the game will be played on by clicking on the course's row

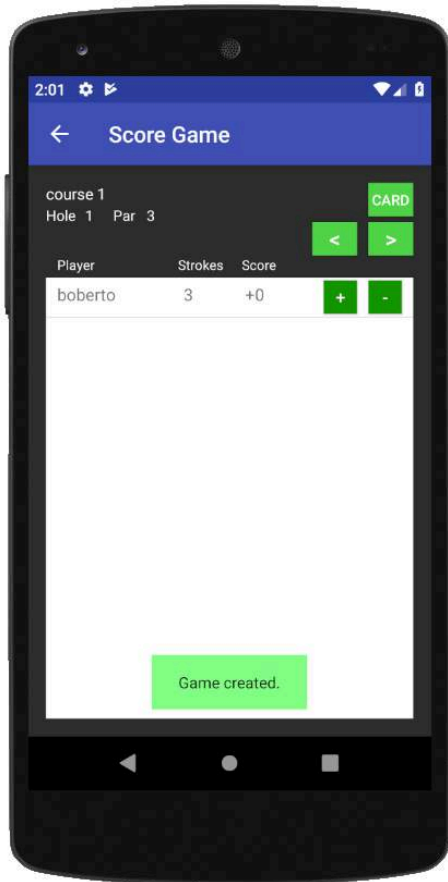


5. Select the players that will be playing the game. Players can be deselected. Selected players are highlighted in green.



6. Give the game a name in the 'Game Name' box.
7. Click 'Start Game'

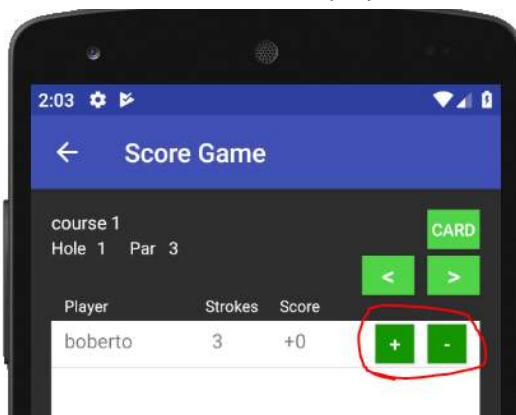
If successful, the app will navigate to the first hole of the created game:



2.14.1.10 Scoring a Game

Scoring a game is what the app is all about. After creating a game or resuming a game, the game can be scored.

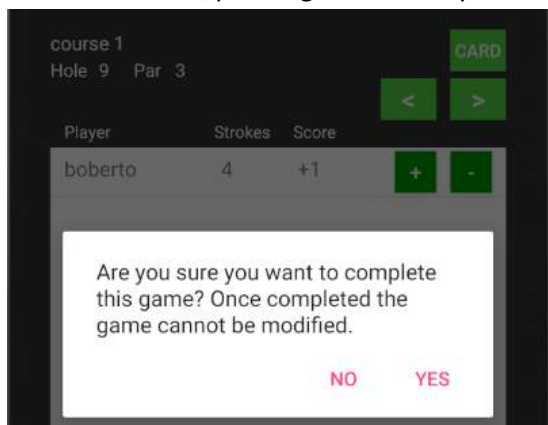
1. Use the +/- buttons on each players row to increase/decrease their stroke count



- Use the left/right arrows to switch the hole being scored. < goes backwards and > goes forwards one hole

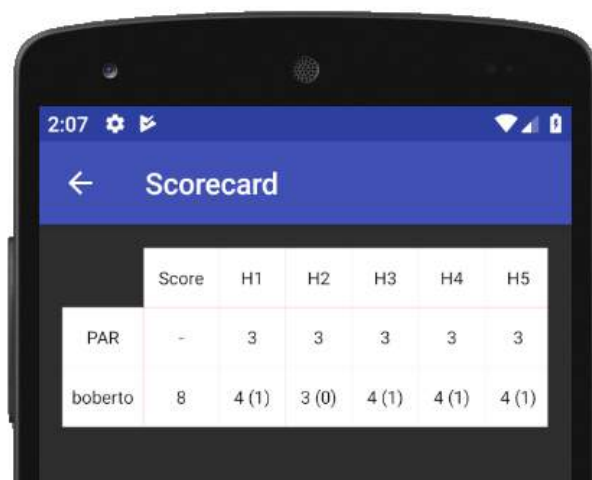


- On the final hole, pressing > will ask if you want to complete the game



- Click 'Yes' to complete the game or 'No' to continue scoring.

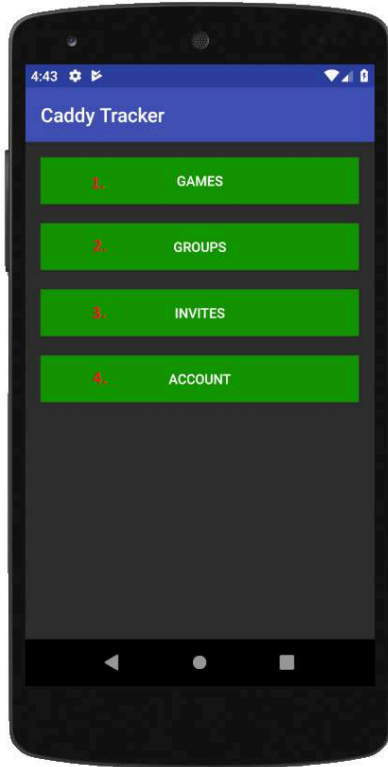
If 'Yes' is clicked, the app will navigate to the scorecard:



2.14.1.11 *Watching a Game*

Watching a game is how players can view the game as it is being scored live. Games that show up in the Watch Game section are games that are in progress, meaning they have not been completed yet.

1. Go to the main menu and Click 'Games'



2. Click 'Watch Game'



3. Select the game you want to watch (it is possible the game is not being scored currently. Check in with your friends to see if they are scoring it)



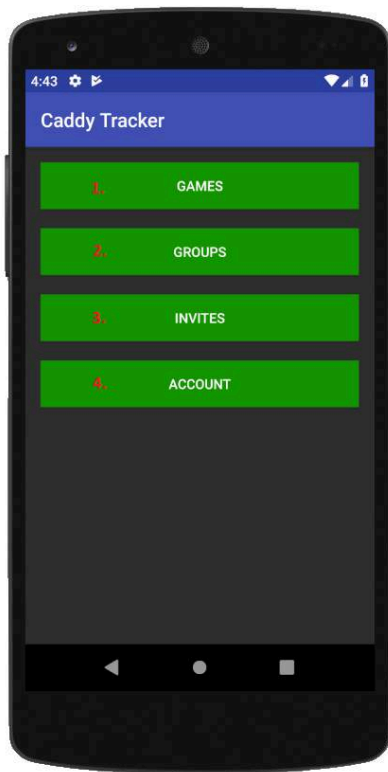
If successful you will be brought to a blank screen. Upon the scorer saving a set of holes for the game, the scorecard will be displayed.



2.14.1.12 Viewing Game Results

Game results is how you can see the full details of a completed game.

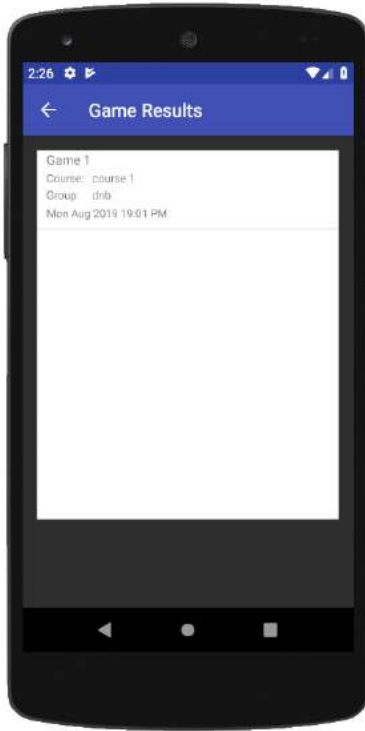
1. Go to the main menu and Click 'Games'



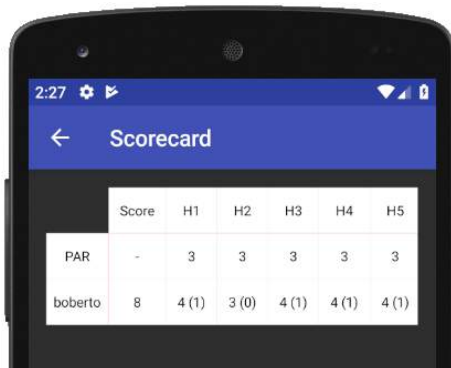
2. Click 'Game Results'



3. Select the game to view results for



If successful, the app will display the scorecard for the selected game:



3 Conclusion

3.1 Enhancements to Network and Security Expertise

I enhanced my experience with networking and security because of this project.

3.1.1 Networking

3.1.1.1 Android

Before this project I had no knowledge of how to implement an Android application that had networking capabilities. Over the course of working on my project, I gained a lot of practice implementing both HTTP requests and communicating with TCP.

The TCP portion of the project was especially valuable as this was a main area focused on during my schooling. My Android project can connect to a server over TCP and communicate with it. To ensure the communication worked, I had to use my knowledge learned in school to design a communication workflow that was agreed upon by both the server and client application so they could talk to each other effectively.

3.1.1.2 C#

Like the android side, I gained a lot of practice working with TCP communication using C#. I built a multi-threaded TCP server that is capable of broadcasting multiple frof games asynchronously.

I also built an API server that can receive and handle HTTP requests that are authorized by OAuth.

3.1.2 Security

I put a lot of effort into developing a set of applications that communicates securely and I am very pleased with the outcome. Entity data to/from the server is encrypted using a combination of AES and RSA encryption. This means that data flowing to the server can only be decrypted by the server and data flowing to the Android client can only be decrypted by the client. Data sent to/from the broadcasting server is also encrypted in the same way as it is for a client to/from the API server.

Another security aspect I gained practice with was how to store passwords as hashed values rather than plaintext. This is a very important part of any app that stores sensitive data. For example, this keeps passwords a secret from people that must inspect the database table data.

A smaller security implementation that I focused on was how users could be invited to groups anonymously. I wanted to separate the data required to invite from any personal information of the user. I believe this help keeps the app users anonymous from other unwanted users. To accomplish this, each user is given a friend code which is a unique 10 character code given at random when a user creates an account. It is up to the user to determine who they give this information to. The friend code also reduces the chance of being invited at random as it is a randomized 10 character string instead of something guessable such as an email.

Hosting wise, I also learned how to set up firewall rules on AWS. This was important as it allowed me to block any attempts to connect my RDS from anything except my EC2 server.

Lastly, I learned how to set up OAuth on my API server in order to authenticate users that make API requests.

3.2 Lessons Learned Above and Beyond the Classroom Teachings

3.2.1 Combined Different Technologies into a Single Solution

I created a complicated full-stack project. A lot of work went into connecting the applications together in a way that was maintainable and efficient.

3.2.2 Hosting Servers

I learned how to deploy servers on AWS. I used a combination of AWS services to get allow public access to my servers. More detail is described in the deployment section.

3.2.3 Use of MVVM pattern for Android

I learned how to use Android's Model-View-ViewModel pattern. It took some time to get used to, especially how to data-bind the UI pieces to observables. This was very helpful because it allowed me to track changes the user made to UI inputs directly on the objects stored on the view models. The view model pattern also helped reduce the amount of additional coding just to get simple things to work which allowed me to focus more on project specific features.

3.2.4 Setup OAuth Authentication

My project uses OAuth to authenticate user's. This is a well-known industry standard. The resources online were helpful when figuring out how to hook it up to an ASP.NET Web API project. It brings a lot of value to my project because it shows I know how to use proper tools to build my project instead of doing everything from scratch.

4 References

4.1.1.1 Using OAuth with ASP.NET Web API

- <https://blogs.msdn.microsoft.com/mvpawardprogram/2017/05/02/adding-webapi-oauth-auth/>
- <https://www.linkedin.com/pulse/aspnet-web-api-oauth-20-token-based-authentication-aamir-sajjad/>
- <https://bitoftech.net/2014/10/27/json-web-token-asp-net-web-api-2-jwt-owin-authorization-server/>

4.1.1.2 API Calls on Android using Retrofit and OkHttp

- <https://square.github.io/retrofit/>
- <https://android.jlelse.eu/rest-api-on-android-made-simple-or-how-i-learned-to-stop-worrying-and-love-the-rxjava-b3c2c949cad4>
- <https://android.jlelse.eu/consuming-rest-api-using-retrofit-library-in-android-ed47aef01ecb>

4.1.1.3 Android MVVM

- <https://developer.android.com/reference/android/arch/lifecycle/ViewModel>